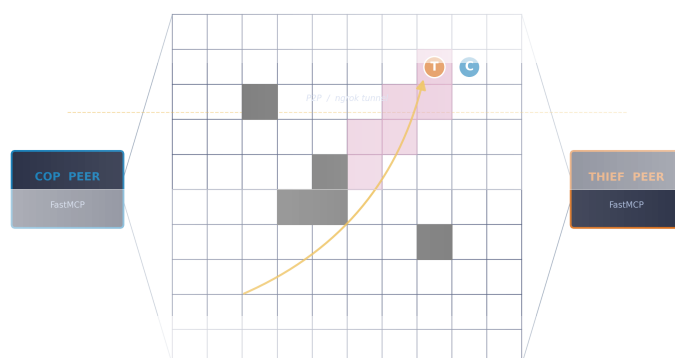


מרוץ שוטר-גנב מבקר בְּרֶשֶׁת עִמִּיתִים

Distributed Cops-and-Robbers over a Peer-to-Peer Network

ספר חוקים והנחיות לפרויקט הגמר 2026 – המחלקה למדעי המחשב, אוניברסיטת חיפה

ד"ר יורם ראובן סגל



כל הזכויות שמורות - © ד"ר יורם ראובן סגל

2026

גרסת ספר 1.0.21 | גרסת קוד לדוגמה 1.11

תקציר

ספר זה הוא מדריך החוקים וההנחיות המלא לפרויקט הגמר בקורס "אורקסטרציה של סוכני AI" (Orchestration of AI Agents) במחלקה למדעי המחשב באוניברסיטת חיפה. הפרויקט מעמיד את הסטודנטים במשימת פיתוח שהיא שיאו של הקורס: בניית שתי ישויות אוטונומיות סימטריות – שוטר וגנג – המתמודדות זו מול זו במרוץ מרדף על גבי לוח בגודל נתון (למשל 10×10), ללא שרת מרכזי המכריע ביניהן. אף אחד מן השניים אינו רואה את מצב העולם האמיתי: כל סוכן בונה, באופן סימטרי, אמונה על מיקום יריבו מתוך מפת ריח דועכת ומרמז מילולי שעשוי להיות כוזב. המערכת ממודלת פורמלית כתהליך החלטה מרקובי מבוזר וניתן לתצפית חלקית (Dec-POMDP), ומתנהלת ברשת עמיתים (Peer-to-Peer) שבה כל סוכן הוא בו-זמנית שרת ולקוח מעל פרוטוקול Model Context Protocol בעזרת ספריית FastMCP.

הספר פורש את מלוא רוחב היריעה של הפיתוח בפרויקט: מידול הבעיה תחת אי-ודאות; ארכיטקטורת ה-P2P והמנהור לאינטרנט הציבורי; מכניקת הלוח, המחסומים ומערכת הניקוד; מנגנון עקבות הריח מבוסס הסטיגמרגיה; פרוטוקול ההתחייבות-חשיפה הקריפטוגרפי (Commit-Reveal) שמבטיח יושרה ללא שופט; מודול האסטרטגיה המשלב למידת חיזוקים, עדכון בייסיאני ומודלי שפה; ממשק המשתמש וסימולטור השחזור; ולבסוף מבנה הליגה, ההוגנות החישובית ואוטומציית הדיווח מעל Gmail API. כל פרק צמוד לעקרונות שנלמדו לאורך הקורס, וממיר אותם מתיאוריה למערכת חיה הפועלת בתנאי רשת אמיתיים.

מילה אישית – לפני שמתחילים במרוץ

אני פונה אליכם, הסטודנטיות והסטודנטים שהגיעו עד הנקודה הזו בקורס: לא במקרה פרויקט הגמר איננו עוד תרגיל בכיתה, אלא מרוץ. לאורך סמסטר שלם בניתם את התשתית – הבנתם כיצד רשת נוירונים לומדת, כיצד Transformer מקשיב, כיצד סוכן בודד מאציל משימות לתת-סוכנים, וכיצד שני סוכנים משוחחים זה עם זה מעל שרת MCP. עכשיו הגיע הרגע לחבר את כל החלקים לכדי ישות אחת, אוטונומית, שיוצאת אל העולם ומתמודדת מול ישות אחרת שאינכם שולטים בה.

המרוץ הזה שונה מכל מה שעשיתם עד כה בנקודה אחת מהותית: **אין שופט**. אין שרת מרכזי ששומר את האמת, מכריע מחלוקות ומגן עליכם מרמאות. במקום זאת, האמת נבנית מלמטה – משני יריבים שאינם סומכים זה על זה, אך מחויבים להוכיח את יושרתם באמצעות מתמטיקה. זהו בדיוק האתגר שמערכות הבינה המלאכותית המבוזרות של העולם האמיתי מתמודדות עימו: תיאום ללא שליט, אמון ללא רשות מרכזית, והחלטה נבונה תחת ערפל מידע.

מדוע חוקים נוקשים דווקא לטובתכם

החוקים בספר הזה הם חוקי ברזל, וזה מכוון. פרוטוקול קריפטוגרפי מדויק, מבנה JSON מחייב, ומגבלות זמן קשיחות אינם באים להקשות עליכם, אלא לְאַפֵּסֶר משחק הוגן בין קבוצות שאינן מכירות זו את זו. ככל שהמפרט חד יותר, כך גדל החופש שלכם לחדש בתוך המסגרת – באסטרטגיה, בהטעיה, ובארכיטקטורה. משמעת בפרטים היא מה שמשחרר יצירתיות בגדול.

אני מזמין אתכם לקרוא ספר זה לא כרשימת דרישות, אלא כמפת דרכים אל מערכת שאתם תהיו גאים בה. קראו לאט, פנו לפי סדר העדיפויות בפיתוח שמוגדר בפרק האחרון, ובדקו כל שלב לפני שאתם ממשיכים הלאה. הניצחון האמיתי איננו רק בלכידת הגנב – הוא ביכולת שלכם לבנות מערכת שעומדת בתנאי רשת אמיתיים, מוכיחה את יושרתה, ומסתגלת לאי-הוודאות. זו המיומנות שתלווה אתכם הרבה מעבר לקורס.

י"ג ט"ו תשפ"ג
f80

הבהרה: מה מחייב ומה רק ממחיש

קראו לפני הכול – עיקרון היסוד

ברירת המחדל היא שאין כלל מחייב, אלא אם נכתב במפורש שהוא כלל מחייב. כל האיורים, הדוגמאות, קטעי הקוד והתרחישים בספר זה הם המחשה של אופן ניהול המשחק – הם אינם מהווים את חוקי המשחק ואינם מחייבים את המשתתפים, אלא אם צוין לצדם במפורש שהם חלק מחוקי המשחק וכובלים את הצדדים. במקום שבו לא נאמר שכלל הוא מחייב – הכלל אינו מחייב, וכל צד רשאי להסכים עם יריבו על התנהגות אחרת, או לפעול כראות עיניו במסגרת החוק. מקור החובה היחיד הוא **טבלת הפרמטרים המחייבת** שבסוף הספר (נספח ה). הערכים המוגדרים שם הם מייצגים מחייבים: מותר להעלותם בהסכמה, אך אסור להנמיכם.

מפתח: שמות-קוד לערכים כמותיים

לאורך הספר, כל ערך כמותי מופיע כשם-קוד עברי הכלוא בסוגריים מרובעים – למשל [גודל הלוח], [מכסת המחסומים] או [קצב דעיכת הריח]. משמעות המוסכמה פשוטה: הערך המספרי בפועל אינו נקבע בגוף הטקסט אלא אך ורק בטבלת הפרמטרים המחייבת שבסוף הספר (נספח ה). כך ניתן לשנות ערך יחיד במקום אחד מבלי שהספר יסתור את עצמו, וכל צד יודע בדיוק מהו הרף המחייב ומהו רק ערך לדוגמה.

הנחיות כלליות לספר ולמבנהו

חופש אקדמי במקרה של סתירה

ספר זה נכתב כמיטב היכולת להיות עקבי, אך ייתכן שתמצאו בו סתירה – שני מקומות שנראים כמכתיבים התנהגות שונה. במקרה כזה נתונה לכם החופש האקדמי לבחור באחת מן האפשרויות ולהמשיך על פיה, **ובלבד שתצינו זאת במפורש** בדוח שלכם: היכן זיהיתם את הסתירה, במה בחרתם, ומדוע. בחירה מנומקת ומתועדת לא תיזקף לחובתכם. עם זאת, מקור האמת המחייב היחיד לערכים הכמותיים נותר **טבלת הפרמטרים המחייבת** שבנספח האחרון של הספר.

מבנה הספר. הספר בנוי מ-11 פרקים ומכמה נספחים. הפרקים פורשים את התאוריה, ארכיטקטורת ה-P2P, הקריפטוגרפיה, מודול האסטרטגיה, הממשק והליגה. הנספחים כוללים מדריכים תפעוליים (OAuth/Gmail, קובץ התצורה, וההגשה ב-GitHub), נספח המתאר מאגר קוד לדוגמה – מימוש סימולציה בסיסי, פתוח וציבורי, שנועד ללימוד בלבד – ולבסוף **טבלת הפרמטרים המחייבת**, הנספח האחרון והמקור המחייב היחיד לערכים המספריים.

מילות מפתח: Dec-POMDP · מערכת רב-סוכנים מבוזרת · סימטריה בין סוכנים · רשת עמיתים (P2P) · Model Context Protocol · FastMCP · מנהור (ngrok) · פרומונים דינמיים ועקבות ריח · זיכרון קולקטיבי של נחיל · Commit-Reveal · SHA-256 · אפס-ידיעה (Zero-Knowledge) · מפת אמונות בייסאנית · מרחק מנהטן · הנדסת פרומפטים ואסטרטגיית LLM · למידת חיזוקים (כלי אופציונלי) · מכונת מצבים · Orchestrator · Watchdog · Gatekeeper · Token Bucket · Gmail API · OAuth 2.0 · הוגנות חישובית · סימולטור שחזור

תוכן העניינים

iii	מילה אישית — לפני שמתחילים במרוץ	
iv	הבהרה: מה מחייב ומה רק ממחיש	
v	הנחיות כלליות לספר ולמבנהו	
1	מסגרת תאורטית ומידול הבעיה	1
1	מטרות הפרק	1.1
1	From Single Agent to Orchestration מסוכן בודד לתזמור מערכת	1.2
2	הפורמליזם: Dec-POMDP	1.3
4	אי-ודאות כמשאב, לא רק כמכשול . Uncertainty as a Resource	1.4
5	סיכום הפרק	1.5
7	ארכיטקטורת רשת מבוזרת (P2P) ותשתית FastMCP	2
7	מטרות הפרק	2.1
	שינוי הפרדיגמה: ביזור מלא של ניהול המצב The Paradigm	2.2
8	Shift to Full Decentralization	
	פרוטוקול MCP ושילוב מודלי שפה The MCP Protocol and LLM	2.3
8	Integration	
11	מנהור והפרדת סביבות Tunneling and Environment Separation	2.4
14	סיכום הפרק	2.5
15	מכניקת הפיזיקה, הלוח ומערכת הניקוד	3
15	מטרות הפרק	3.1

	A Discrete Space and a Shared Contract	3.2
16	Board Dimensions and Start Points	3.3
	Movement, Barriers and Spatial Engineering	3.4
19	Win Conditions and Scoring	3.5
20	סיכום הפרק	3.6
21		
23	עקבות פרומונים דינמיים וזיכרון קולקטיבי של הנחיל	4
23	מטרות הפרק	4.1
24	Indirect Coordination תיאום עקיף דרך שינוי הסביבה	4.2
26	Emission & Decay מודל הפליטה והדעיכה	4.3
28	Scent-Map Tactics השימוש הטקטי במפת הריח	4.4
30	סיכום הפרק	4.5
31	פרוטוקול אבטחה קריפטוגרפי ואפס-ידיעה	5
31	מטרות הפרק	5.1
32	The Temptation to Cheat הפיתוי לרמות ברשת נטולת-שופט	5.2
	Commit-Reveal מעל SHA-256	3.5
33	256	
37	Mutual Audit and Log Integrity ביקורת הדדית ושלמות היומן	5.4
38	Step-0 and Computational Fairness צעד-אפס והוגנות חישובית	5.5
38	סיכום הפרק	5.6
39	מודול האסטרטגיה וקבלת ההחלטות	6
39	מטרות הפרק	6.1
	Why a Separate Strategy מדוע דרוש מודול אסטרטגיה נפרד	6.2
40	Module	
	Reinforcement Learning למידת חיזוקים – כלי אופציונלי אחד	6.3
41	as One Optional Tool	

45	Distance Heuristics and Belief Heatmap	6.4
48	LLM Integration for Prompt Engineering	6.5
49	סיכום הפרק	6.6
51	7 ממשק משתמש (GUI) וסימולטור השחזור	
51	מטרות הפרק	7.1
52	Two Axes: Live Monitoring vs. Retrospective Witness	7.2
52	The Live GUI: Heatmap and Turn Banner	7.3
54	The Replay Viewer and Integrity Enforcement	7.4
56	The Verification Engine: A Code Sketch	7.5
58	סיכום הפרק	7.6
59	8 עיצוב ארכיטקטורת הסוכן ומנגנוני אמינות עמוקים	
59	מטרות הפרק	8.1
59	Separation of Concerns	8.2
60	Orchestrator and State Machine	8.3
62	Reliability Patterns Watchdog ו-Deadline Tracker	8.4
66	סיכום הפרק	8.5
67	9 הליגה, הוגנות חישובית ואוטומציית דיווחים	
67	מטרות הפרק	9.1
67	From Lab to Arena	9.2
69	Gmail API Reporting Automation	9.3
76	סיכום הפרק	9.4

77	סדר עדיפויות מומלץ בפיתוח ותהליך הפיתוח	10
77	מטרות הפרק	10.1
78	מדוע בונים בשכבות Why Build in Layers	10.2
	שבעת שלבי סדר העדיפויות בפיתוח – שבעה קובצי The PRD	10.3
79	Seven Development Priorities	
83	אבני דרך ומשמעת פיתוח Milestones and Development Discipline	10.4
84	סיכום הפרק	10.5
85	סיכום ומבט קדימה	11
85	מטרות הפרק	11.1
	הקשת של הספר: ממידול אי-ודאות אל ליגה חיה The Arc of	11.2
86	the Book	
	הפרויקט כפיתוח מערכות, לא כתרגיל תכנות Systems Devel-	11.3
87	opment, Not a Coding Task	
88	ארבעת מדדי ההצלחה The Four Metrics of Success	11.4
90	רשימת בדיקה אחרונה לפני הגשה Final Pre-Submission Checklist	11.5
92	מבט קדימה: אל עבר AI מבוזר אוטונומי Looking Forward	11.6
93	References	
97	נספח א: מדריך הגדרת Gmail API ו-OAuth 2.0	
97	חמשת שלבי ההגדרה The Five Setup Steps	1
99	אנטומיה של אסימון: Access מול Token Anatomy Refresh	2
100	מימוש: זרימת שליחה מינימלית ב-Python Send-Only Flow	3
101	סיכום הקבצים Required Files	4
103	נספח ב: תצורה אחידה לקובץ ההגדרות	
	מדוע חוקה משותפת? קובץ תצורה בעולם ללא שופט Why a	1
103	Shared Constitution?	
104	הקובץ הקנוני The Canonical Config File	2
105	מילון הפרמטרים Parameter Dictionary	3

107	Where Each Parameter Is Used	4
	מן הדוגמה אל המימוש: סכמת התצורה בפועל - Ex-From	5
107	ample to the Reference Schema	
109	נספח ג: דרישות הגשה ב-GitHub ודוח אקדמי	
	מאגר ה-GitHub: מבנה, ענפים ותיוג Repository, Branches and	1
109	Tagging	
	הדוח	2
	האקדמי:	
111	The Academic README Report README.md	
113	Submission Checklist רשימת התיוג להגשה	3
115	נספח ד: מאגר הקוד לדוגמה – מימוש סימולציה בסיסי	
116	מה המאגר מדגים What the Example Shows	1
116	מבנה הקוד Code Layout	2
117	כיצד מריצים How to Run	3
117	תנאי שימוש How You May Use It	4
119	נספח ה: טבלת הפרמטרים המחייבת	
120	לוח וסוכנים	1
121	תנועה, מחסומים וניקוד	2
121	מנגנון הריח הדינמי	3
122	רשת וליגה	4
122	מגביל-הקצב וההגנה Rate Limiter & Gatekeeper	5

רשימת האיורים

1	תצפית חלקית ב-Dec-POMDP	3
2	מבנה P2P סימטרי	12
3	הזירה הבדידה The discrete arena	18
4	שדה הריח סביב הסוכן	27
5	דעיכת ריח לאורך תורות	28
6	רצף Commit-Reveal	35
7	מודול האסטרטגיה בתוך PeerRuntime	40
8	מפת אמונות Bayesian belief heatmap	47
9	מוק-אפ של הממשק החי של השוטר	54
10	זרימת האימות בסימולטור השחזור	55
11	מכונת המצבים של מהלך המשחק	61
12	המתזמר כשער מרכזי	64
13	צינור ה-Gatekeeper	71

72	מפלס דלי-האסימונים לאורך זמן	14
82	seven stages לפיתוח בת	15

רשימת הטבלאות

1	חלוקת האחריות בין רכיבי הסוכן והאינטגרציה של כל רכיב . . .	9
2	טבלת הניקוד: תנאי הכרעה וחלוקת נקודות	20
3	מיפוי שבעת השלבים (שבעה קובצי PRD) אל פרקי הספר Mapping	
81	the seven PRD stages to book chapters	81
4	מדדי ההצלחה של הפרויקט וקריטריון ההגשה Success metrics	
88	and the submission criterion	88
5	הקבצים הנדרשים לתשתית OAuth, מקורם ורגישותם	101
6	מילון מלא של מפתחות	
106	Full dictionary of config keys config/game.toml	106
7	רשימת תיוג ההגשה	113
8	פרמטרי הלוח ועמדות הפתיחה	120
9	פרמטרי התנועה והניקוד	121
10	פרמטרי הפרומונים הדינמיים	121
11	פרמטרי הרשת והליגה	122

122	פרמטרי מגביל-הקצב (תבנית ה-Gatekeeper)	12
---------------	--	----

1 מסגרת תאורטית ומידול הבעיה

1.1 מטרות הפרק

בסיום פרק זה תדעו: מדוע מרוץ שוטר-גנב מבוזר איננו בעיית תכנון של סוכן בודד אלא בעיית תזמור של מערכת רב-סוכנים; כיצד ממדלים סביבה תחרותית תחת אי-ודאות באמצעות הפורמליזם של Dec-POMDP; ומהי המשמעות היישומית של כל אחד מרכיבי השמינייה הסדורה (tuple) המתמטית, מהמצב ועד למקדם ההיוון.

1.2 מסוכן בודד לתזמור מערכתי - From Single Agent to Orchestration

תחום הבינה המלאכותית המבוזרת (Distributed Artificial Intelligence) מתמודד עם אתגר שבו מספר ישויות אוטונומיות פועלות במרחב משותף, כאשר לכל ישות מידע חלקי בלבד על מצב העולם ועל כוונות רעותה. מהו ההבדל המהותי בין אימון סוכן בודד בסביבה סטטית לבין הפרויקט שלפניכם? בסביבה סטטית, העולם ממתין בסבלנות להחלטת הסוכן. כאן, לעומת זאת, העולם עצמו הוא יריב חושב – הגנב מתכנן, מטעה, ומשנה את פני הלוח בזמן שהשוטר מנסה להסיק היכן הוא.

הפרויקט מעביר אתכם, אם כן, ממוקד של אלגוריתם אל מוקד של מערכת. אין די בכך שסוכן ידע לבחור מהלך טוב; עליו לתזמר תקשורת, לנעול חתימות, לנהל תורות, ולהתאושש מכשלים – כל זאת מול צד שאיננו סומכים עליו ואיננו שולטים בו. זוהי קפיצת המדרגה שהקורס הוביל אליה: מן ההרצאות על סוכן בודד ותת-סוכנים, דרך השיחה בין שני סוכנים מעל MCP, ואל עבר עימות אוטונומי מלא.

1.3 הפורמליזם: Dec-POMDP

הסביבה ממודלת פורמלית כמשחק מבוזר הניתן לתצפית חלקית, הידוע בספרות המחקרית כ-Dec-POMDP (Decentralized Partially Observable Markov Decision Process) [1], [2]. מודל זה מרחיב את ה-POMDP הקלאסי [3] למקרה של מספר מקבלי-החלטה מבוזרים, ומספק תשתית לקבלת החלטות תחת אי-ודאות קריטית. הבעיה מוגדרת באמצעות השמינייה הסדורה המתמטית הבאה:

השמינייה הסדורה המגדירה את מרחב המשחק

$$\langle n, S, \{A_i\}, P, R, \{\Omega_i\}, O, \gamma \rangle$$

שמינייה סדורה (tuple)

שמינייה סדורה היא אוסף מסודר של שמונה רכיבים, שבו לכל רכיב תפקיד ומקום קבועים; כאן היא מגדירה במלואה את מרחב המשחק, ממספר הסוכנים ועד מקדם ההיוון.

המשתנים המגדירים את מרחב המשחק מנותחים כך:

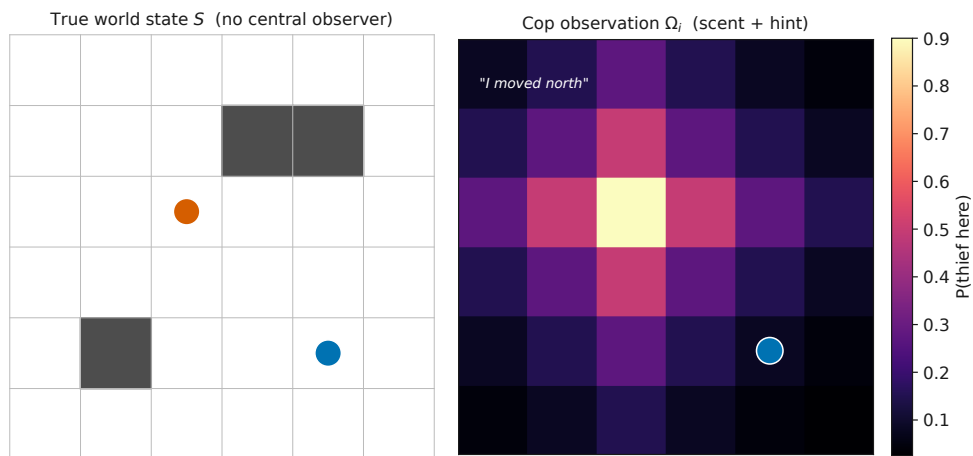
- n – מספר הסוכנים. כאן $n = 2$ (שוטר וגנב). משמעות מעשית: כל החלטה נשקלת ביחס ליריב יחיד ורציונלי, ולא מול טבע אקראי.
- S – מרחב המצבים: תמונת העולם המלאה. מכיל את הקואורדינטות המדויקות של כל סוכן על הגריד, את פריסת המחסומים הסטטיים, ואת רשת עקבות הריח המשתנה דינמית בכל צעד. משמעות מעשית: המצב הוא רב-ממדי, ולכן סריקה ממצה שלו (Brute Force) אינה ישימה – עובדה שתניע את בחירת האלגוריתמים בפרק 6.
- $\{A_i\}$ – מרחב הפעולות: מה שכל סוכן רשאי לעשות. מורכבים מפעולות תנועה פיזיות, פעולות בנייה (הצבת חסמים), ופעולות תקשורתיות (העברת רמזים בשפה טבעית, שעשויים להיות כוזבים). משמעות מעשית: מרחב הפעולה מערב פיזיקה ופסיכולוגיה גם יחד.
- P – פונקציית המעבר: כיצד העולם משתנה בעקבות הפעולות. $P(s' | s, a_1, a_2)$ מגדירה את ההסתברות להגיע למצב חדש בהינתן הפעולות

המשותפות. משמעות מעשית: מכיוון שאין שרת מרכזי, שני הצדדים חייבים להסכים על אותה פונקציית מעבר – היא מקודדת בקובץ התצורה המשותף.

- R – פונקציית התגמול: מה משתלם ומה נקנס. מספקת את התמריץ האלגוריתמי ללמידה. משמעות מעשית: היא מתורגמת ישירות מטבלת הניקוד של פרק 3.

- $O, \{\Omega_i\}$ – מרחב התצפיות: מה שכל סוכן באמת קולט. ליבת אי-הוודאות. אף סוכן אינו רואה את יריבו: הן השוטר והן הגנב ניזונים, כל אחד, מעקבות הריח המתפוגגות של היריב ומהצהרותיו המילוליות. משמעות מעשית: כאן נולד הצורך של כל צד במפת אמונות (Belief) הסתברותית על מיקום יריבו.

- γ – מקדם ההיוון (Discount Factor): כמה חשוב העתיד מול ההווה. $\gamma \in [0, 1]$ קובע את המשקל של תגמול עתידי מול מיידי. משמעות מעשית: γ גבוה מעודד סבלנות אסטרטגית (למשל, בניית מלכודת מחסומים לאורך תורות רבים).



איור 1: מצב העולם האמיתי S (משמאל) אינו נגיש לאף סוכן; כל סוכן בונה את התצפית שלו Ω_i (מימין) על מיקום יריבו, ממפת ריח דועכת ומרמז מילולי – שעשוי להיות כוזב. המערך סימטרי: השוטר והגנב נסתרים זה מזה באותה מידה. The true world state (left) is hidden from every agent; each agent infers a belief (right) about its opponent from a decaying scent field and a possibly-false hint. The setup is symmetric.

מה רואים באיור: בצד שמאל מוצג מצב האמת המלא – מיקומי שני הסוכנים

והמחסומים. בצד ימין מוצגת אותה סצנה כפי שאחד הסוכנים (כאן השוטר) חווה אותה: לא נקודה חדה, אלא ענן הסתברות. **כיצד לפרש:** ככל שהגוון בהיר יותר, כך ההסתברות שהיריב שוהה בתא גבוהה יותר. **סימטריה:** התמונה זהה במהופך עבור הגנב, הבונה בדיוק באותו אופן ענן הסתברות על מיקום השוטר – המערך כולו דו-צדדי. **ניתוח "מה יקרה אם":** אם הרמז המילולי ("נעתי צפונה") סותר את מפת הריח, על הסוכן הקולט להנמיך את מקדם האמון ולעדכן את מפתו – נושא שנפתח בפרק 6.

1.4 אי-ודאות כמשאב, לא רק כמכשול Uncertainty as a Re-source

קל לתפוס את התצפית החלקית כמגבלה בלבד. אך שימו לב לתובנה העמוקה יותר: אותה אי-ודאות שמקשה על השוטר היא גם נשקו של הגנב – ולהפך, שכן המערך סימטרי. היכולת לשלוח רמז מילולי כוזב, לתמרן את היריב, או להיעלם אל מאחורי מחסום – כל אלו הם ניצול אקטיבי של פונקציית התצפית O . חשוב להדגיש: ערוץ ההטעיה היחיד הוא הרמז המילולי. הריח, לעומת זאת, הוא תופעה טבעית שאיננה נתונה לשליטה – סוכן אינו יכול לשתול שובל ריח מטעה במקום שאיננו שוהה בו; כל שביכולתו הוא לחזק את הריח בתא שבו הוא עצמו נמצא, על ידי שהייה בו או חזרה אליו, וזהו מחיר ולא יתרון, שכן הוא מסייע ליריב לאתרו. הפרויקט מלמד אתכם, אפוא, לחשוב על מידע לא כעל נתון קבוע אלא כעל שדה קרב: מי ששולט בזרימת המידע, שולט במרוץ.

חיבור לקורס

הפורמליזם כאן איננו מופשט בלבד. ה- Dec-POMDP קושר יחד שלושה חוטים מהקורס: הרעיון של סוכנים ותת-סוכנים בעבודה מתוזמרת (הרצאה L05), השיחה בין שני סוכנים מעל MCP הקוראים לכלים חיצוניים (הרצאה L09), והתפיסה המבוזרת שבה אין בקרה מרכזית ואף סוכן אינו רואה את התמונה המלאה (הרצאה L11). הפרקים הבאים יפרקו כל רכיב בשמינייה הסדורה למערכת קוד ממשית.

1.5 סיכום הפרק

מידלנו את המרוץ כ-Dec-POMDP: שני סוכנים, מרחב מצבים רב-ממדי, פעולות פיזיות ותקשורתיות, ותצפית חלקית שהיא לב אי-הוודאות. הבנו שהמעבר מסוכן בודד לתזמור מערכתי הוא עיקר האתגר, וכי אי-הוודאות היא בו-זמנית מכשול ומשאב. בפרק הבא נצלול אל התשתית שמאפשרת לשני הסוכנים לתקשר ללא שופט – ארכיטקטורת ה-P2P מעל FastMCP.

2 ארכיטקטורת רשת מבוזרת (P2P) ותשתית FastMCP

2.1 מטרות הפרק

בסיום פרק זה תדעו: מדוע ביזור מלא של ניהול המצב (State Management) מבטל את הצורך בשופט מרכזי ומחליף אותו במשא ומתן קריפטוגרפי בין עמיתים; כיצד פרוטוקול MCP ותשתית FastMCP מאפשרים לכל סוכן להיות שרת ולקוח בו-זמנית; ומדוע חשיפת השרת לאינטרנט הציבורי דרך מנהור (Tunneling) והפרדה מוחלטת של סביבות העבודה אינם המלצה אלא תנאי הכרחי לחוקיות הארכיטקטורה.

2.2 שינוי הפרדיגמה: ביזור מלא של ניהול המצב The Paradigm

Shift to Full Decentralization

בארכיטקטורות משחק מסורתיות שוכן במרכז שרת המשחק (Game Server). הוא מחזיק את אמת הקרקע (Ground Truth), מכריע בסכסוכים, ומעדכן את הלקוחות. אך מה קורה כאשר אנו מסירים את השופט מן הזירה? הפרויקט שלפניכם עושה בדיוק זאת: הוא משלים את שינוי הפרדיגמה אל ביזור מלא של ניהול המצב. אין עוד גורם מרכזי שדברו הוא חוק.

תחת זאת, המשחק רץ מעל רשת עמית-לעמית (Peer-to-Peer, בקיצור P2P), שבה כל סוכן מחזיק את האמת המקומית שלו בלבד. שני הצדדים אינם סומכים זה על זה, ולכן כל מהלך מאומת מול היריב באמצעות משא ומתן קריפטוגרפי. זהו מעבר מהותי מן העולם המרוכז אל עולם מבוזר, שבו אמון אינו מונח כמובן מאליו אלא נבנה, צעד אחר צעד, מתוך חתימות ואימותים. הספרות המקצועית על מערכות מבוזרות מזהירה זה מכבר כי הסרת נקודת שליטה יחידה מעבירה את מרכז הכובד מן החישוב המקומי אל התיאוס בין הרכיבים – וזהו בדיוק האתגר שלפניכם [4].

2.2.1 מדוע לא שרת מרכזי?

שרת מרכזי הוא נקודת כשל יחידה (Single Point of Failure) וגם נקודת אמון יחידה: מי שמחזיק בו יכול, עקרונית, לשנות את תוצאות המשחק. ביזור מסלק את שתי החולשות בבת אחת, אך גובה מחיר – כל סוכן חייב לאמת באופן עצמאי כי היריב לא רימה. כאן נכנס לתמונה פרוטוקול התקשורת.

2.3 פרוטוקול MCP ושילוב מודלי שפה The MCP Protocol and

LLM Integration

התקשורת בין הסוכנים נשענת על (MCP) Model Context Protocol – תקן פתוח המחבר מודלי שפה גדולים (LLMs) אל מקורות נתונים ואל כלים חיצוניים [5], [6]. במימוש שלנו נשתמש בספריית Python בשם FastMCP [7], המפשטת את בניית השרת והלקוח כאחד.

התובנה הארכיטקטונית המרכזית היא של סימטריה: כל סוכן הוא בו-זמנית שרת (Server) – החושף כלים (Tools), כגון קבלת הודעה בשפה טבעית – וגם לקוח (Client) – הקורא לשרת של היריב כדי לשלוח נתונים או להריץ

שאלות. אין כאן צד "חזק" וצד "חלש"; שני העמיתים שקולים לחלוטין בתפקידם ברשת.

כלי (Tool) MCP-ב

כלי הוא פונקציה שהשרת חושף כלפי חוץ, המתוארת בסכמה מובנית כך שהצד הקורא (ואף מודל שפה) יכול להפעילה בבטחה מרחוק. ב-MCP Fast מסמנים פונקציה ככלי באמצעות המעטר (Decorator) `@mcp.tool`.

2.3.1 חלוקת האחריות בין רכיבי הסוכן

ארכיטקטורת הסוכן מתפרקת לשלושה רכיבים בעלי אחריות מובחנת. השרת המקומי מנהל את המשאבים והתקשורת האסינכרונית; מנוע הלקוח מריץ את לוגיקת המשחק וקורא למודל האסטרטגי; ומודל השפה מספק את השכבה הלשונית והפסיכולוגית. הטבלה הבאה מסכמת את החלוקה ואת נקודת האינטגרציה של כל רכיב.

טבלה 1: חלוקת האחריות בין רכיבי הסוכן והאינטגרציה של כל רכיב

רכיב	תחומי אחריות	נקודת האינטגרציה
שרת FastMCP מקומי	ניהול משאבים, חשיפת פעולות ליריב, ועיבוד תגובות אסינכרוני	שימוש במעטרים כגון <code>@mcp.tool</code> לקבלת חתימות קריפטוגרפיות
מנוע הלקוח (Client Engine)	לוגיקת המשחק, קריאה למודל האסטרטגי, ותזמון התורות	התחברות לכתובת ה-URL של היריב והפעלת כליו מעל הרשת
מודל שפה (LLM)	הפקת רמזים בשפה טבעית, פענוח טקסט, והנדסת פרומפט למשחק פסיכולוגי	נגישות דרך API: מקומי (Ollama) או ענני (Claude, Gemini)

הבחנה חשובה: מודל השפה אינו מכריע מהלכים חוקיים – הוא מייצר את השכבה הרטורית והמטעה של המשחק. ההכרעה החוקית נותרת באחריות מנוע הלקוח והאימות הקריפטוגרפי, ולכן רמז מילולי לעולם אינו נאמן כשלעצמו.

2.3.2 שרת FastMCP מינימלי

הקוד שלהלן מדגים את שלד השרת: יצירת מופע FastMCP, חשיפת כלי יחיד המקבל חתימה קריפטוגרפית מן היריב, והרצת השרת. שימו לב כי המעטר @mcp.tool הוא כל שנדרש כדי שהפונקציה receive_move תהפוך לנקודת קצה שהיריב יכול לקרוא לה מרחוק.

דוגמה: שרת FastMCP מינימלי החושף כלי

```
from fastmcp import FastMCP

# Each agent runs its own server instance (local truth)
mcp = FastMCP("police_thief_peer")

@mcp.tool
def receive_move(signed_move: str, signature: str) -> dict:
    """Expose an action to the opponent over the network.

    The opponent (acting as a client) calls this tool to
    submit
    a cryptographically signed move. We verify the signature
    against the shared config before accepting the move.
    """
    is_valid = verify_signature(signed_move, signature)
    # Return an acknowledgement; never trust an unverified
    move
    return {"accepted": is_valid, "move": signed_move if
    is_valid else None}

if __name__ == "__main__":
    # Bind the server so a tunnel can expose it publicly
    mcp.run(transport="http", host="0.0.0.0", port=8000)
```

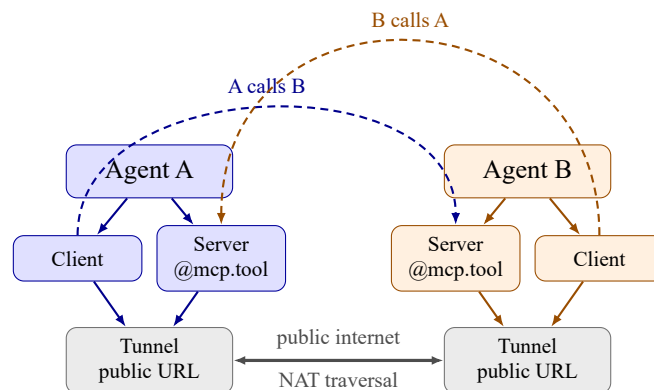
ארכיטקטורה זו היא ההרחבה הישירה של הרצאה L09 בקורס "אורקסטרציה של סוכני AI" ושל תרגיל ex06 שבהם שני סוכני בינה מלאכותית שוחחו זה עם זה מעל פרוטוקול MCP וקראו לכלים חיצוניים (למשל API של Gmail/Google). שם למדתם שסוכן יכול להיות גם שרת וגם לקוח, והפעלתם קריאות כלים חיצוניות; כאן אנו הופכים את השיחה הידידותית לעימות תחרותי מלא, שבו כל "אמירה" של היריב טעונה אימות. אם ב-ex06 המטרה הייתה שיתוף פעולה מבוסס שיחה, בפרויקט זה המטרה היא לנצח יריב שאינו סומך עליכם – ואינכם סומכים עליו.

2.4 מנהור והפרדת סביבות - Tunneling and Environment Separation

כדי שהסוכנים יפעלו כישויות עצמאיות מול קבוצות אחרות ברחבי האינטרנט, הרצת השרתים על localhost מותרת אך ורק בשלבי הקידוד המוקדמים. בפועל, כל קבוצה חייבת לחשוף את שרת ה-FastMCP שלה אל האינטרנט הציבורי באמצעות כלי מנהור, כגון ngrok [8] או Localtonet.

2.4.1 מדוע נדרש מנהור? חציית NAT

רוב המחשבים יושבים מאחורי חומת אש ומאחורי תרגום כתובות רשת (NAT), ולכן אינם נגישים ישירות מן האינטרנט. כלי המנהור מייצר כתובת URL ציבורית העוקפת את חומת האש ומבצעת חציית NAT (NAT Traversal) – בעיה יסודית בתקשורת עמית-לעמית, שפרוטוקולים כגון STUN נועדו לפתור על ידי גילוי הכתובת הציבורית של מארח פרטי [9]. התוצאה המעשית: היריב, בכל מקום בעולם, יכול להתחבר אל השרת שלכם מרחוק דרך אותה כתובת ציבורית.



איור 2: מבנה P2P סימטרי: כל סוכן הוא גם שרת (החושף כלי @mcp.tool) וגם לקוח (הקורא לכלי היריב), ושני הצדדים מחוברים דרך כתובות URL ציבוריות הנוצרות במנהור מעל האינטרנט. A symmetric P2P layout: each agent is both a server exposing an @mcp.tool and a client calling the opponent's tool, connected via public URLs through tunnels.

מה רואים באיור: שני סוכנים זהים במבנה, Agent A ו-Agent B, שלכל אחד רכיב שרת ורכיב לקוח, וכל אחד חשוף לאינטרנט דרך מנהור נפרד. **כיצד לפרש:** החץ הדו-כיווני האמצעי מייצג את הערוץ הציבורי החוצה NAT; החצים המעוקלים מראים כיצד הלקוח של צד אחד קורא לכלי שעל שרת הצד השני – סימטריה מלאה, ללא שרת מרכזי ביניהם. **ניתוח "מה יקרה אם":** אם אחד המנהורים ייפול, הצד שכנגד יאבד את היכולת לאמת מהלכים ויגיע ל"מבוי סתום" (Deadlock) בתזמון התורות – ולכן חוסן המנהור הוא חלק בלתי נפרד מחוסן המשחק עצמו.

2.4.2 הפרדה מוחלטת של סביבות העבודה

מעבר למנהור, הארכיטקטורה דורשת הפרדה מוחלטת של סביבות העבודה. חשוב להבחין בין שני שלבים: בזמן משחק הליגה עצמו שתי הקבוצות ממילא נפרדות באופן אינהרנטי – כל אחת רצה על מחשב אחר, במקום אחר – ולכן ההפרדה בשלב זה מובטחת מאליה. משמעת ההפרדה המתוארת כאן חשובה דווקא בשלב הפיתוח המקומי, כאשר צוות אחד בונה על אותה מכונה גם את השוטר וגם את הגנב; שם הסיכון לחפיפה מקרית (זיכרון או משתנים משותפים) הוא ממשי, והוא עלול להטעות את הפיתוח ולהציג התנהגות שלא תשוחזר כלל בליגה. לכן קוד השוטר וקוד הגנב חייבים לרוץ בשני תהליכים (Processes) נפרדים לחלוטין, תחת ספריות תצורה נפרדות – למשל `config/thief/` מול `config/police/`. כל ניסיון לשתף זיכרון או לקרוא משתנים משותפים אינו רק באג טכני; הוא הפרה של כללי הביזור עצמם, שכן הוא יוצר "דלת אחורית" שדרכה סוכן אחד עשוי לראות את האמת המקומית של יריבו.

כלל הפרדה מחייב

קוד השוטר וקוד הגנב חייבים לרוץ בשני תהליכים נפרדים לחלוטין, תחת ספריות תצורה נפרדות (`config/thief/` מול `config/police/`). **אסור** בתכלית האיסור לשתף זיכרון, לייבא מודול משותף המחזיק מצב חי, או לקרוא משתנים משותפים בין שני הצדדים. שיתוף כזה מעניק לצד אחד גישה ל"אמת המקומית" של רעהו, שובר את מודל האמון-האפס (Zero-Trust) של הארכיטקטורה, ופוסל את הפתרון – גם אם המשחק "עובד" טכנית.

2.5 סיכום הפרק

ראינו כי הפרויקט משלים את ביזור ניהול המצב: אין שרת מרכזי, ובמקומו רשת P2P שבה כל עמית מחזיק אמת מקומית ומאמת מהלכים קריפטוגרפית. פרוטוקול MCP, במימוש FastMCP, הופך כל סוכן לשרת ולקוח בו-זמנית, כאשר מודל השפה מוסיף את השכבה הפסיכולוגית. חשיפת השרת דרך מנהור פותרת את חציית ה-NAT, והפרדת התהליכים והתצורה שומרת על שלמות מודל האמון-האפס. בפרק הבא נעבור מן התשתית התקשורתית אל שכבת האמון עצמה – המנגנונים הקריפטוגרפיים שמאפשרים לאמת מהלך של יריב שאיננו סומכים עליו.

3 מכניקת הפיזיקה, הלוח ומערכת הניקוד

3.1 מטרות הפרק

בסיום פרק זה תדעו: כיצד מרחב גאומטרי בדיד וקבוצת חוקים פשוטה מגדירים זירת עימות שלמה; מדוע הגדלת [גודל הלוח] (ביחס לגרסאות מוקדמות בגודל 5×5) מנפחת את מרחב המצבים באופן אקספוננציאלי ומסכלת סריקה ממצה; כיצד יתרון הצבת המחסומים הופך את השוטר לאדריכל מרחב; וכיצד טבלת ניקוד קומפקטית מתרגמת ניצחון לאות תגמול שאסטרטגיה – היוריסטית, מבוססת-LLM או, אופציונלית, למידת חיזוקים – יכולה למקסם.

3.2 מרחב בדיד וחזקה פיזיקלי משותף A Discrete Space and a Shared Contract

בניגוד לסימולציות פיזיקליות רציפות, המשחק מתרחש במרחב גאומטרי בדיד: רשת תאים סופית שבה כל מיקום, כל מהלך וכל חסימה ניתנים לספירה מדויקת. אך מהיכן נובעים חוקי הפיזיקה כאשר אין שרת מרכזי שאוכף אותם? כאן טמונה החלטת התכן המהותית של הפרויקט: אין שופט חיצוני (no external judge). חוקי הפיזיקה נאכפים על ידי הסוכנים עצמם, כל אחד בתורו, בהתאם לקובץ תצורה מוסכם מראש – `config/game.toml` – המשותף לשני הצדדים בזהות מלאה.

קובץ זה הוא החזקה של המשחק: מידות הלוח, נקודות ההתחלה, מכסת המחסומים וספי הניקוד מקודדים בו כערכים קשיחים. מכיוון ששני הסוכנים טוענים בדיוק את אותו קובץ, שניהם מחשבים את אותה פונקציית מעבר ואת אותם תנאי הכרעה – ובכך נמנעת מחלוקת על "מה חוקי" עוד בטרם החלה. המבנה המלא של הקובץ מפורט בנספח ב.

החזקה נקבע במשא ומתן

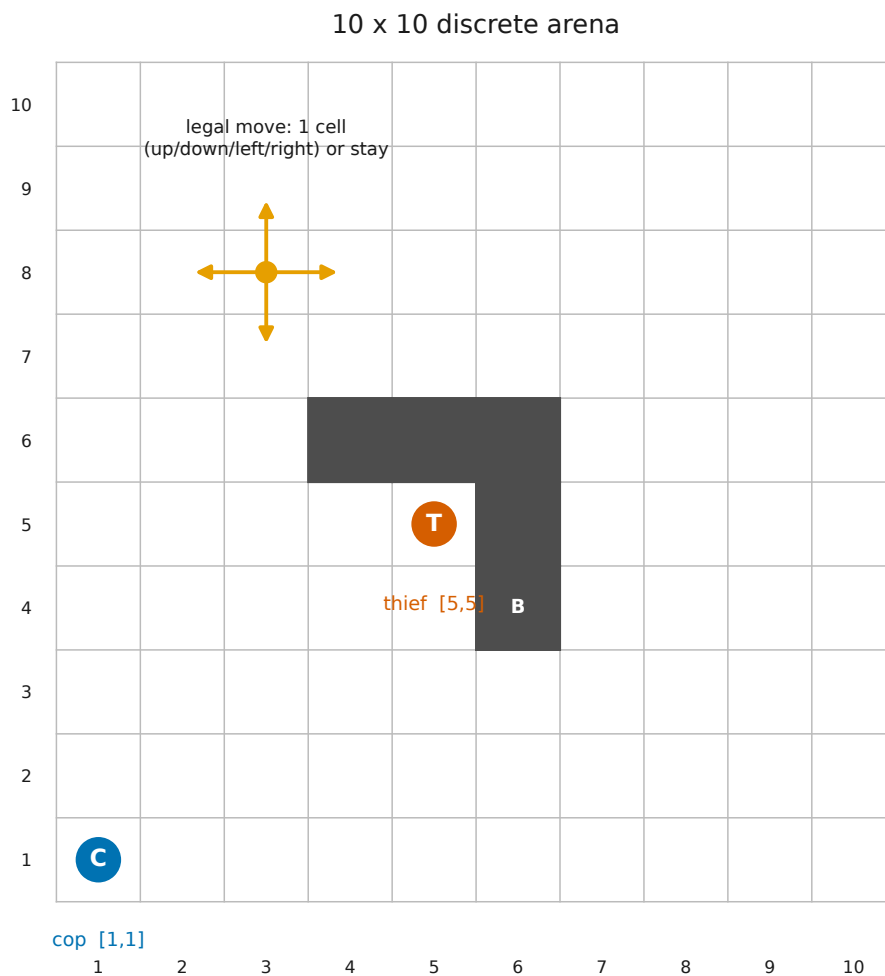
חזקה המשחק אינו מוכתב מלמעלה אלא נקבע במשא ומתן בין כל זוג קבוצות, ולפיכך רשאי להשתנות מזוג לזוג. תנאי הכרחי הוא שהחזקה יהיה מוסכם הדדית על שני הצדדים. עם זאת, אסור לו להחליש או לזלזל את ההוראות המוגדרות בספר זה: החזקה המוסכם הוא רצפה ולא תקרה. מנגד, הקבוצות רשאיות לשדרג את החוקים, ואף חכם לעשות כן – מותר ואף רצוי לנצל באופן חוקי כל פרצה שאיננה מוגדרת כאן, לטובת שני הצדדים או לשם יתרון תחרותי – כל עוד הכול חוקי ומוסכם בין הצדדים.

3.3 מידות הלוח ונקודות הפתיחה Board Dimensions and Start Points

ברירת המחדל של הלוח היא רשת בגודל [גודל הלוח] (למשל 10×10) תאים. הגדלה זו, ביחס לגרסאות מוקדמות שהשתמשו ב- 5×5 , איננה קוסמטית: היא מגדילה את מספר צירופי המצבים האפשריים באופן אקספוננציאלי. מרחב המצבים של Dec-POMDP (ראו פרק 1) גדל כמכפלה של מיקומי שני הסוכנים

ושל כל פריסות המחסומים האפשריות; הכפלת צלע הלוח מעלה את מספר התאים פי ארבעה, ואת מרחב המצבים בסדרי גודל. התוצאה המעשית היא שסריקה ממצה (Brute Force) של כל המצבים הופכת בלתי ישימה חישובית – קושי שהוא מהותי לבעיות מהמחלקה של Dec-POMDP [1] – ומכאן שהצדדים נאלצים לפנות ללמידה ולהיוריסטיקות במקום למניית כלל המצבים.

נקודות הפתיחה אינן אקראיות אלא אסטרטגיות, ואינן קבועות מראש: הן נקבעות בשלב המשא ומתן בין שתי הקבוצות, וכל פריסה חוקית המוסכמת על הצדדים מותרת. הפריסה שבה הגנב (THIEF) ניצב במרכז הלוח והשוטר (COP) בפינה היא זוגיה בלבד: הגנב במרכז נהנה ממספר מרבי של נתיבי מילוט לכל הכיוונים, ואילו השוטר מוצב במרחק אסטרטגי מוגדר. עמדות אלה מתועדות ב-[עמדת פתיחה – גנב] וב-[עמדת פתיחה – שוטר] (למשל [5,5] לגנב ו-[1,1] לשוטר), נטענות מקובץ `config/game.toml`, וערכיהן המדויקים מרוכזים בטבלת הפרמטרים בנספח ה. כך ניתן לשנות את מאזן הכוחות ההתחלתי, בהתאם למוסכם בין הצדדים, מבלי לגעת בקוד הסוכנים.



איור 3: לוח בגודל [גודל הלוח] (בדוגמה 10×10): בפריסה לדוגמה הגנב (T) ניצב במרכז $[5,5]$, השוטר (C) בפניה $[1,1]$, ומספר מחסומים (B) שהציב השוטר אוטומים תאים. החץ הכתום ממחיש את קבוצת המהלכים החוקית – תא אחד לכל אחד מארבעת הכיוונים האורתוגונליים, או עמידה במקום. A 10x10 board: the thief (T) at the center, the cop (C) in a corner, cop-placed barriers (B), and an arrow showing the legal orthogonal move set.

מה רואים באיור: רשת בת מאה תאים, שני הסוכנים בעמדות הפתיחה שלהם, שרשרת מחסומים אחת שהחלה להיסגר, וכוכב כתום שממנו יוצאים ארבעה חצים אל התאים הצמודים. **כיצד לפרש:** החצים מגדירים בדיוק אילו מעברים חוקיים מכל תא – צפון, דרום, מזרח, מערב, או השהיה; אין אלכסונים. המחסומים (B) הם תאים שחורים שאיש אינו יכול לחצות. **ניתוח "מה יקרה אם":** אילו הלוח היה 5×5 , מרכז הגנב ופינת השוטר היו במרחק של שני צעדים

בלבד, והמרדף היה מוכרע כמעטמיד; דווקא ההתרחבות אל [גודל הלוח] הנוכחי היא שיוצרת את המרחב הדרוש למרדף ארוך, ללמידה ולתמרון.

3.4 תנועה, מחסומים והנדסת מרחב Movement, Barriers and Spatial Engineering

בכל תור רשאי סוכן לבצע מהלך יחיד: לנוע תא אחד באחד מארבעת הכיוונים האורתוגונליים (מעלה, מטה, שמאלה, ימינה), או לבחור להישאר במקומו. תנועה אלכסונית אסורה. אילוץ פשוט זה הוא שמעניק למרדף את אופיו הרשתי, וקושר אותו ישירות למשפחת בעיות ה"שוטרים ושוודדים" ולגרסת המרדף-בריחה על גרפים שנחקרו במתמטיקה של תורת הגרפים [10], [11].

לשוטר עומד יתרון א-סימטרי בהנדסת מרחב: בתור שבו הוא מוותר על תנועה, הוא רשאי להציב מחסום פיזי בכל תא שבמרחק צעד אחד ממנו – התא שבו הוא ניצב עצמו או אחד מארבעת התאים הצמודים האורתוגונליים. יכולת זו הופכת אותו ממרדף פסיבי לאדריכל של הזירה.

חוק המחסום

בתור שבו השוטר מוותר על תנועה הוא רשאי להציב מחסום בכל תא שבמרחק צעד אחד ממנו – התא שבו הוא עומד או אחד מארבעת התאים הצמודים האורתוגונליים – והתא ההוא הופך לבלתי עביר עבור שני השחקנים עד תום המשחק. למחסום אין הפיכות: תא שנחסם נותר חסום. **הצבה כלוכדת:** אם השוטר מציב מחסום על התא שבו ניצב הגנב ברגע זה – הגנב נלכד. בדומה, גנב שנכלא ללא מהלך חוקי כלשהו (כל התאים הצמודים חסומים במחסומים ו/או בשולי הלוח) נחשב אף הוא ללכוד. **חובת הצהרה:** על השוטר להכריז באמת על כל הצבת מחסום ועל מיקומה המדויק; אין להציב מחסום בהיחבא, ואסור לשוטר לשקר לגבי מיקומו. מכסת המחסומים המרבית של השוטר היא [מכסת המחסומים], ולפיכך כל הצבה היא החלטת ניהול-משאבים: על השוטר "לסחוט" את הגנב אל פינה מבלי לחסום בטעות את נתיבי הגישה של עצמו.

[מכסת המחסומים] היא לב האתגר האסטרטגי של השוטר. מחסום שהוצב באופן חמדני עלול לכלוא את השוטר עצמו מאחורי קיר שבנה, או לפתוח לגנב

פרצת מילוט חדשה. ניהול המשאב הזה – מתי לחסום, היכן, וכמה מחסומים לשמור לשלב הסגירה – הוא בעיה אסטרטגית בפני עצמה, הנדונה בהרחבה בפרק 6.

חוקי ברזל: תנועה והצהרת אמת

אין אלכסונים. מהלך אלכסוני איננו חוקי; ניסיון לבצעו נדחה על ידי הסוכן היריב האוכף את הפיזיקה. **חובת אמת בתפיסה.** כאשר השוטר מכריז (Capture Claim), מוטלת על הגנב חובה קריפטוגרפית להשיב אמת (פרוטוקול Capture). ניסיון לשקר בשלב זה יתגלה בהכרח בשלב ביקורת היומן (log-audit) ויגרור פסילה מערכתית מוחלטת. **הצהרת מחסום גלויה.** על השוטר להכריז באמת על כל הצבת מחסום ועל מיקומה המדויק; אין להציב מחסום בהיחבא ואסור לשקר לגבי מיקומו.

3.5 תנאי הכרעה וטבלת הניקוד Win Conditions and Scoring

מערכת הניקוד מאזנת בין שני מתחים מנוגדים: הקושי של השוטר לאתר שחקן נסתר, מול הקושי של הגנב לשרוד בסביבה עוינת ההולכת ונסגרת. במקום ניצחון בינארי, כל תרחיש סיום מזכה כל צד בניקוד שונה, ובכך מקודד את הערך של כל תוצאה – תרגום שממנו נגזרת ישירות פונקציית התגמול R של הפרק הקודם.

טבלה 2: טבלת הניקוד: תנאי הכרעה וחלוקת נקודות

אירוע סיום	תנאי הכרעה	ניקוד לשוטר	ניקוד לגנב
תפיסה מוצלחת	השוטר נוחת על תא הגנב ומכריז Capture Claim	[ניקוד לכידה – שוטר]	[ניקוד לכידה – גנב]
הישרדות ממושכת	הגנב שורד [סף ההישרדות] צעדים תקפים ללא תפיסה	[ניקוד הישרדות – שוטר]	[ניקוד הישרדות – גנב]
הפסד טכני	צד קורס, חורג מזמן, או מבצע זיוף קריפטוגרפי	0	0

שימו לב לסימטריה השבורה שבטבלה. תפיסה מזכה את השוטר בתגמול

הגבוה ביותר ([ניקוד לכידה – שוטר]), ומגלמת את מטרתו העיקרית; אך הישרדות ממושכת – סבלנות בזמן של [סף ההישרדות] צעדים תקפים ללא תפיסה – מזכה את הגנב בתגמול הגבוה ביותר שלו ([ניקוד הישרדות – גנב]). ההפסד הטכני מאפס את שני הצדדים כאחד, ובכך מתמרץ את שניהם לשמור על תקינות פרוטוקולרית ולא לנצח "בפסק זמן".

בעת הכרזת תפיסה, כאמור, מוטלת על הגנב חובה קריפטוגרפית להשיב אמת. הכרזת תפיסה איננה, אפוא, שאלה של אמון בין יריבים אלא של הוכחה הניתנת לאימות בדיעבד: כל תשובה נחתמת ונרשמת ביומן, וכל ניסיון להתכחש למצב אמיתי יתגלה בשלב ביקורת היומן ויוביל לפסילה. כך הופך הניקוד עצמו מהצהרה שבידי היריב לעובדה הניתנת לאכיפה מתמטית.

חיבור לקורס

חוקים פשוטים אלה – רשת סופית, מהלך אורתוגונלי יחיד, מכסת מחסומים ידועה, ואות תגמול חד-משמעי (טבלת הניקוד) – מגדירים משחק מבוזר וסופי שבו שני סוכנים מתאמים את פעולותיהם ללא שרת מרכזי ובלא שופט. זהו בדיוק המרחב שנדון בקורס "אורקסטרה של סוכני AI": בשיעור L09 ראינו שני סוכנים המשווחים מעל MCP וקוראים לכלים חיצוניים, ובשיעור L11 ראינו נחיל סוכנים מבוזר המתאם עצמו בלא בקרה מרכזית. את השאלה כיצד ממירים את טבלת הניקוד לאסטרטגיית משחק – באמצעות היוריסטיקות, אסטרטגיה מבוססת-LLM, או, כאפשרות אחת בלבד, למידת חיזוקים – נפתח בפרק פרק 6.

3.6 סיכום הפרק

הגדרנו את הזירה הפיזיקלית של המרדף: מרחב בדיד של [גודל הלוח] תאים, שבו חוקי הפיזיקה נאכפים על ידי הסוכנים עצמם לפי חוזה תצורה משותף הנקבע במשא ומתן בין הצדדים. ראינו כי ההתרחבות מגריד 5×5 מוקדם מנפחת את מרחב המצבים ומסכלת סריקה ממצה, כי [מכסת המחסומים] הופכת את השוטר לאדריכל מרחב תחת אילוץ ניהול-משאבים, וכי טבלת ניקוד א-סימטרית מתרגמת כל תרחיש סיום לאות תגמול הניתן למקסום. בפרק הבא נעבור מן החוקים הסטטיים אל האסטרטגיות הדינמיות שהסוכנים מפעילים כדי לנצח בזירה זו.

4 עקבות פרומונים דינמיים וזיכרון קולקטיבי של הנחיל

4.1 מטרות הפרק

בסיום פרק זה תדעו כיצד מנגנון ביולוגי פשוט – פיזור עקבות ריח והתפוגגותן – פותר, ולו חלקית, את בעיית התצפית החלקית שהוצגה בפרק 1. תבינו מהי סטיגמרגיה (Stigmergy) ומדוע היא מהווה מנגנון תיאום עקיף בין סוכנים; תשלטו במודל המתמטי של פליטה ודעיכה של עוצמת הריח בכל תא; ותראו כיצד כל סוכן ממנף את מפת הריח ההיסטורית של יריבו כדי לחשוף רמזים מילוליים כוזבים ולתחזק מפת אמונות הסתברותית.

4.2 תיאום עקיף דרך שינוי הסביבה Indirect Coordination

כיצד מתאמים ביניהם מיליוני נמלים ללא מפקד מרכזי, ללא שפה, וללא זיכרון משותף? התשובה, שגילו חוקרי התנהגות בעלי-חיים, איננה טמונה בנמלים עצמן אלא בסביבה. כל נמלה מותירה אחריה עקבות של פרומונים, וכל נמלה אחרת מגיבה להם. הסביבה עצמה הופכת ללוח המחוונים המשותף – מנגנון שכונה Stigmergy, תיאום עקיף באמצעות שינוי הסביבה [12], [13]. עיקרון זה, שעמד בבסיס אלגוריתם מושבת הנמלים המפורסם [14], הוא בדיוק הכלי שנרתום כאן כדי לתקוף את אי-הוודאות.

אחת התרומות המרכזיות לפתרון בעיית התצפית החלקית בפרויקט שלנו היא, אם כן, מנגנון עקבות הריח – מנגנון שקיבל השראה ישירה מהתנהגות הנמלים. הרעיון פשוט וחזק כאחד, והוא סימטרי לחלוטין: כאשר סוכן נע על הלוח – הגנב והשוטר כאחד – הוא מפזר אחריו פרומונים וירטואליים הדועכים עם הזמן. אף סוכן אינו רואה בכך תקשורת מכוונת; אך יריבו, הקורא את הסביבה, הופך את השובל הפיזי הזה למקור מידע יקר ערך. הריח טבעי ובלתי-נשלט: הוא נפלט מעצם התנועה או השהייה, ואיש אינו יכול לשתול שובל מטעה – כל צד פולט את ריחו שלו, וכל צד קורא את שדה הריח של יריבו בלבד.

4.2.1 מה נלמד בהרצאה L11: פרמונים דינמיים וזיכרון הנחיל

המנגנון המתואר כאן הוא בדיוק מה שנלמד בהרצאה L11 של הקורס אורקסטרציה של סוכני AI. במקום בקרה מרכזית, נחיל הסוכנים מתאם את עצמו דרך עקבות פרמונים דינמיים המקודדות את היעילות ההקשרית של כל פעולה: פעולה מוצלחת מעדכנת את הייצוג המשותף (ה-embedding) של הנחיל, ובכך מעלה את ההסתברות לבחור בנתיבים מוצלחים בסבבים הבאים – זהו זיכרון קולקטיבי הנחקק בסביבה ולא בראשו של סוכן יחיד. לצד זאת פועל מנגנון דעיכה/התפוגגות (fading) המונע קיבעון: בלעדיו היה הנחיל ננעל על אופטימום מקומי, שכן עקבות ישנות היו מצטברות לנצח ומשתקות כל חקירה של נתיבים חדשים. שדה הריח וכלל הדעיכה שנגדיר מיד הם התרגום הישיר של רעיון זה לזירת השוטר והגנב.

חיבור לקורס

בהרצאה L11 של הקורס אורקסטרציה של סוכני AI ראינו נחיל של סוכנים המתאם את עצמו ללא בקרה מרכזית, באמצעות עקבות פרמונים דינמיים וזיכרון קולקטיבי של הנחיל (בסגנון SwarmSys): הצלחות מעדכנות ייצוג משותף, ומנגנון דעיכה מונע היתקעות באופטימום מקומי. מנגנון עקבות הריח בפרויקט זה הוא היישום הישיר של אותו רעיון – תיאום עקיף וא-סינכרוני, שבו הודעה אינה נשלחת לאיש אלא נחקקת בסביבה המשותפת וממתינה שם למי שיִדע לקרוא אותה. הבנת קוטב זה, לצד התיאום הישיר שראינו בהרצאות הקודמות, חיונית לכל מי שמעצב מערכות של סוכנים אוטונומיים.

4.3 מודל הפליטה והדעיכה Emission & Decay

בכל פעם שסוכן נע או נותר במקומו, נוצר סביב מיקומו שדה ריח בגודל [גודל שדה הריח] (למשל 5×5). במרכז הפליטה – התא שבו שוהה הסוכן – עוצמת הריח נקבעת ל[עוצמת הריח במוקד]. ככל שמתרחקים מן המרכז, העוצמה יורדת לפי התפלגות רדיאלית: תאים קרובים סופגים עוצמה גבוהה, ותאים בשולי שדה הריח סופגים שארית דהויה בלבד. בכך מתקבל טביעת ריח מרוכזת, המסמנת את סביבת הסוכן ולא נקודה בודדת בלבד. הדבר תקף לשני הצדדים כאחד – גם השוטר וגם הגנב מותירים שדה ריח משלהם.

בתום כל תור מלא – כלומר לאחר שהשוטר וגם הגנב השלימו את מהלכם – עוברים כל עקבות הריח הקיימים על הלוח תהליך של דעיכה מערכתית (Decay). קצב הדעיכה הוא [קצב דעיכת הריח] לכל תור. העדכון המתמטי של עוצמת הריח בתא (i, j) נתון בנוסחה הבאה:

עדכון עוצמת הריח בתא

$$\tau_{ij}(t+1) = \max\left(0, (1-\rho) \cdot \tau_{ij}(t) + \Delta\tau_{ij}\right)$$

המשתנים המרכיבים את הנוסחה מנותחים כך:

- $\tau_{ij}(t)$ – **עוצמת הריח בתא בזמן הנוכחי**. ערך רציף בתחום $[0, 0.9]$ המבטא כמה "טרי" הוא השובל בתא (i, j) . משמעות מעשית: זהו למעשה ציון ודאות מקומי – ערך גבוה מרמז שהסוכן שאת שדהו אנו קוראים עבר כאן לא מכבר.

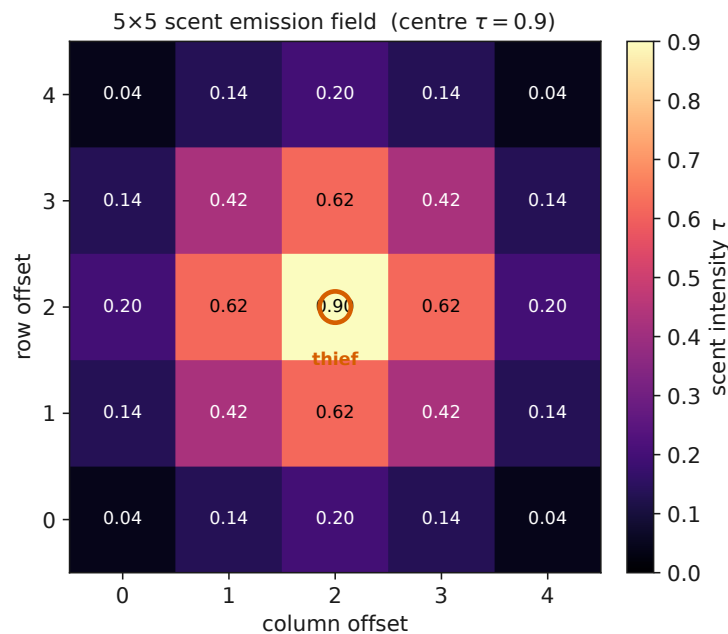
- ρ – **קצב הדעיכה (Decay Rate)**. כאן $\rho = 0.10$. הגורם $(1-\rho)$ מכווץ בכל תור את הריח הקיים ל-90% מערכו. משמעות מעשית: דעיכה איטית זו היא בחירה תכנונית מכוונת – היא מותירה שובל היסטורי מספיק ארוך כדי שיהיה שמיש טקטית, אך לא נצחי.

- $\Delta\tau_{ij}$ – **הפליטה החדשה**. תוספת העוצמה בתא בתור הנוכחי, הנקבעת לפי הקרבה הרדיאלית של התא למרכז הפליטה של הסוכן (ובמרכז עצמו $\Delta\tau = 0.9$). אם הסוכן רחוק, $\Delta\tau_{ij} = 0$. משמעות מעשית: זהו הרכיב המחבר בין נוכחות הסוכן לבין הסביבה – הוא "כותב" אל הלוח.

- $\max(0, \cdot)$ – **קטימה לאפס**. מבטיחה שעוצמת הריח לעולם אינה שלילית.

משמעות מעשית: תא שמעולם לא ספג ריח, או שדעך לחלוטין, פשוט "שקט" – נעדר מידע ולא מידע שלילי.

הנוסחה מגלמת מתח יפה בין שני כוחות: הרכיב $(1-\rho) \cdot \tau_{ij}(t)$ הוא השכחה – המחיקה ההדרגתית של העבר; והרכיב $\Delta \tau_{ij}$ הוא הזיכרון – חקיקת ההווה. שיווי המשקל ביניהם הוא שקובע כמה עמוק בעבר יכול כל סוכן להביט אל שובל יריבו.



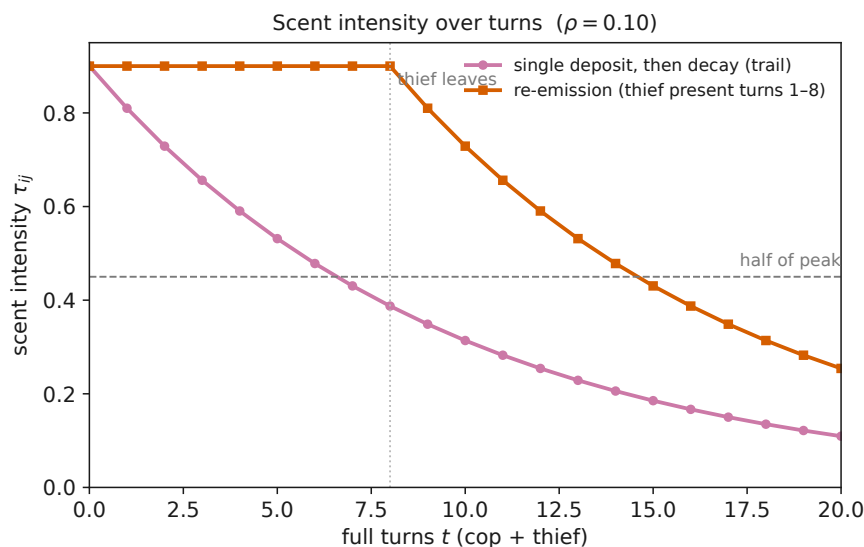
איור 4: שדה הפליטה בגודל [גודל שדה הריח] סביב הסוכן (הגנב או השוטר):
 במרכז $\tau = 0.9$, והעוצמה דועכת רדיאלית עם המרחק מן המרכז. Radial scent field around an agent.

מה רואים באיור: מטריצת 5×5 של תאים, כאשר התא המרכזי – מיקום הסוכן הפולט – צבוע בבהיר ביותר ונושא את הערך 0.90, והתאים סביבו מתכהים ככל שהם מתרחקים. **כיצד לפרש:** הבהירות מייצגת את עוצמת הריח τ ; הנפילה הרדיאלית פירושה שהריח איננו "כתם" אחיד אלא גבעה חלקה שראשה במרכז. תא בפינת הריבוע מקבל שארית זעומה בלבד, ולכן השוטר ייחס לו ודאות נמוכה. **ניתוח "מה יקרה אם":** אילו הפליטה הייתה נקודתית (תא בודד בעוצמה 0.9 ואפס סביבו), רעש מדידה קטן היה מוחק את כל האות; ההתפלגות הרדיאלית מעניקה למנגנון עמידות, שכן גם אם התא המדויק פוספס, שכניו עדיין מסמנים את הכיוון.

4.4 השימוש הטקטי במפת הריח Scent-Map Tactics

מכיוון שהריח דועך לאט, הוא מותיר אחריו "שובל ריח" (Scent Trail) היסטורי – לא צילום רגע, אלא סרט קצר של תנועת הסוכן בתורות האחרונים. כל סוכן יכול לדגום את הלוח ולקבל את מפת הריח של יריבו. כאן מתחוללת קפיצת המדרגה: הצלבת מפה זו עם ההצהרות המילוליות של אותו יריב מאפשרת מיזול אמונות (Belief Modeling) – היכולת לתחזק התפלגות הסתברותית על מיקומו האמיתי של היריב [15]. הסימטריה מלאה: השוטר קורא את שובל הגנב, והגנב קורא את שובל השוטר – כל צד מצליב את מפת הריח של יריבו מול הרמזים המילוליים שאותו יריב מספק.

האיור הבא ממחיש מדוע שובל כזה שורד די זמן כדי להיות שמיש.



איור 5: התפתחות עוצמת הריח τ_{ij} לאורך תורות עבור $\rho = 0.10$: פליטה בודדת ולאחריה דעיכה טהורה (השובל) לעומת פליטה חוזרת בעת שהיית הסוכן. Single deposit decay vs. re-emission.

מה רואים באיור: שני עקומים. האחד ("פליטה בודדת") יורד באופן מעריכי מ-0.9 – זהו תא שהסוכן עבר בו פעם אחת ונטש. השני ("פליטה חוזרת") נותר גבוה כל עוד הסוכן שוהה בסביבה (תורות 1-8), ורק לאחר עזיבתו מתחיל לדעוך. **כיצד לפרש:** קו האמצע המקווקו מסמן את מחצית עוצמת השיא; ניתן לראות שהשובל הבודד חוצה אותו רק סביב התור השביעי – כלומר הריח נותר "קריא" במשך כשישה עד שבעה תורות. **ניתוח "מה יקרה אם":** אילו

הכפלנו את ρ ל-0.20, העקום היה צולל מהר בהרבה, השובל היה מתקצר, והיריב הרודף היה מאבד את זיכרון העבר; אילו הקטנו את ρ כמעט לאפס, הלוח היה מתמלא בריח נצחי ומאבד את יכולתו להבחין בין ישן לחדש.

גילוי שקר: הגנב "נע צפונה" בעוד הריח בדרום-מזרח

נניח שהשוטר דוגם את הלוח ומקבל את מפת הריח הבאה, המרוכזת בפינה הדרום-מזרחית:

- תא דרום-מזרחי (1, 4): $\tau = 0.81$ (שובל טרי מאוד).
- תא סמוך (1, 3): $\tau = 0.63$.
- כל התאים בצפון הלוח, למשל (5, 2): $\tau = 0.00$ – ריק לחלוטין מריח.

כעת מגיעה ההצהרה המילולית של הגנב: "נעתי צפונה". נבחן את הטענה כמותית. אילו נע הגנב צפונה בתור האחרון, היינו מצפים למצוא בצפון עקבה טרייה בעוצמה של כ- $0.81 \approx 0.9 \cdot 0.9 = 0.9 \cdot (1 - \rho)$. במקום זאת אנו מודדים בצפון $\tau = 0.00$. הפער בין הצפוי (≈ 0.81) לנמדד (0.00) הוא מוחלט: אין שום שארית ריח שתתמוך בטענה, בעוד שכל המסה הריחנית מרוכזת בקוטב ההפוך של הלוח.

השוטר מסיק אפוא, בביטחון גבוה, שהגנב משקר. הוא מנמיך את מקדם האמון שהוא מייחס להצהרות המילוליות, מעדכן את מטריצת ההסתברות שלו כך שמשקל רב יינתן לתאי הדרום-מזרח, ומכוון מחדש את וקטור הרדיפה – לא לעבר צפון המוצהר, אלא לעבר מקור הריח האמיתי. כך הופכת המניפולציה של הגנב לחרב פיפיות: עצם הניסיון להטעות, כשהוא נסתר בעדות הסביבה, מסגיר את מיקומו.

חשוב להדגיש: מפת הריח אינה יכולה לשקר – היא נפלטת מעצם התנועה ואינה ניתנת לזיוף. מה שנחשף כאן הוא רמז מילולי כוזב, שנתפס דווקא משום שהסביבה האמינה סתרה אותו – ולא "שובל שקרי" (שכזה אינו קיים). והסימטריה מלאה: אותו הליך עצמו זמין לגנב, המצליב את שובל הריח של השוטר מול הרמזים שהשוטר מוסר – כל צד מגן על עצמו בכך שהוא מאמת את דברי יריבו מול העדות הבלתי-ניתנת-לזיוף שהותיר בסביבה.

עדכון מטריצת ההסתברות המלא – כיצד בדיוק מתרגם כל סוכן את מפת הריח של יריבו ואת הצהרתו לכדי מפת אמונות מספרית, וכיצד הוא משלב

את שתי העדויות בכלל בייסיאני – יפורט בפרק פרק 6, שם נבנה את מנגנון האמונות (Belief) על יסודות הפרק הנוכחי.

4.5 סיכום הפרק

ראינו כיצד עיקרון ביולוגי – הסטיגמרגיה של הנמלים – מתורגם למנגנון יישומי המקל על בעיית התצפית החלקית. הגדרנו את מודל הפליטה בגודל [גודל שדה הריח] עם [עוצמת הריח במוקד] והתפלגות רדיאלית, ואת כלל הדעיכה $\tau_{ij}(t+1) = \max(0, (1-\rho)\tau_{ij}(t) + \Delta\tau_{ij})$ עם $\rho = 0.10$. הדגשנו שהמנגנון סימטרי – שני הסוכנים פולטים ריח וכל אחד קורא את שובל יריבו – ושהריח טבעי ובלתי-ניתן-לזיוף. הראינו שדעיכה איטית זו מייצרת שובל היסטורי בן כשישה תורות, ושהצלבתו עם ההצהרות המילוליות של היריב מאפשרת לכל סוכן לחשוף רמזים כוזבים. בפרק הבא נעבור מן העקבה שכל סוכן מותיר בסביבה אל הערוץ שבו הוא מדבר במפורש – ונבחן כיצד נבנית התקשורת המילולית עצמה, על אמינותה המפוקפקת.

נעילה קריפטוגרפית של מודל הפליטה והדעיכה לפני סדרה

לפני שהסדרה נפתחת, על שתי הקבוצות להחליף ביניהן את מודל הפליטה והדעיכה במלואו – כולל דוגמה מספרית קונקרטית (למשל: תא במרכז מקבל $\tau = 0.9$, ולאחר תור אחד של דעיכה בקצב ρ מתקבל $0.9 \cdot (1 - \rho)$). על הצדדים לאמת שהם מפרשים את אותה נוסחה בדיוק באותו אופן, ורק לאחר מכן לעול את ההסכמה קריפטוגרפית – למשל באמצעות hash (SHA-256) של הנוסחה המוסכמת יחד עם הדוגמה המספרית. כך כל סטייה עתידית בהתנהגות המנגנון תתגלה מיד. **מותר ואף מומלץ** שקבוצה אחת תספק לרעותה את קוד מנגנון הריח המשותף עצמו, כדי שיובטח ששני הצדדים מריצים בדיוק את אותה התנהגות – ולא נותר שום מרווח לפרשנות שתפגע בהוגנות הסדרה.

5 פרוטוקול אבטחה קריפטוגרפי ואפס-ידיעה

5.1 מטרות הפרק

בסיום פרק זה תדעו: מדוע רשת עמית-לעמית ללא מנהל משחק אובייקטיבי סובלת מפיתוי מובנה לרמאות; כיצד מנגנון Commit-Reveal המבוסס על פונקציות גיבוב (Hash) הופך את הרמאות לבלתי-אפשרית מבחינה מעשית; כיצד ביקורת הדדית של יומני המשחק מגלה כל זיוף בדיעבד; וכיצד הצהרת חומרה חתומה ב"צעד-אפס" מבטיחה הוגנות חישובית בין מתחרים בעלי מכונות שונות בתכלית.

5.2 הפיתוי לרמות ברשת נטולת-שופט The Temptation to Cheat

דמיינו משחק שחמט שבו אין לוח פיזי משותף ואין שופט המשגיח על הכללים; כל שחקן מנהל עותק פרטי של הלוח ומדווח לרעהו על מהלכיו. במערכת מבוזרת מסוג זה – רשת עמית-לעמית (P2P) שבה שני הסוכנים מדברים ישירות זה עם זה מעל שרת FastMCP, בלא מנהל משחק אובייקטיבי – נולד פיתוי מובנה לרמאות. שלושה סוגי הונאה מאיימים על שלמות המרוץ: מסע בזמן – שינוי של מהלך שכבר בוצע; שינוי מהלך לאחר שנחשף מהלך היריב; והתכחשות למיקום או להצהרה קודמים. כל עוד כל צד הוא גם השחקן וגם רושם-הפרוטוקול של עצמו, אין דבר המונע ממנו לכתוב מחדש את ההיסטוריה לטובתו.

הפתרון אינו משפטי אלא מתמטי. במקום להסתמך על אמון, המערכת נשענת על מנגנון Commit-Reveal המבוסס על פונקציות גיבוב קריפטוגרפיות. רעיון היסוד, המוכר בספרות כ"הטלת מטבע בטלפון" [16], הוא זה: מחייבים כל צד להתחייב על החלטתו בעודה חתומה וסתומה, ורק לאחר שהיריב נעל את התחייבותו-שלו – נחשפת ההחלטה. כך נמנעת האפשרות לשנות בחירה לאחר מעשה, שכן השינוי היה שובר את החתימה הקריפטוגרפית שכבר הועברה.

חיבור לקורס

בקורס "אורקסטרציה של סוכני AI", בהרצאה L09, ראיתם כיצד שני סוכני AI משוחחים זה עם זה מעל MCP וקוראים לכלים חיצוניים – שני תהליכים עצמאיים המחליפים הודעות ישירות, בלא רכיב-על מרכזי המפקח עליהם. פרק זה מוסיף לאותה ארכיטקטורה של קריאות-כלים בין סוכן לסוכן את שכבת השלמות (Integrity): כאשר אין שרת מרכזי מהימן שמכתיב אמת אחת, הצורך לוודא את יושרתה של תקשורת מבוזרת חייב לנבוע מן הקריפטוגרפיה עצמה. זהו העיקרון שמבדיל מערכת מבוזרת שצירה ממערכת מבוזרת אמינה.

5.3 מנגנון Commit-Reveal מעל SHA-256

בכל צעד משחק מבצע כל סוכן ארבעה שלבים קריפטוגרפיים מחייבים, לפי הסדר. שלבים אלו הופכים כל מהלך לאירוע מחויב שלא ניתן להכחישו או לשנותו בדיעבד.

Nonce – מספר חד-פעמי

Nonce (קיצור של Number used once) הוא מחרוזת אקראית ייחודית הנוצרת מחדש בכל התחייבות. תפקידו כפול: ראשית, הוא מבטיח שגם אם סוכן חוזר על אותה פעולה בדיוק, הגיבוב המתקבל יהיה שונה בכל פעם. שנית, הוא מסכל התקפת מילון (Dictionary Attack) – ניסיון של היריב לנחש את התוכן הסתום על-ידי גיבוב מוקדם של כל האפשרויות הסבירות. ללא ה-Nonce, מרחב המהלכים הקטן היה מאפשר לפצח כל התחייבות בשבריר שנייה.

5.3.1 שלב 1 – התחייבות Commit

הסוכן בוחר את מהלכו הפיזי ואת הרמז שישלח (לרבות דגל Intent המציין אם הרמז אמיתי או כוזב), ומגריל Nonce ייחודי. ארבעת רכיבי הנתונים משורשים יחד ומקודדים לכדי גיבוב קריפטוגרפי בודד. הסוכן משגר דרך שרת ה-FastMCP את החתימה H_{commit} בלבד – לא את תוכנה.

חתימת ההתחייבות הקריפטוגרפית

$$H_{commit} = \text{SHA256}(\text{State} \parallel \text{Move} \parallel \text{Intent} \parallel \text{Nonce})$$

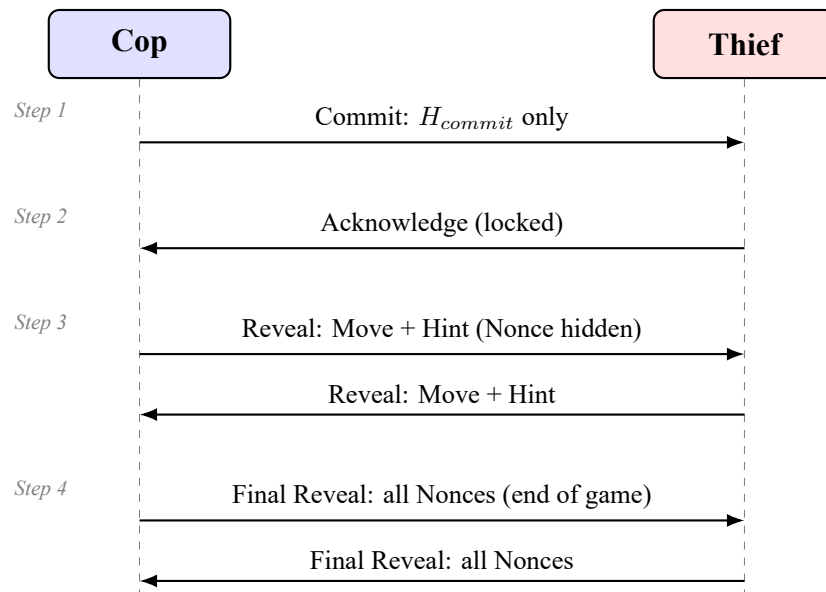
הסימן $\parallel$ הוא אופרטור השרשור (Concatenation): הוא מדביק את ייצוגי הבתים של הרכיבים זה לזה לכדי מחרוזת אחת רציפה, לפני החלת פונקציית הגיבוב. אין הוא חיבור מספרי אלא הצמדה של רצפי-בתים. משתני הנוסחה מנותחים כך:

- H_{commit} – חתימת ההתחייבות. מחרוזת בת 256 סיביות המתקבלת מפונקציית SHA-256 [17]. משמעות מעשית: זוהי "טביעת האצבע" של

- המהלך; היא נשלחת ליריב אך אינה חושפת דבר על תוכנו.
- *State* – **מצב הלוח**. תמונת המצב שעליה מבוסס המהלך, המקבעת את ההתחייבות לצעד משחק ספציפי. משמעות מעשית: מונעת שימוש חוזר בהתחייבות ישנה בהקשר חדש.
 - *Move* – **הפעולה הפיזית**. המהלך הנבחר (תנועה, הצבת חסם וכדומה). משמעות מעשית: זהו הליבה שאותה מבקשים לנעול מפני שינוי.
 - *Intent* – **דגל הכוונה**. ערך בוליאני (false/true) המציין אם הרמז המילולי הנלווה אמיתי או מטעה. משמעות מעשית: מחייב את הסוכן להכריז מראש על כנותו, כך שלא יוכל לטעון בדיעבד ששיקר "בכוונה".
 - *Nonce* – **מספר חד-פעמי**. מחרוזת אקראית קריפטוגרפית. משמעות מעשית: מבטיחה ייחודיות הגיבוב ומסכלת התקפת מילון, כמוסבר בהגדרה לעיל.

5.3.2 שלבים 2-4 – אישור, חשיפה וביקורת Acknowledge, Reveal, Audit

- לאחר ההתחייבות ממשיך הפרוטוקול בשלושה שלבים נוספים:
- **אישור (Acknowledge)**. היריב מאשר כי קיבל את ההתחייבות וכי הוא נעול עליה. אישור זה מונע מן השולח לסגת מהתחייבותו, ובה בעת מבטיח שהחשיפה תתרחש רק כששני הצדדים כבר קיבעו את מהלכיהם.
 - **חשיפה (Reveal)**. הסוכן שולח ליריב את הפעולה (*Move*) ואת המשפט המילולי. ה-*Nonce* נשאר חבוי בשלב זה, כדי למנוע הנדסה-לאחור של החתימות בטרם עת.
 - **ביקורת סופית (Audit / Final Reveal)**. רק בתום המשחק כולו נחשפים כל ערכי ה-*Nonce*, לשם ביקורת הדדית מלאה.



איור 6: רצף חילופי ההודעות בין השוטר לגנב לאורך ארבעת שלבי Com-Audit→Reveal→Acknowledge→mit. שימו לב שה-Nonce נחשף אך ורק בשלב הביקורת הסופי, בתום המשחק. Message sequence between Cop and Thief across the four Commit-Reveal phases; nonces are revealed only in the final audit.

מה רואים באיור: שני קווי-חיים אנכיים (Cop משמאל, Thief מימין) וחיצים אופקיים המתארים את סדר ההודעות מלמעלה למטה. תחילה עוברת ההתחייבות הסתומה, אחר-כך האישור הנעילה, לאחריו החשיפה ההדדית של המהלכים, ולבסוף – בתום המשחק – חשיפת כל ה-Nonce. **כיצד לפרש:** הפרדת הזמן בין ההתחייבות לחשיפה היא הלב הקריפטוגרפי; משנשלח H_{commit} , המהלך נעל מתמטית אף שתוכנו טרם ידוע. **ניתוח "מה יקרה אם":** אם סוכן ינסה לחשוף בשלב 3 מהלך שאינו תואם את ההתחייבות ששלח בשלב 1, הגיבוב שיחושב מחדש בשלב הביקורת לא יתאים ל- H_{commit} המקורי – והרמאות תיחשף חד-משמעית.

הקוד שלהלן ממחיש את שני קצות המנגנון: `commit()` היוצר את החתימה, ו-`verify()` המשחזר אותה ומשווה. שימו לב לשימוש במודול `secrets` להגרלת Nonce קריפטוגרפי, ולא ב-`random` הצפוי מדי.

מימוש commit() ו-verify() מעל SHA-256

```
import hashlib
import secrets

def commit(state: str, move: str, intent: bool) -> tuple[str, str]:
    # Generate a fresh cryptographic nonce (defeats dictionary attacks)
    nonce = secrets.token_hex(16)
    # Concatenate the fields into one byte string, then hash it
    payload = "|".join([state, move, str(intent), nonce])
    h_commit = hashlib.sha256(payload.encode("utf-8")).hexdigest()
    # Send only h_commit now; keep nonce secret until the final audit
    return h_commit, nonce

def verify(state: str, move: str, intent: bool, nonce: str, h_commit: str) -> bool:
    # Re-synthesize the opponent's hash from the revealed data
    payload = "|".join([state, move, str(intent), nonce])
    recomputed = hashlib.sha256(payload.encode("utf-8")).hexdigest()
    # Any mismatch proves tampering occurred
    return secrets.compare_digest(recomputed, h_commit)
```

מסגרת אפס-ידיעה (Zero-Knowledge)

מנגנון ה-Commit-Reveal מגלם רוח של הוכחת אפס-ידיעה [18]: כל סוכן – הן השוטר הן הגנב – מוכיח כי בחר מהלך חוקי וקיבע אותו, מבלי לחשוף מהו המהלך בטרם עת. בשלב ההתחייבות היריב מקבל ודאות מוחלטת שקיימת החלטה נעולה – אך אפס ידיעה על תוכנה. רק בחשיפה נחשף התוכן, וגם אז ניתן לאמתו מול ההתחייבות המקורית. כך מופרדת המחויבות מן הגילוי.

5.4 ביקורת הדדית ושלמות היומן - Mutual Audit and Log Integrity

אמינות המערכת כולה נשענת על בדיקת שלמות שלאחר-מעשה (Post-Mortem Integrity Check). בתום המשחק מגיש סוכן השוטר את יומנו המלא, לרבות חשיפות ה-Nonce של כל צעדיו, וכך גם הגנב. כל צד משחזר את נתוני יריבו דרך SHA-256: הוא לוקח את ה-State, ה-Move, ה-Intent וה-Nonce שהיריב חשף, מגבב אותם מחדש, ומשווה את התוצאה לחתימה שהוכרזה בשלב ההתחייבות. עקרון זה, שבו טביעת-אצבע קריפטוגרפית קצרה מעידה על שלמותו של גוש נתונים שלם, עומד גם בבסיס חתימות מבוססות-גיבוב [19].

זיוף גורר הפסד טכני

כל אי-התאמה בין הגיבוב המחושב-מחדש לבין הגיבוב שהוכרז בשלב ההתחייבות מוכיחה חד-משמעית כי בוצע זיוף (Tampering). אין כאן מקום לפרשנות או לספק סטטיסטי: פונקציית SHA-256 רגישה לכל סיבית בודדת, ולכן שינוי זעיר במהלך משנה את החתימה כליל. הקבוצה שרימתה סופגת הפסד טכני כבד – הפסד מוחלט של המשחק, ללא תלות בתוצאה על הלוח. הקריפטוגרפיה, ולא שיקול-דעת אנושי, היא הפוסקת.

5.5 צעד-אפס והוגנות חישובית - Step-0 and Computational Fairness

ness

תחרות בין סוכנים מעלה שאלת צדק: כלום ראוי שסוכן הרץ על מחשב נייד צנוע יתמודד באותם תנאים מול יריב הרץ על מחשב-על המסוגל לבצע חיפוש-עץ עמוק או להריץ מודל שפה כבד? הוגנות חישובית (Computational Fairness) דורשת שהיתרון החומרתי לא יכריע את המרוץ לבדו. לפיכך, בטרם המהלך הראשון, מבוצע "צעד-אפס" (Step-0).

בצעד זה אוספים הסוכנים את מפרט מכונתם: מספר ליבות המעבד (CPU cores), נפח הזיכרון (RAM), נוכחות מאיץ גרפי (GPU), ושם מודל השפה הרץ. המפרט נארז למחרוזת JSON ונחתם קריפטוגרפית באמצעות מפתח המסופק מראש, כך שלא ניתן לזייפו בדיעבד. במקביל, כל צריכת האסימונים (Tokens) של מודל השפה מנוטרת ונעולה אף היא קריפטוגרפית, כדי למנוע הכחשה של משאבי החישוב שנצרכו בפועל.

בחישוב ציוני הליגה מיישם המרצה נוסחת נרמול (Normalization) המעניקה בונוסים לפתרונות יעילים מבחינה אלגוריתמית – כאלה שהשיגו תוצאות טובות תוך צריכת משאבים מזערית. כך מתהפך התמריץ: לא עוצמת החומרה הגולמית זוכה לתגמול, אלא תחכום האלגוריתם. פתרון קל ומהיר הרץ על מכונה צנועה ומנצח יריב כבד – הוא ניצחון הפיתוח על פני כוח הזרוע החישובי.

5.6 סיכום הפרק

ראינו כי ברשת עמית-לעמית נטולת-שופט, האמון אינו יכול להיות הנחה – עליו להיות מוכח. מנגנון Commit-Reveal מעל SHA-256 נועל כל מהלך בחתימה קריפטוגרפית עוד בטרם נחשף תוכנו, ובכך מסכל מסע בזמן, שינוי בדיעבד והתכחשות. ביקורת הדדית של היומנים בתום המשחק חושפת כל זיוף ומענישה אותו בהפסד טכני, ואילו צעד-אפס והצהרת החומרה החתומה מבטיחים שההוגנות תישמר גם בין מכונות שאינן שוות בעוצמתן. הפרק הבא ימשיך משכבת השלמות אל שכבת האסטרטגיה: כיצד סוכן בונה מפת אמונות ומקבל החלטות טובות תחת אי-הוודאות שנשמרה כאן ביושרה.

6 מודול האסטרטגיה וקבלת ההחלטות

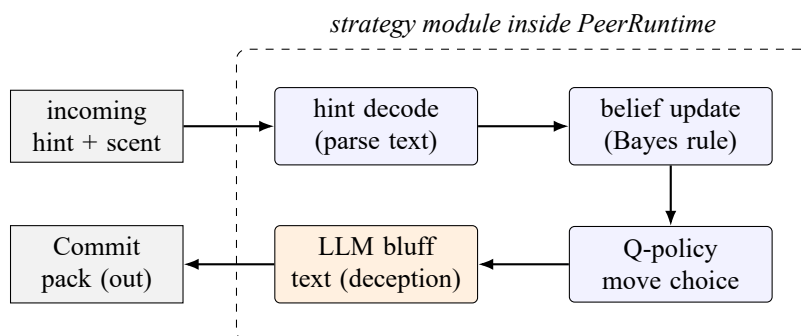
6.1 מטרות הפרק

בסיום פרק זה תדעו מדוע היגיון הנהיגה של הסוכן חייב להיות עצמאי ולא להישען באופן עיוור על מודל שפה; ותכירו מגוון של דרכים חלופיות ושקולות לממש את קבלת ההחלטות – היוריסטיקות מרחק (Manhattan) בשילוב מפת אמונות בייסיאנית, קבלת החלטות מבוססת מודל שפה, ולמידת חיזוקים כאפשרות אחת אופציונלית ומומלצת בלבד. חשוב להדגיש: הקורס לא לימד למידת חיזוקים, וניתן לבנות סוכן חזק לחלוטין באמצעות היוריסטיקות או תשאול ישיר של מודל שפה, ללא RL כלל. נראה כיצד בונה כל צד – שוטר וגנב באופן סימטרי – מפת אמונות הסתברותית ומעדכן אותה בכלל Bayes, וכיצד משתלב מודל השפה בתהליך – לא כמנוע ניווט, אלא כמנתח התנהגות ומחולל טקסט מחושב. נצרף את כל אלה למודול אסטרטגיה יחיד המתחבר לשכבת ה-PeerRuntime.

6.2 מדוע דרוש מודול אסטרטגיה נפרד Why a Separate Strategy Module

תשתית הסימולטור שבנינו בפרקים הקודמים מנהלת את ה-pipeline: העברת הודעות, נעילת חתימות (signature locking), וניהול תורות. אך שימו לב להבחנה מהותית: תשתית שיודעת להעביר הודעה אינה יודעת מה להחליט. היגיון הנהיגה של הסוכן חייב להיות חכם ועצמאי, ואל לו להסתמך באופן עיוור על מודל שפה – משום שמודלי שפה נוטים להזות (hallucinate) במרחבים קרטזיים, ולבלבל בין כיוונים, מרחקים וקואורדינטות.

מכאן נובעת דרישה ברורה בפיתוח: על הסטודנטים לממש מודול אסטרטגיה נפרד, המתחבר אל שכבת ה-PeerRuntime בנקודה מדויקת – מיד לאחר פענוח הרמז (hint) הנכנס, ולפני אריזת ה-Commit היוצא. בין שתי נקודות אלו יושבת כל האינטליגנציה של הסוכן: עדכון האמונה, בחירת המהלך החוקי, וחיבור טקסט ההטעיה. הפרדה זו איננה קפריזה ארכיטקטונית; היא הגבול המפריד בין רכיב תקשורת גנרי לבין סוכן חושב.



איור 7: זרימת ההחלטה בתוך מודול האסטרטגיה: הרמז הנכנס מפוענח, מפת האמונות מתעדכנת בכלל Bayes, מדיניות התנועה (היוריסטית, מבוססת שפה, או Q אופציונלית) בוחרת מהלך חוקי, מודל השפה מחבר טקסט הטעיה, והכול נארז ל-Commit. Decision flow inside the strategy module: decode hint, update belief by Bayes, choose a legal move by the Q-policy, compose bluff text via the LLM, and pack the Commit.

מה רואים באיור: שרשרת של חמישה שלבים החסומה בתוך מלבן מקווקו המסמן את גבול מודול האסטרטגיה בתוך ה-PeerRuntime – מן הרמז הנכנס (משמאל למעלה) ועד ל-Commit היוצא. **כיצד לפרש:** השלב הקרטזי (בחירת המהלך) והשלב המילולי (טקסט ההטעיה) מופרדים בבירור; מודל השפה

מקבל את ההחלטה על התנועה כעובדה נתונה. ניתוח "מה יקרה אם": אם היו מתירים למודל השפה לבחור את המהלך עצמו, הזיה קרטזית אחת הייתה מתרגמת ישירות למהלך בלתי חוקי או מתאבד; ההפרדה מבטיחה שהאלגוריתם שומר על חוקיות התנועה ללא תלות במודל.

6.3 למידת חיזוקים – כלי אופציונלי אחד - Reinforcement Learning as One Optional Tool

לפני שנצלול לפרטים, נדגיש: למידת חיזוקים היא אחת האפשרויות לממש את מדיניות התנועה – כלי אופציונלי ומומלץ בלבד, ולא "מה שהקורס לימד". הקורס לא כלל למידת חיזוקים, וקבוצות רבות יבנו סוכן מנצח ללא RL כלל, על בסיס שני המסלולים השקולים המתוארים בהמשך: קבלת החלטות מבוססת מודל שפה, או היוריסטיקות טהורות (Manhattan בשילוב אמונה בייסיאנית). הפרק מציג את שלושת המסלולים כשווי-ערך, והבחירה ביניהם נתונה לקבוצה. מכיוון שהגרید חסום לגודל [גודל הלוח] (למשל 10×10), מרחב המצבים סופי – אם כי גדול מאוד, בשל צירוף מיקומי השחקנים ופריסת המחסומים. סופיות זו היא התנאי שמאפשר, לקבוצות הבחורות בכך, לאמן את הסוכן בשיטות למידת חיזוקים קלאסיות, ובראשן Q-Learning [20], [21]. במסלול זה הסוכן מתחזק טבלה (או רשת) הממפה כל מצב אפשרי אל משקלי הפעולות הזמינות בו, ומעדכן ערכים אלו באמצעות משוואת Bellman [22]. לקבוצות הבחורות בכך, עדכון ערך ה-Q נעשה לפי משוואת Bellman:

עדכון Q לפי משוואת Bellman

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

רכיבי המשוואה מנותחים כך:

- $Q(s, a)$ – ערך הפעולה. האומדן הנוכחי לתגמול המצטבר הצפוי מביצוע פעולה a במצב s . משמעות מעשית: זהו התא בטבלה שאותו הסוכן מעדכן ומתוכו יבחר בהמשך את מהלכו.

- r – התגמול המיידי. התגמול המתקבל מיד לאחר הפעולה, נגזר ישירות מטבלת הניקוד. משמעות מעשית: תפיסה או שרידה מתורגמות למספר

קונקרטי שמניע את הלמידה.

- α — **קצב הלמידה (learning rate)**. $\alpha \in (0, 1]$ קובע כמה משקל ניתן למידע החדש מול הידע הצבור. משמעות מעשית: α גבוה מדי גורם לתנודתיות ולשכחת ניסיון קודם, ואילו α נמוך מדי מאט את ההתכנסות.

- γ — **מקדם ההיוון (discount factor)**. $\gamma \in [0, 1)$ קובע את חשיבות התגמול העתידי — תפיסה או שרידה ארוכות-טווח — מול הניקוד המיידי. משמעות מעשית: γ גבוה מעודד סבלנות אסטרטגית, למשל בניית מלכודת מחסומים לאורך תורות רבים.

- $\max_{a'} Q(s', a')$ — **הערך העתידי המיטבי**. האומדן הטוב ביותר לתגמול מן המצב הבא s' . משמעות מעשית: כאן "זולג" ידע העתיד לאחור אל ההווה, ומאפשר לסוכן לתכנן מעבר לצעד הבודד.

כדי להימנע מהיתקעות בלולאות קבועות במהלך המרדף, משלבים מנגנון Epsilon-Greedy: בהסתברות קטנה ε הסוכן בוחר פעולה אקראית לחלוטין במקום הפעולה בעלת ערך ה-Q הגבוה ביותר. מנגנון זה מעודד חקירה (Exploration) של מסלולי בריחה או מרדף חדשים, ומונע ניצול-יתר (Exploitation) של מדיניות שנתקעה במעגל.

מי שבחר במסלול ה-RL יבחין שהפרויקט הוא בעל צד יריב חושב, ולכן — אם השתמשתם בלמידת חיזוקים — הוא בעצם מקרה של למידת חיזוקים רב-סוכנית (Multi-Agent RL) [23], [24], שבה סביבת הלמידה עצמה משתנה ככל שהיריב לומד ומשתפר. עם זאת, זהו מסלול אחד מבין כמה, ואינו הכרחי.

6.3.1 שני מסלולים חלופיים ושקולים ללא RL - Two Equal Alternatives With- out RL

למידת חיזוקים היא כאמור אפשרות אחת בלבד. שני מסלולים נוספים, שווי-ערך לחלוטין, מאפשרים לבנות סוכן חזק גם בלעדית:

- **קבלת החלטות מבוססת מודל שפה.** במקום לאמן טבלת Q, אפשר לתשאל את מודל השפה ישירות: בונים פקודה (prompt) המכילה את סטטיסטיקות האמונה, ריכוזי הריח והמהלכים החוקיים, ומבקשים מהמודל להמליץ על המהלך הבא. האלגוריתם עדיין אוכף חוקיות, אך שיקול הבחירה מופקד בידי המודל. מסלול זה אינו דורש שלב אימון כלל.

- **היוריסטיקות טהורות (Bayes + Manhattan).** אפשר לוותר לחלוטין על למידה ולהסתמך על כלל החלטה דטרמיניסטי: לעדכן את מפת האמונות בכלל Bayes, ואז לבחור בכל תור את המהלך החוקי הממזער את מרחק ה-Manhattan אל תא האמונה הגבוהה ביותר. זהו מסלול פשוט, שקוף וניתן לניפוי-שגיאות בקלות – ולעיתים קרובות תחרותי לחלוטין מול RL. שלושת המסלולים – RL, מודל שפה, והיוריסטיקות – הם אזרחים שווי-זכויות במודול האסטרטגיה; בחרו את המתאים למשאבים ולסגנון הקבוצה.

דוגמה: עדכון Q ובחירת פעולה epsilon-greedy

```
\begin{lstlisting}[language=Python]
import random

def q_update(Q, s, a, r, s_next, actions, alpha=0.1, gamma=0.95):
    # Bellman update: blend old estimate with the observed target
    best_next = max(Q[(s_next, a2)] for a2 in actions) #
    max_a = argmax_a Q(s', a')
    td_target = r + gamma * best_next # r
    + gamma * max_a Q
    td_error = td_target - Q[(s, a)] #
    temporal-difference
    Q[(s, a)] += alpha * td_error #
```

```

    move toward target
    return Q[(s, a)]

def choose_action(Q, s, actions, epsilon=0.1):
    # epsilon-greedy: explore with prob epsilon, else exploit
    # the best Q
    if random.random() < epsilon:
        return random.choice(actions) #
    exploration
    return max(actions, key=lambda a: Q[(s, a)]) #
    exploitation
\end{lstlisting}

```

6.4 היוריסטיקות מרחק ומפת אמונות Distance Heuristics and

Belief Heatmap

שני הצדדים סימטריים לחלוטין: אף אחד מהם אינו רואה את מיקום היריב האמיתי. כל צד יודע היכן הוא עצמו, ומקבל את מפת הריח של היריב (כל צד תופס את שדה הריח של הצד השני, לא את שלו) ורמז מילולי שעשוי להיות כוזב. משום כך כל צד בונה מפת אמונות (belief map) משלו: מטריצה בגודל [גודל הלוח] (למשל 10×10) המייצגת את ההסתברות הסטטיסטית לכך שהיריב הנסתר שוהה בכל תא [15]. כך השוטר בונה אמונה על מיקום הגנב מתוך מפת הריח והרמזים שקיבל, ובאופן סימטרי הגנב בונה אמונה על מיקום השוטר הנסתר מתוך מפת הריח של השוטר והרמזים שלו, ומשתמש בה כדי לתכנן מסלול בריחה. בכל רמז נכנס מיישם הצד את כלל Bayes כדי לעדכן את ההסתברויות, תוך הצבת מקדם מהימנות (reliability) לטקסט – שהרי הטקסט עשוי להיות כוזב. השוטר מבקש אז למזער את מרחק ה-Manhattan אל התא בעל ההסתברות הגבוהה ביותר, בעוד הגנב מבקש למקסם אותו ולהתרחק ממנו.

מרחק Manhattan על גריד אורתוגונלי

$$D = |x_{\text{cop}} - x_{\text{target}}| + |y_{\text{cop}} - y_{\text{target}}|$$

רכיבי הנוסחה מנותחים כך:

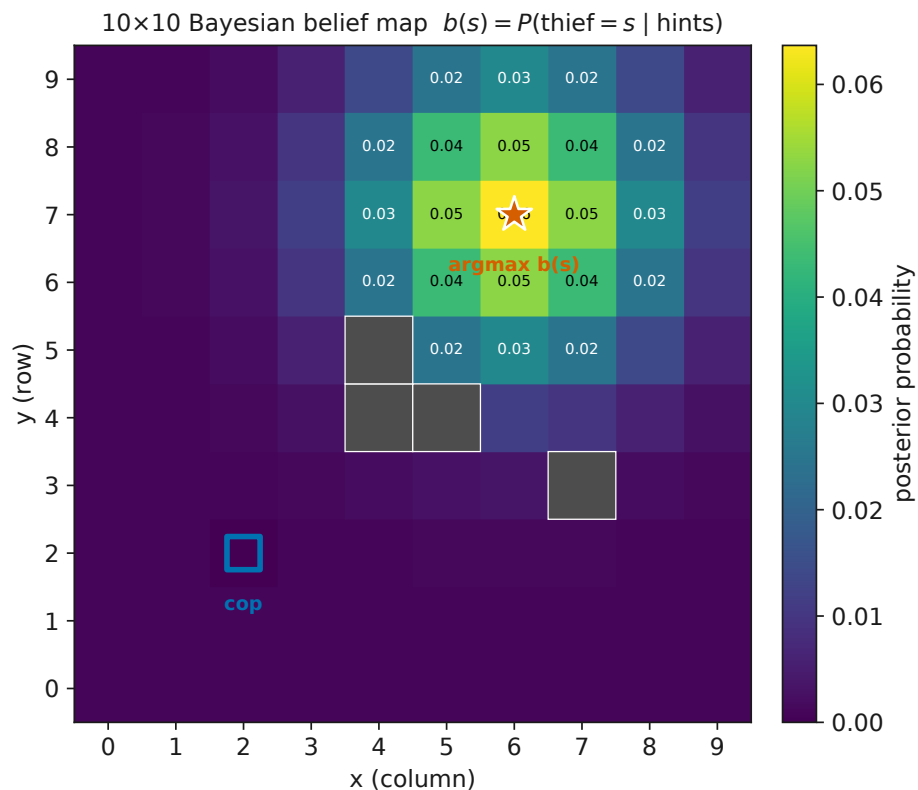
- $(x_{\text{cop}}, y_{\text{cop}})$ – **מיקום השוטר**. הקואורדינטות הידועות של הסוכן. משמעות מעשית: זהו הרכיב הוודאי היחיד במשוואה.

- $(x_{\text{target}}, y_{\text{target}})$ – **התא היעד**. תא בעל האמונה הגבוהה ביותר, $\arg \max_s b(s)$, מתוך מפת האמונות. משמעות מעשית: היעד אינו הגנב ה"אמיתי" אלא הניחוש ההסתברותי הטוב ביותר עליו.

- D – **מרחק Manhattan**. סכום ההפרשים המוחלטים בשני הצירים. משמעות מעשית: פונקציה זו הולמת תנועה אורתוגונלית על גריד, שבה אין תנועה אלכסונית, ולכן היא אומדן קביל (admissible) למספר הצעדים המזערי.

החלטת תנועה לפי מרחק Manhattan

נניח שהשוטר ניצב בתא $(2,2)$, ומפת האמונות מציבה את שיא ההסתברות בתא $(7,6)$. אזי $D = |2 - 7| + |2 - 6| = 5 + 4 = 9$. מבין הפעולות החוקיות, מזרח $(3,2)$ נותן $D = 8$ וצפון $(2,3)$ נותן אף הוא $D = 8$, בעוד מערב $(1,2)$ נותן $D = 10$. הסוכן יבחר, אם כן, במהלך המזרחי או הצפוני – שניהם ממזערים את D בצעד אחד – ויעדיף ביניהם לפי ערך ה-Q. כך משתלבים ההיגיון ההסתברותי (בחירת היעד) וההיגיון הלמידתי (בחירת המהלך).



איור 8: מפת אמונות בייסיאנית בגודל [גודל הלוח]: כל תא צבוע לפי ההסתברות $b(s)$ שהיריב הנסתר שווה בו, לאחר עדכון Bayes של רמז ריח. כאן מוצגת מפת השוטר (אמונה על הגנב); הגנב מחזיק מפה סימטרית משלו (אמונה על השוטר). הכוכב מסמן את $\arg \max_s b(s)$, הריבוע הכחול את מיקום הצד המחזיק במפה, והריבועים הכהים מחסומים בעלי אמונה אפסית. A Bayesian belief map that each agent maintains: cells are shaded by the posterior probability that the hidden opponent occupies them; the star marks the mode, the blue square the map-holder, dark squares are zero-belief barriers.

מה רואים באיור: רשת בגודל [גודל הלוח] שבה ריכוז ההסתברות (הגוון הבהיר) מתרכז סביב תא יחיד, בעוד שאר הלוח נושא אמונה אחידה נמוכה; מחזיק המפה והמחסומים מסומנים בנפרד. נזכור ששני הצדדים מחזיקים מפה כזו – זו של השוטר על הגנב, וזו של הגנב על השוטר. **כיצד לפרש:** הכוכב הוא היעד שאליו יכוון מזעור מרחק ה-Manhattan (עבור השוטר) או ממנו תתרחק הבריחה (עבור הגנב); ההתפלגות אינה נקודה חדה אלא ענן, המשקף את אי-הוודאות שנותרה גם לאחר העדכון. **ניתוח "מה יקרה אם":** אם רמז חדש יסתור את הקודם, מסת ההסתברות תיפרש מחדש – השיא יכול לנדוד או

להתפצל לשני מוקדים, והשוטר יאלץ להכריע לאיזה מהם לכוון תחילה.

6.5 שילוב מודל השפה להנדסת פקודות LLM Integration for Prompt Engineering

אף שההיגיון המרחבי מטופל כולו בידי האלגוריתם, מודל השפה נותר קריטי למשחק המילולי. מודול האסטרטגיה מנחה את הלקוח להשתמש בכלי (Tool) כנגד שרת ה-FastMCP כדי לאסוף נתונים, בונה פקודה (prompt) עשירה הכוללת את הסטטיסטיקות ואת מפות הריח, ושולח אותה למודל השפה כדי לחבר טקסט הטעיה מחושב – או ניתוח פסיכולוגי של שפת היריב. בכך פועל מודל השפה כמסווג בלופים (Bluff Classifier) וכפרופיילר התנהגותי, בעוד האלגוריתם שומר על חוקיות התנועה. חלוקת עבודה זו – שפה למודל, מרחב לאלגוריתם – היא ליבת התכנון של הסוכן, והיא נשענת על יכולות הקשב של ארכיטקטורת ה-Transformer [25].

אל תסמכו על מודל השפה לחשיבה מרחבית

לעולם אל תעבירו למודל השפה את ההכרעה על מהלך התנועה עצמו. מודלי שפה נוטים להזות (hallucination) בעת חישוב קואורדינטות, כיוונים ומרחקים במרחב קרטזי, ועלולים להחזיר בביטחון מלא מהלך בלתי חוקי, מהלך המתנגש במחסום, או מהלך המרחיק מן היעד [26]. תפקיד המודל הוא מילולי בלבד: חיבור טקסט, סיווג בלופים ופרופיילינג. ההכרעה המרחבית שמורה לאלגוריתם, שהוא היחיד המסוגל להבטיח חוקיות מתמטית.

מודול האסטרטגיה נשען על מספר שכבות מקורס אורקסטרציה של סוכני IA. ליבת הקבלת החלטות המומלצת היא מבוססת שפה טבעית: הסוכן מנוהל ומתשאל דרך מודל שפה, ברוח גישת הסוכנים והאורקסטרציה שנלמדה ב-L05. יכולת מודל השפה לנתח את שפת היריב ולחולל טקסט הטעיה נשענת על יסודות הלמידה העמוקה (נוירונים, גרדיאנט, פונקציית loss) מ-L02 ועל מנגנון הקשב (attention) של ה-Transformer מ-L04 [25]. הפעלת מודל שפה מקומי דרך Ollama כפי שתרגלתם ב-L08 מאפשרת לסוכן לחשוב מילולית ללא תלות בשירות חיצוני, ובכך לשמור על אוטונומיה מלאה מול היריב. למידת חיזוקים לא נלמדה בקורס והיא אפשרות אחת בלבד לצד היוריסטיקות ומודל שפה – כלי נוסף בארגז הכלים, לא אבן יסוד.

6.6 סיכום הפרק

בנינו מודול אסטרטגיה עצמאי המתחבר ל-PeerRuntime בין פענוח הרמז לאריזת ה-Commit, והפרדנו בבירור בין ההיגיון המרחבי לבין ההיגיון המילולי. ראינו שלושה מסלולים שווי-ערך לקבלת ההחלטות: קבלת החלטות מבוססת מודל שפה, היוריסטיקות טהורות (מפת אמונות בייסיאנית ומרחק Manhat-tan), ולמידת חיזוקים כאפשרות אחת אופציונלית (שלא נלמדה בקורס). ראינו כיצד כל צד – שוטר וגנב באופן סימטרי – בונה מפת אמונות על היריב הנסתר מתוך מפת הריח של הצד השני, וכיצד מודל השפה משרת את המשחק המילולי – ובגישה המומלצת אף את בחירת המהלך – אך לעולם אינו מופקד על החישוב המרחבי המדויק. בפרק הבא נעבור מן ההכרעה הבודדת אל הרכבת המערכת השלמה, ונראה כיצד רכיבים אלו פועלים יחד לאורך מרוץ מלא.

7 ממשק משתמש (GUI) וסימולטור השחזור

7.1 מטרות הפרק

בסיום פרק זה תדעו: מדוע ניטור בזמן אמת (Observability) הוא רכיב אינטגרלי בפיתוח מערכות P2P מורכבות, ולא קישוט חיצוני; כיצד לתרגם טבלת הסתברות מתמטית לווזואליזציה נגישה של מפת חום (Heatmap) בממשק המקומי של השוטר; כיצד באנר תור משקף את מנגנון הסנכרון הא-סינכרוני של המרוץ; ומעל הכול – כיצד לבנות Replay Viewer המשמש כמערכת עדות מהימנה, המאמתת קריפטוגרפית כל צעד בעבר ומזהה כל ניסיון זיוף של יומן המשחק.

7.2 שני צירים: ניטור חי מול עדות בדיעבד Two Axes: Live

Monitoring vs. Retrospective Witness

חלק אינטגרלי מפיתוחה של מערכת P2P מורכבת הוא היכולת לנטר את פעולות הסוכנים בזמן אמת ולוודא את חוקיותן בדיעבד. שני צרכים אלו – ניטור והוכחה – מגדירים את שני הכלים שפרק זה עוסק בהם, ואין הם חופפים. הממשק החי (Live GUI) עונה על השאלה "מה קורה עכשיו?", בעוד שסימולטור השחזור (Replay Viewer) עונה על השאלה הקשה יותר: "האם מה שקרה בעבר אכן קרה כפי שנטען?".

ההבחנה אינה טכנית בלבד אלא מהותית. בסביבה מבוזרת ללא שופט מרכזי, ההיסטוריה של המשחק אינה מאוחסנת אצל רשות מהימנה – היא נשמרת בקובץ יומן מקומי אצל כל שחקן. עובדה זו פותחת פתח לפיתוי: שחקן עלול לנסות לשכתב את עברו כדי לזכות בדיעבד. הפרק שלפניכם מראה כיצד הצפנה (עליה נשענו בפרק 5) הופכת את היומן ממסמך הניתן לזיוף למסמך עדות בלתי ניתן לערעור.

אמת מקומית (Local Truth)

אמת מקומית היא עקרון תכנוני שלפיו הממשק של כל סוכן מציג רק את המידע הנגיש לו – מיקומו שלו, מפת הריח שהוא חש, והרמזים שקיבל – ולעולם לא את מצב הלוח האובייקטיבי המלא. אין "מבט ציפור" (bird's-eye view) המראה בו-זמנית את מיקומי שני הצדדים. עיקרון זה נובע ישירות מהפורמליזם של Dec-POMDP (1): התצפית Ω_i של כל סוכן היא תת-קבוצה חלקית של המצב האמיתי S , ולפיכך ממשק שיחשוף את S המלא היה מפר את חוקי המשחק.

7.3 הממשק החי: מפת חום ובאנר תור The Live GUI: Heatmap

and Turn Banner

כל צד – שוטר וגנב – מריץ את התוכנה שלו מתוך GUI ייעודי (למשל Tkinter או PyQt). כפי שהובהר בהגדרת האמת המקומית, הממשק אינו חושף את מצב הלוח האובייקטיבי אלא את האמת המקומית בלבד. שני מנגנוני תצוגה מרכזיים הופכים את המתמטיקה המופשטת של המרוץ למידע נשלט ונגיש עבור הסטודנטים.

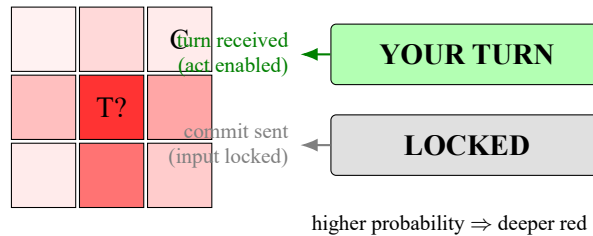
7.3.1 ויזואליזצית מפת חום Heatmap Visualization

מנגנון מפת החום סימטרי לחלוטין: כל אחד משני הצדדים מריץ GUI משלו, ובכל אחד מהם מוצג גריד המשתנה באופן דינמי המציג את מפת האמונות של אותו סוכן לגבי היריב בלבד. אצל השוטר, תאים שבהם ההסתברות לנוכחות הגנב גבוהה – בהינתן הרמזים שקיבל ומפת הריח של הגנב שהוא חש – נצבעים בגווניס הולכים ומתעצמים של אדום; ובמקביל ובאופן זהה, חלון הגנב מציג את מפת האמונות שלו לגבי מיקום השוטר, הנבנית מתוך מפת הריח של השוטר והרמזים שהגנב קיבל. אף צד אינו רואה את מיקום היריב אלא רק אומדן ההסתברות המתעדכן בזמן אמת. כך מתורגמת אצל כל סוכן טבלת ההסתברות המתמטית, שהיא אובייקט מופשט, למידע חזותי נגיש ונשלט: הסטודנט אינו נדרש לקרוא מטריצה של מספרים אלא לזהות במבט את מוקד החשד. זהו יישום ישיר של מפת האמונות (Belief) שנבנתה בפרקים הקודמים.

7.3.2 מחוון התור Turn Indicator

כדי לשקף את מנגנון הסנכרון הא-סינכרוני, הממשק כולל באנר מצב-תור חזותי. הבאנר נדלק בירוק כאשר שרת ה-MCP של היריב שידר שהתור עבר לסוכן המקומי. ברגע שהסוכן המקומי בחר את מהלכו, חתם עליו ב-Commit ושידר אותו ליריב, הבאנר הופך לאפור והממשק ננעל מפעילות עד שהתור יתקבל בחזרה. באנר זה אינו נוי גרפי בלבד; הוא ייצוג חזותי של מכונת המצבים הא-סינכרונית שמונעת מהשחקן לפעול מחוץ לתורו.

Cop Live GUI (Local Truth)



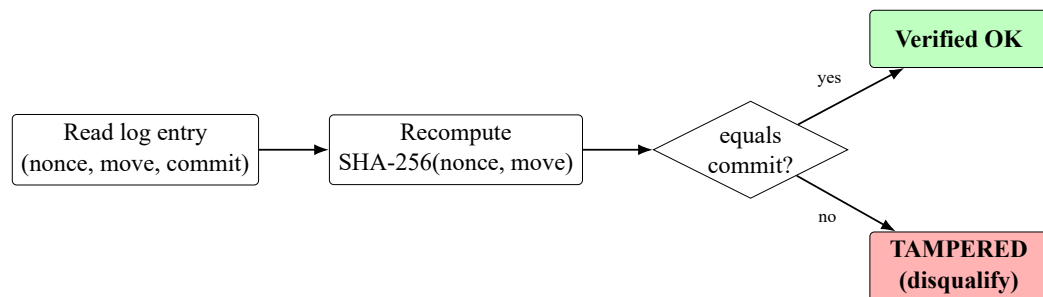
איור 9: מוק-אפ של הממשק החי: גריד אמונות שבו עוצמת האדום מבטאת את ההסתברות לנוכחות הגנב (T?), לצד באנר מצב-תור הנדלק בירוק (YOUR TURN) כשהתור מתקבל ומאפיר (LOCKED) לאחר שידור ה-Commit. Mock-up of the cop's Live GUI: a belief grid where red intensity encodes thief-presence probability, beside a turn-state banner that is GREEN when the turn arrives and GRAY/LOCKED after the commit is broadcast.

מה רואים באיור: משמאל, גריד בגודל 3 על 3 המייצג את חלון השוטר; התא הכהה ביותר מסומן T? ומבטא את ההסתברות הגבוהה ביותר לנוכחות הגנב, בעוד התא C מציין את מיקום השוטר עצמו. מימין, שני מצבי הבאנר: ירוק (YOUR TURN) ואפור (LOCKED). **ניצד לפרש:** ככל שגוון האדום עז יותר, כך גבוהה יותר ההסתברות המצטברת ממפת הריח ומהרמזים; הבאנר הירוק מסמן שהתור התקבל מה-MCP של היריב וניתן לפעול, והאפור מסמן שהממשק ננעל לאחר שליחת ה-Commit. **ניתוח "מה יקרה אם":** אם השחקן ינסה ללחוץ על מהלך בזמן שהבאנר אפור, הממשק יתעלם מהקלט – הנעילה אוכפת את התור הא-סינכרוני ומונעת מצב מרוץ (race condition) שבו שני הצדדים פועלים בו-זמנית על אותו צעד.

7.4 סימולטור השחזור ואכיפת שלמות The Replay Viewer and Integrity Enforcement

דרישת הגשה מחייבת היא בניית Replay Viewer. מטרתו לספק מערכת עדות מהימנה לסיום המשחק. השחקן טוען את קובץ היומן הסופי (למשל logs/police_match.json), ומשתמש הצופה יכול לצעוד קדימה ואחורה בזמן באמצעות כפתורי בקרה. ייחודו של הכלי אינו בתצוגה הגרפית אלא באימות הקריפטוגרפי: בכל צעד מריץ המנוע פונקציית אימות חיה הלוקחת את ה-Nonce ואת המהלך המופיעים ביומן הגלוי, מקודדת אותם מחדש באמצעות SHA-256 [17], ומשווה לערך המחויבות (Commitment) המקורי.

אם הערכים תואמים, מוצגת חותמת ירוקה "Verified OK". אם נמצא שינוי ולו הקטן ביותר בנתוני העבר – ניסיון לזיוף יומן – מדפיס הצופה באנר אדום בוהק "TAMPERED", והמשחק נפסל מיידית. עיקרון זה נשען ישירות על תכונת עמידות-ההתנגשות של פונקציית הגיבוב שראינו בפרק 5: מכיוון שאי-אפשר למצוא קלט חלופי המניב את אותו hash, כל שינוי בזוג (Nonce, מהלך) מתגלה בהכרח.



איור 10: זרימת האימות הקריפטוגרפי בסימולטור השחזור: קריאת רשומת יומן, חישוב מחדש של SHA-256 על ה-Nonce והמהלך, השוואה לערך המחויבות, והסתעפות לחותמת ירוקה Verified OK או לבאנר אדום TAMPERED הפוסל את המשחק. The Replay Viewer verification flow: read a log entry, recompute SHA-256 over the nonce and move, compare against the stored commitment, and branch to a green Verified OK or a red TAMPERED that disqualifies the match.

מה רואים באיור: שרשרת של ארבעה שלבים – קריאת רשומת היומן, חישוב הגיבוב מחדש, צומת החלטה (equals commit?), ושתי תוצאות אפשריות: תיבה ירוקה (Verified OK) או תיבה אדומה (TAMPERED). **כיצד לפרש:** הזרימה היא דטרמיניסטית – עבור אותו קלט תתקבל תמיד אותה תוצאה; הצומת המרכזי הוא נקודת ההכרעה שבה נחתך גורלו של הצעד. **ניתוח "מה יקרה אם":** אם שחקן שינה במרמה מהלך יחיד ביומן אך השאיר את ערך המחויבות המקורי, החישוב מחדש יניב hash שונה, הצומת יפנה למסלול ה-no, והבאנר האדום יופיע – המשחק נפסל אף שהשינוי היה זעיר.

7.5 מנוע האימות: סקיצת קוד The Verification Engine: A Code Sketch

לב הסימולטור הוא פונקציית צעד אחת המופעלת על כל רשומה. היא מקבלת רשומת יומן, מחשבת מחדש את הגיבוב על צירוף ה-Nonce והמהלך, משווה לערך המחויבות השמור, ומחזירה תו סטטוס. ההסבר בעברית ניתן כאן מחוץ לקוד, שכן ההערות בתוך הקוד עצמו הן באנגלית בלבד.

דוגמה: replay verifier step

```
import hashlib

def verify_step(entry):
    # Recompute the commitment from the visible log fields.
    payload = f"{entry['nonce']}|{entry['move']}".encode("utf-8")
    recomputed = hashlib.sha256(payload).hexdigest()

    # Compare against the original commitment stored in the log.
    if recomputed == entry["commit"]:
        return "Verified OK"    # green stamp: reveal matches
    # Any mismatch means the past data was altered.
    return "TAMPERED"          # red banner: disqualify the match

def replay(log):
    # Walk every recorded step; the whole match is void on first tamper.
    for entry in log:
        if verify_step(entry) == "TAMPERED":
            return "TAMPERED"
    return "Verified OK"
```

הפונקציה `verify_step` ממחישה את העיקרון: ה-Nonce והמהלך הגלויים מקודדים מחדש, וההשוואה ל-`commit` השמור היא בינארית – אין "כמעט

תואם". הפונקציה replay עוברת על היומן כולו; די בכשל אחד כדי לפסול את המשחק כולו. הערה: הסקיצה מפשטת את הקלט לצורך ההמחשה; בפועל החתימה מכסה את מלוא רכיבי הצעד – Nonce, Intent, Move, State – כמפורט בפרוטוקול שבפרק 5.

זרישת הגשה ופסילה

בניית ה-Replay Viewer היא זרישת הגשה מחייבת של הפרויקט, ולא רכיב רשות. יתרה מזו, תוצאת TAMPERED אחת – כלומר גילוי של אפילו השינוי הזעיר ביותר בנתוני העבר של היומן – פוסלת את המשחק לאלתר. אין ערעור ואין תיקון בדיעבד: מערכת העדות הקריפטוגרפית נועדה בדיוק כדי שלא יהיה מקום לשיקול דעת אנושי בשאלה אם היומן זויף.

חיבור לקורס

הצורך לנטר סוכנים ולאמת את פעולותיהם בדיעבד הוא ביטוי ישיר לעקרון ה-Observability מפיתוח מערכות ייצור [27]: מערכת שאי-אפשר להתבונן בה מבפנים אי-אפשר לתפעל ואי-אפשר לבטוח בה. הממשק החי וסימולטור השחזור הם שתי צורות של ניטור מערכת מבוזרת [4] – האחת בזמן אמת והאחרת בדיעבד. מבחינה קורסית, פרק זה הוא המשך ישיר של ההרצאה על סוכני AI ותת-סוכנים (L05) בקורס אורקסטרציה של סוכני IA: שם למדנו על סוכנים ותת-סוכנים, אורקסטרטור, פקודות וכישורים, וכן על צריכת הטוקנים וחלונות ההקשר של מערכות סוכנים. כשם שסוכן נזקק לכלים (agent tooling) כדי לפעול, המפתח נזקק לכלים כדי לראות מה עשה הסוכן, לעקוב אחר התנהגותו ולהוכיח בדיעבד כי פעל כשורה – זוהי בדיוק תצפיתיות (Observability) של מערכות סוכנים.

7.6 סיכום הפרק

ראינו כי ניטור אינו נספח אלא רכיב אינטגרלי במערכת P2P: הממשק החי מתרגם את מפת האמונות למפת חוס נגישה ומשקף את הסנכרון הא-סינכרוני בבאנר תור, כל זאת תחת עקרון האמת המקומית שאינו מתיר מבט ציפור. סימולטור השחזור, לעומתו, הופך את היומן ממסמך הניתן לזיוף לעדות קריפטוגרפית: כל צעד עובר אימות SHA-256 חי, וכל שינוי זעיר מפעיל את הבאנר האדום TAMPERED ופוסל את המשחק. בכך נסגר המעגל שנפתח עם ההצפנה של פרק 5, והפרויקט זוכה לא רק במנגנון פעולה אלא במנגנון אמון – הבסיס למרוץ אוטונומי הוגן בין שני יריבים שאינם סומכים זה על זה.

8 עיצוב ארכיטקטורת הסוכן ומנגנוני אמינות עמוקים

8.1 מטרת הפרק

בסיום פרק זה תדעו: מדוע סוכן משחק אוטונומי איננו סקריפט לינארי אלא מערכת מבוצרת הדורשת פיתוח קפדני לפי עקרון הפרדת האחריות (Separation of Concerns); כיצד תבנית ה-Orchestrator מרכזת את כל תת-המערכות מאחורי שער כניסה יחיד ומכפיפה את מהלך המשחק למכונת מצבים חוקית; ומהם דפוסי היציבות – Deadline Tracker ו-Watchdog – המגנים על הסוכן מפני קיפאון ומפני נתקים ברשת עמית-לעמית.

8.2 הפרדת אחריות כעיקרון-על Separation of Concerns

מדוע מערכת שמנצחת בסימולציה נכשלת לעיתים במשחק אמיתי מול יריב מרוחק? התשובה נעוצה לרוב לא באלגוריתם ההחלטה אלא בפיתוח המערכת סביבו. סוכן המשתתף במשחק רב-משתתפים מבוסס בינה מלאכותית, כפי שממליצים הפרוטוקולים לתחום זה, אינו יכול לערבב בקוד אחד את ניהול התקשורת, את קבלת ההחלטות ואת רישום היומנים. עירוב כזה מוליד מערכת שברירת שבה תקלה בתת-מערכת אחת מפילה את כולן.

הפתרון הפיתוחי הוא חלוקה למודולים בעלי אחריות בודדת וברורה, המתואמים בידי רכיב מרכזי אחד. הפרק שלפניכם עוסק בשלד הארכיטקטוני הזה: כיצד בונים מתזמר (Orchestrator) שמשמש שער יחיד לכל תת-המערכות, וכיצד עוטפים אותו בשכבת אמינות שמניחה מראש שהעולם – הרשת, המודל, והיריב – ייכשל בדיוק ברגע הקריטי [27].

8.3 תבנית ה-Orchestrator ומכונת המצבים Orchestrator and State Machine

בלב הארכיטקטורה עומד רכיב מרכזי המשמש Gateway – שער כניסה יחיד – לכל תת-המערכות. המתזמר הוא שמאתחל את חיבורי ה-MCP, מפעיל את מודול ההחלטה, ומתקשר עם מנהלי היומנים. במקום שכל מודול יכיר את רעהו ישירות (מבנה שמוליד תלות הדדית סבוכה), כל התקשורת עוברת דרך נקודה אחת. תבנית זו נשענת על עקרונות עיצוב מוכרים בפיתוח תוכנה [28] ועל דפוסי שער-כניסה מעולם המיקרו-שירותים [4].

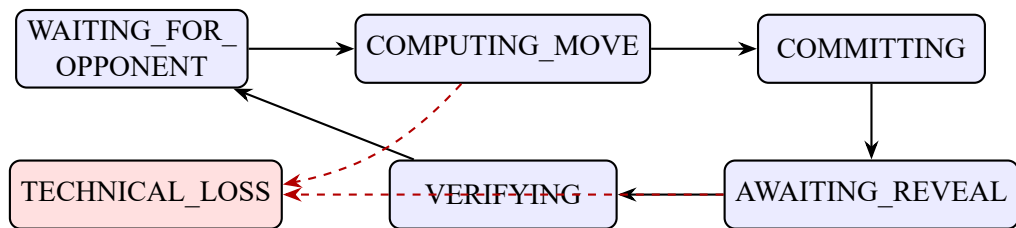
Orchestrator – מתזמר

רכיב תוכנה מרכזי המשמש נקודת כניסה יחידה (Single Gateway) לכל תת-המערכות של הסוכן. הוא אחראי לאתחול החיבורים, להפעלת מודול ההחלטה, לתיאום בין הרכיבים ולתקשורת עם מנהלי היומנים – אך אינו מכיל בעצמו לוגיקת החלטה או תקשורת ברמה נמוכה. תפקידו לתאם, לא לבצע.

המשחק כולו נשלט בידי מכונת מצבים (State Machine) קפדנית, המבטיחה שרק מעברים חוקיים בין שלבי המשחק יתאפשרו. שלב ההמתנה ליריב (WAIT-ING_FOR_OPPONENT) יכול לעבור אך ורק לשלב חישוב המהלך (COMPUT-ING_MOVE), וזה בתורו אל שלב ההתחייבות (COMMITTING), וכן הלאה. מעבר בלתי-חוקי נדחה מיד, ובכך נמנעים מצבי קיפאון (Deadlock) שבהם שני הצדדים ממתינים זה לזה עד אינסוף.

Deadlock – קיפאון

מצב שבו שתי ישויות או יותר ממתינות זו למשאב או להודעה שבידי רעותה, כך שאף אחת אינה יכולה להתקדם. במערכת עמית-לעמית ללא שופט מרכזי, קיפאון עלול לתקוע משחק שלם ללא כל הודעת שגיאה. מכונת מצבים החוסמת מעברים בלתי-חוקיים היא קו ההגנה הראשון מפני קיפאון.



איור 11: מכונת המצבים החוקית של תור משחק בודד: המערכת עוברת במחזוריות בין המתנה ליריב, חישוב מהלך, התחייבות, המתנה לחשיפה ואימות; חץ שגיאה מוביל מכל שלב תקשורתי אל הפסד טכני. The legal game-turn state machine cycles through waiting, computing, committing, revealing and verifying; a dashed error edge leads from any communication phase to a technical loss.

מה רואים באיור: חמישה מצבים תקינים המסודרים במעגל – WAIT- ,COMMITTING ,COMPUTING_MOVE ,ING_FOR_OPPONENT – כאשר האימות מחזיר את המערכת להתנהגות. בנוסף מופיע מצב שגיאה, TECHNICAL_LOSS, שאליו מובילים חצים מקווקווים. **כיצד לפרש:** החצים המלאים הם המעברים החוקיים היחידים; כל ניסיון לקפוץ ממצב אחד למצב שאינו יעד חוקי שלו נדחה. החצים המקווקווים מייצגים יציאת חירום – מעבר לשלב תקשורתי שנכשל. **ניתוח "מה יקרה אם":** אם היריב מתנתק בזמן AWAITING_REVEAL, המערכת אינה נתקעת בהמתנה נצחית אלא עוברת באורח מבוקר אל TECHNICAL_LOSS ומודיעה על תוצאה – בדיוק ההתנהגות שמכונת מצבים חוקית מבטיחה.

מימוש מכונת המצבים נשען על טבלת מעברים המפרטת, לכל מצב, אילו מצבי-יעד חוקיים. הקוד הבא משרטט מחלקה מינימלית הדוחה כל מעבר שאינו רשום בטבלה:

דוגמה: מכונת מצבים עם טבלת מעברים

```

class GamePhaseMachine:
    # Transition table: each state maps to its set of legal
    successors
    TRANSITIONS = {
        "WAITING_FOR_OPPONENT": {"COMPUTING_MOVE"},
        "COMPUTING_MOVE": {"COMMITTING", " "
  
```

```

TECHNICAL_LOSS"},
    "COMMITTING":          {"AWAITING_REVEAL"},
    "AWAITING_REVEAL":     {"VERIFYING", "TECHNICAL_LOSS"},
    "VERIFYING":           {"WAITING_FOR_OPPONENT"},
    "TECHNICAL_LOSS":      set(), # terminal state
}

def __init__(self):
    self.state = "WAITING_FOR_OPPONENT"

def transition(self, target):
    # Reject any transition not listed in the table
    if target not in self.TRANSITIONS[self.state]:
        raise ValueError(
            f"Illegal transition: {self.state} -> {target}")
    self.state = target
    return self.state

```

המחלקה שומרת את המצב הנוכחי, וכל בקשת מעבר נבדקת מול קבוצת היעדים החוקיים. מעבר בלתי-חוקי מעורר חריגה מיידית במקום להשאיר את המערכת במצב לא-מוגדר – כך הופכים באג לוגי לשגיאה גלויה שנתפסת בזמן פיתוח, ולא לקיפאון שקט בזמן משחק.

8.4 דפוסי אמינות: Reliability Watchdog ו-Deadline Tracker Patterns

מערכות עמית-לעמית (P2P) חשופות מטבען לנתקים ולעיכובים קריטיים במודל השפה. סוכן חסון אינו יכול להניח שכל בקשה תיענה; עליו לממש דפוסי מעקב אקטיביים המבחינים בין "עדיין ממתין" ל"נכשל ויש לפעול" [27]. שני הדפוסים המרכזיים כאן הם עוקב-מועד (Deadline Tracker) וכלב-שמירה (Watchdog).

8.4.1 Deadline Tracker – עוקב מועדים

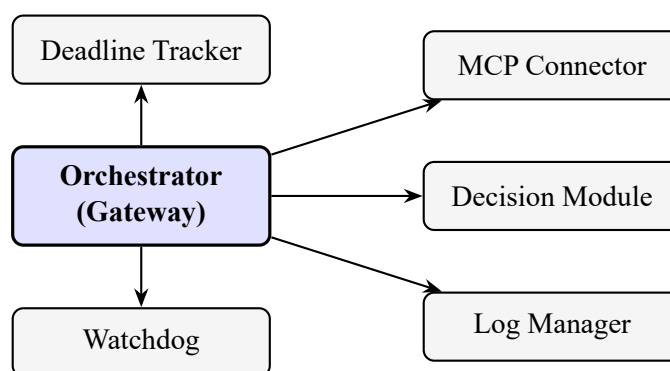
כל בקשה הנשלחת מעל שרת ה-FastMCP נושאת חותמת זמן (Timestamp) ומועד תפוגה (Expiry Deadline). אם לא הגיעה תשובה בתוך הזמן המוקצב, המערכת מבצעת ניסיון חוזר (Retry) או משדרת הודעת הפסד-טכני. דפוס זה הוא מימוש קונקרטי של תבנית ה-Timeout מספרות היציבות: לעולם אל תמתין ללא גבול למשאב חיצוני שאינו בשליטתך.

החמצת מועד היא כשל, לא סבלנות

בקשה שחלף מועד התפוגה שלה חייבת להיחשב ככשל – ולא כהזמנה להמתין עוד. השארת בקשה "תלויה" ללא מועד תפוגה היא המתכון הישיר לקיפאון: התהליך הראשי נתקע בהמתנה, כלב-השמירה מזהה שאין פעימת-לב, והמשחק קורס. על כל בקשה מעל MCP לשאת מועד תפוגה, ובחלוף המועד על המערכת לבצע Retry מבוקר או להכריז על הפסד טכני ולסגור את התור בצורה נקייה.

8.4.2 Watchdog – כלב שמירה

בעוד ה-Deadline Tracker שומר על בקשה בודדת, ה-Watchdog שומר על המערכת כולה. זהו תהליך רקע עצמאי המנטר את לולאת המשחק הראשית. אם הוא מזהה שהמערכת קפאה למשך דקות ארוכות ללא פליטת פעימת-לב (Heartbeat) – עקב קריסת מודל או כשל תקשורת – הוא יכול לבצע כיבוי מבוקר (Controlled Shutdown) ולשמר את המצב (State Persistence) לצורך התאוששות מאוחרת יותר.



איור 12: המתזמר משמש שער יחיד המתפצל אל חמש תת-מערכות: מחבר ה-MCP, מודול ההחלטה, מנהל היומנים, עוקב-המועדים וכלב-השמירה. כל תקשורת בין-מודולרית עוברת דרכו. The Orchestrator acts as a single gateway fanning out to five subsystems: the MCP connector, the decision module, the log manager, the deadline tracker and the watchdog.

מה רואים באיור: רכיב מרכזי מודגש – ה-Orchestrator – שממנו יוצאים חצים אל חמישה מודולים נפרדים: MCP Connector, Decision Module, Log Manager, Watchdog ו-Deadline Tracker. **כיצד לפרש:** כל חץ מייצג ערוץ שליטה יחיד; אין חצים בין המודולים ההיקפיים עצמם, ובכך מומחש עקרון השער היחיד – אף מודול אינו מכיר את רעהו ישירות אלא רק את המתזמר. **ניתוח "מה יקרה אם":** אם נרצה להחליף את מנוע ההחלטה במודל אחר, די בכך שנחליף מודול בודד ונשמר על אותו ממשק מול המתזמר; שאר המערכת אינה מושפעת – זהו כוחה של הפרדת האחריות.

הקוד הבא משרטט את לב ה-Watchdog: בדיקת פעימת-לב תקופתית המחליטה אם המערכת הראשית עדיין חיה:

דוגמה: בדיקת פעימת-לב של ה-Watchdog

```
import time

def watchdog_check(last_heartbeat, timeout_sec=180):
    # last_heartbeat: epoch time of the main loop's last
    signal
    elapsed = time.time() - last_heartbeat
    if elapsed > timeout_sec:
        # Main loop appears frozen: persist state and shut
        down cleanly
        persist_state()          # save game state for later
        recovery
        controlled_shutdown()    # release MCP connections,
        close logs
        return "SHUTDOWN"
    return "ALIVE"
```

התהליך משווה את הזמן שחלף מאז פעימת-הלב האחרונה אל סף קבוע. כל עוד הלולאה הראשית פולטת פעימה בקצב סדיר, ה-Watchdog מחזיר ALIVE ואינו מתערב. אך אם חלפו יותר מן הסף הקצוב – סימן שהמודל קרס או שהתקשורת נתקעה – הוא משמר את המצב ומבצע כיבוי מבוקר, כך שניתן יהיה להתאושש מאוחר יותר במקום לאבד את המשחק כולו.

רעיון המתזמר כשער-כניסה יחיד לתת-סוכנים איננו חדש לכם. בהרצאה L05 של הקורס אורקסטרציה של סוכני IA, שעסקה בסוכנים ותת-סוכנים, ראיתם כיצד סוכן-על (Orchestrator) מאציל עבודה לקבוצת תת-סוכנים דרך שער יחיד: הוא מפעיל מיומנויות (Skills) ופקודות (Com-mands), ומרכז את כל זרימת המידע ביניהם במקום שכל רכיב יפנה ישירות לרעהו. ה-Orchestrator של סוכן המשחק הוא בדיוק אותו דפוס, מוקשח לתנאי משחק תחרותי: תת-המערכות (מחבר ה-MCP, מודול ההחלטה, מנהל היומנים, עוקב-המועדים וכלב-השמירה) הן ה"תת-סוכנים", וההאצלה דרך שער יחיד – אותה הפרדת אחריות – היא שמאפשרת להחליף, לבדוד ולתקן כל רכיב בנפרד. כיוון ששני הצדדים במשחק בנויים באופן סימטרי, כל אחד מהם מריץ מתזמר ומכונת-מצבים משלו לפי אותו דפוס בדיוק.

8.5 סיכום הפרק

ראינו שסוכן משחק אמין נבנה על שני עמודי-תווך של פיתוח: תיאום ואמינות. המתזמר מרכז את כל תת-המערכות מאחורי שער יחיד ומכפיף את מהלך המשחק למכונת מצבים החוסמת מעברים בלתי-חוקיים ומונעת קיפאון. דפוסי ה-Deadline Tracker וה-Watchdog מניחים מראש שהרשת והמודל ייכשלו, ומספקים ניסיון-חוזר, כיבוי מבוקר ושימור מצב במקום קריסה שקטה. עם שלד אמין זה בידינו, נפנה בפרק הבא אל השכבה שמעליו – הלוגיקה האסטרטגית שממלאת את מודול ההחלטה בתוכן.

9 הליגה, הוגנות חישובית ואוטומציית דיווחים

9.1 מטרות הפרק

בסיום פרק זה תדעו: מדוע פרויקט השוטר-גנב אינו נבחן בתנאי מעבדה סגורים אלא בליגה אקדמית דינמית שבה סוכנים מיוצרים בידי צוותים שונים מתמודדים זה בזה בזמן אמת; כיצד "תמריץ הגיוון" ו"ההוגנות החישובית" מעצבים את פונקציית הניקוד של הליגה; ומהי תבנית ה-Gatekeeper – שלושת מנגנוני ההגנה שבלעדיהם אוטומציית הדיווח מעל Gmail API עלולה להתמוטט אל תוך הצפת שרתים או חסימת חשבון.

9.2 הליגה: מהמעבדה אל הזירה From Lab to Arena

מה מבדיל תרגיל תכנות ממערכת חיה? תרגיל מוכיח את עצמו מול בוחן יחיד וידוע מראש; מערכת חיה חייבת לשרוד מול יריבים שלא ראתה מעולם. הפרויקט שלפניכם שייך לסוג השני. הוא אינו מוגש תחת תנאי מעבדה סגורים, אלא נדרש להוכיח את עצמו בליגה אקדמית דינמית – זירה שבה סוכנים מבתי-יוצר שונים מתחרים זה בזה בזמן אמת, ללא שופט מרכזי וללא לוח זמנים מוכתב מראש.

מבנה זה משנה מן היסוד את כללי המשחק. הצלחה אינה נמדדת עוד מול תרחיש-בדיקה קבוע, אלא מול אוכלוסייה משתנה של יריבים, שכל אחד מהם מביא אסטרטגיה, ארכיטקטורה וכשלים משלו. סוכן שהצטיין מול יריב אחד עשוי להיכשל כישלון חרוץ מול אחר – וזוהי בדיוק המטרה החינוכית: לאמן מערכות עמידות (robust), ולא פתרונות שהותאמו-יתר (overfit) לבוחן יחיד.

9.2.1 מבנה הליגה ושקלול הניקוד League Structure and Weighting

כל קבוצה נדרשת לשחק מול יריבים שונים, וציון הליגה נגזר מאוסף המשחקים הללו. כדי למנוע ניצול לרעה ולעודד אתגרים חדשים, המערכת מיישמת תמריץ גיוון (Diversity Incentive): ניצחון על יריבה שטרם שיחקתם מולה מזכה בתגמול המלא ([תגמול גיוון]). אולם **אין לשחק את אותו משחק שוב ושוב מול אותה קבוצה לצורך צבירת ניקוד**: מול כל יריבה מתקיים משחק אחד נספר בלבד. משחקי חימום (warm-ups) שאינם נספרים – מותרים ואף מומלצים, לצורך בדיקה וכיול לפני המשחק הנספר. משהסתיים המשחק הנספר ושתי הקבוצות הסכימו על תוצאות, הן שולחות את הודעת סיום המשחק, והמפגש מול אותה יריבה נחתם – אין להמשיך לשחק מולה משחק נוסף לצורך ניקוד.

הצהרת מספר המשחקים Game-Count Declaration

בפתח כל משחק מצהירה כל קבוצה בפני יריבתה כמה משחקים נספרים כבר שיחקה עד כה, ולפי ההצהרות ההדדיות נקבע שקלול תמריץ הגיוון. ההצהרה איננה עניין של אמון: בתום כל משחק חוקי שתי הקבוצות שולחות למרצה את סיכום המשחק (ראו § 9), ולכן בכל רגע נתון ידוע למרצה כמה משחקים נספרים שיחקה כל קבוצה בפועל. **הצהרת כוזב שתתגלה בשלב בדיקת הפרויקט – פוסלת את הקבוצה שהצהירה כוזב.**

דרישת הסף המינימלית למעבר הפרויקט צנועה אך חד-משמעית: הפעלה תקינה של לפחות [מינימום משחקים למעבר] מול קבוצות שונות. לצד תמריץ הגיוון, פועל עקרון ההוגנות החישובית (Computational Fairness): המערכת מקטינה את יתרון-הניקוד של מי שנשען על משאבי ענן קיצוניים, ומתגמלת פיתוח יעיל-אלגוריתמית שרץ על מכונות מוגבלות. במילים אחרות, הליגה מתגמלת חוכמה בפיתוח, לא כוח חישוב גולמי – שכן אלגוריתם חכם על מכונה צנועה ראוי לציון גבוה יותר מאלגוריתם בזבזני על חוות שרתים.

9.3 אוטומציית דיווח מעל Gmail API - Gmail API Reporting Automation

בתום כל משחק חוקי מול קבוצה יריבה, אין עוד מקום להתערבות אנושית בדיווח. כל אחת משתי הקבוצות מתוכנתת לשלוח בעצמה – כל קבוצה בנפרד – הודעת סיכום אוטומטית אל המרצה באמצעות Gmail API [29]; אין די בכך שצד אחד בלבד ישלח. אוטומציה זו היא ברכה ומלכודת כאחת: היא מבטיחה דיווח אחיד ומיידי, אך מפקידה בידי קוד – שעלול להכיל באג – את המפתח לחשבון דואר חי. מה קורה כאשר לולאה אינסופית מתחילה לירות אלפי הודעות לדקה?

חיבור לקורס

פרק זה נשען על הרצאה L09 בקורס "אורקסטרציה של סוכני AI", ובמדויק על תרגיל *ex06* – שיחה בין שני סוכנים מעל MCP, שבו סוכן קורא לכלי חיצוני דרך פרוטוקול אחיד. שימו לב להבדל המהותי בין מטרות: ב-*ex06* המטרה הייתה להצליח בתקשורת מבוססת-שיחה בין שני סוכנים – הוכחת עצם היכולת לתאם ולהחליף מסרים. בפרויקט שלפניכם המטרה גבוהה בהרבה: להצליח בתקשורת ציבורית (public) – לא מקומית ולא מעל localhost – ולהריץ משחק שלם ללא שום שופט וללא שרת מרכזי. הדיווח כאן הוא אפוא "סוכן-אל-סוכן" אמיתי: השוטר אינו משוחח עם אדם, אלא מזמן כלי חיצוני – Gmail של Google – כדי למסור מצב לעמיתיו האוטונומיים ולשרתי הקורס. ההבדל הקריטי הוא שGmail הוא משאב מנוהל-מכסה בעולם האמת: קריאה אחת יותר מדי, וספק השירות חוסם. לכן הדיווח האוטונומי בסביבה ציבורית מחייב שכבת-הגנה שלא נדרשה ב-*ex06* – תבנית ה-Gatekeeper שנפרוש להלן.

9.3.1 תבנית ה-Gatekeeper ושלושת מנגנוני ההגנה The Gatekeeper Pattern

כדי למנוע כשלים חמורים – הצפת שרתי דואר (Spamming) או חריגה ממכסות Google (Rate Limit 429) – מומלץ ליישם במודול התקשורת את תבנית ה-Gatekeeper, המורכבת משלושה מנגנוני הגנה מצטברים:

הבהרת מונחים: שלושה סוגי "אסימון"

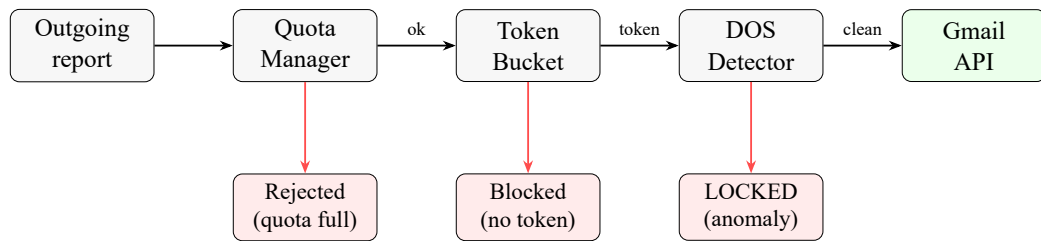
המילה "אסימון" (token) מופיעה בפרויקט בשלושה הקשרים שונים לחלוטין, ואין לבלבל ביניהם:

- **אסימוני-קצב (Token Bucket)** – יחידות לוויות עומס ברכיב מגביל-הקצב שלהלן. אינם קשורים למודל שפה כלל.
 - **טוקנים של מודל שפה (LLM tokens)** – יחידות הטקסט הנצרכות בכל קריאה למודל שפה, שאותן מודדים ומתקצבים (למשל תקציב הטוקנים למשחק).
 - **אסימוני OAuth (Refresh Token/Access)** – אישורי הרשאה מול Gmail (ראו נספח א).
- בהמשך פרק זה, "אסימון" מציין תמיד אסימון-קצב – ולא טוקן של מודל שפה.

- **מנהל מכסה (Quota Manager)**. מונה העוקב אחר מספר הפעולות שבוצעו ביום נתון, ומונע חצייה של סף הבטיחות היומי. זהו הקו האחרון מול חסימת החשבון: אם המכסה מוצתה, אף בקשה נוספת אינה יוצאת.

- **מגביל-קצב מסוג דלי אסימוני-קצב (Token Bucket Rate Limiter)**. אלגוריתם המגביל את קצב הזרקת בקשות ה-API [30]. כל דיווח נדרש ל"אסימון-קצב" (token) תקף עבור חלון-זמן מוגדר; היעדר אסימון-קצב חוסם את השליחה. כך נמנעות התפרצויות (Bursts) שעלולות להדליק חסימה מיידית מצד הספק. אין לצלצל אסימוני-קצב אלה עם טוקנים של מודל שפה.

- **גלאי DOS (DOS Detector)**. מזהה דפוסי-שליחה חריגים המעידים על באג או לולאה אינסופית בקוד הסוכן. משזוהה דפוס כזה, ה-Gatekeeper נועל לחלוטין את הגישה ל-API ומונע השעיית החשבון בידי ספק השירות – עיקרון הידוע בפיתוח מערכות backpressure ו"ניתוק מבודד" (circuit breaker) [27].



איור 13: דיווח יוצא עובר בשלושה שערי-הגנה מצטברים לפני שהוא מגיע אל Gmail API: מנהל המכסה, דלי-האסימונים וגלאי ה-DOS. כל שער עשוי להטות את הבקשה אל ענף דחייה או נעילה. An outgoing report passes three cumulative guard gates before reaching the Gmail API; each may divert it to a reject or lock branch.

מה רואים באיור: דיווח יוצא (משמאל) זורם דרך שלושה שערים סדרתיים – מנהל מכסה, דלי-אסימונים וגלאי DOS – עד שהוא מגיע אל Gmail API (מימין). מכל שער מסתעף ענף-כשל אדום. **ביצד לפרש:** רק בקשה שעברה את שלושת השערים מגיעה אל ה-API; כישלון בכל שער עוצר את הבקשה מוקדם ככל האפשר, לפי עקרון "כשל מהיר" (fail fast). **ניתוח "מה יקרה אם":** אם גלאי ה-DOS מזהה לולאה אינסופית, הוא נועל את כל הצינור (LOCKED) – ומקריב את הדיווח הבודד כדי להציל את החשבון כולו מהשעיה.

9.3.2 דלי אסימוני-הקצב: הכלל המתמטי The Token-Bucket Rule

לב מגביל-הקצב הוא כלל עדכון פשוט: אסימוני-הקצב מתמלאים ברציפות בקצב קבוע r עד לתקרת קיבולת, ונצרכים ביחידה אחת בכל דיווח. בקשה מותרת רק אם נותר בדלי אסימון-קצב שלם. שוב: אלה אסימוני-קצב לוויסות עומס, לא טוקנים של מודל שפה.

כלל דלי-האסימונים Token-Bucket Update

$$\text{tokens} \leftarrow \min(C, \text{tokens} + r \cdot \Delta t), \quad \text{allow} \iff \text{tokens} \geq 1$$

המשתנים המגדירים את הכלל:

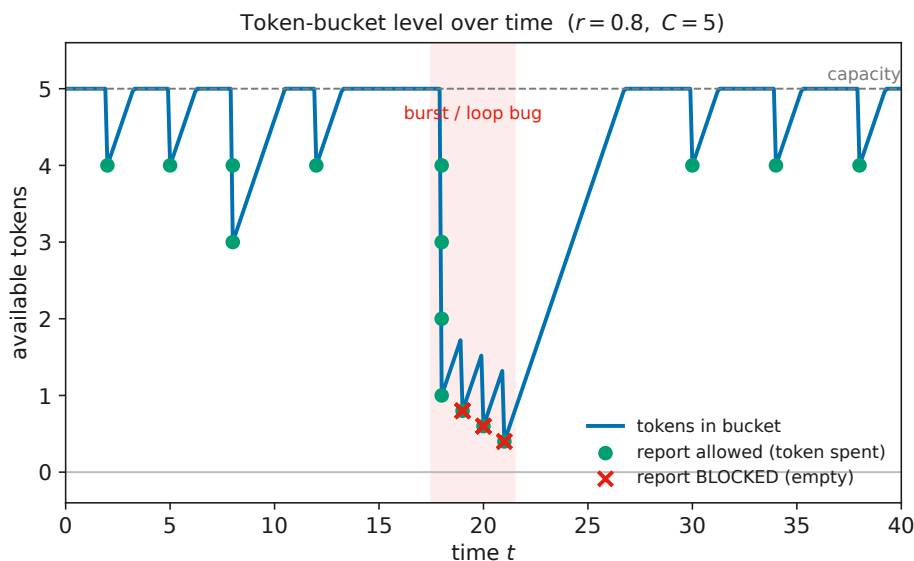
- C – **הקיבולת (capacity)**. מספר האסימונים המרבי שהדלי מסוגל להכיל. משמעות מעשית: C קובע את גודל ההתפרצות המותרת – כמה דיווחים

אפשר לשלוח "בבת אחת" לאחר תקופת שקט.

- r – **קצב המילוי**. מספר האסימונים הנוספים ליחידת זמן. משמעות מעשית: r הוא הקצב הממוצע היציב המותר בטווח הארוך; הוא חייב להישאר מתחת למכסת ה-API של Google.

- Δt – **פרק הזמן שחלף**. הזמן מאז העדכון הקודם. משמעות מעשית: ככל שחלף זמן רב יותר בין דיווחים, כך התמלא הדלי יותר – שקט מתוגמל ביכולת התפרצות עתידית.

משמעות מעשית כוללת: הכלל מפריד בין הקצב הממוצע (הנשלט בידי r) לבין ההתפרצות הרגעית (הנשלטת בידי C). דיווח מתקבל רק כאשר $\text{tokens} \geq 1$; אחרת הוא נחסם עד שהמילוי הרציף יצבור אסימון שלם. כך הסוכן שולח בחופשיות כשהתעבורה נמוכה, אך נבלם בעדינות ברגע שהוא מנסה לפרוץ את סף הבטיחות.



איור 14: מפלס האסימונים בדלי ($C = 5, r = 0.8$) לאורך זמן: מילוי רציף מול ניקוז בהתפרצויות. נקודות ירוקות = דיווח שאושר; X אדום = דיווח שנחסם כשהדלי התרוקן בזמן ההתפרצות. Token level over time under continuous refill and bursty drain; green = allowed report, red X = blocked when the bucket empties during a burst.

מה רואים באיור: העקומה הכחולה היא מפלס האסימונים בדלי לאורך זמן. בתעבורה תקינה הדלי מתמלא ומאפשר דיווחים (נקודות ירוקות), אך בזמן

ההתפרצות המסומנת (סביב $t = 18$) קצב הבקשות מרוקן את הדלי, וכל דיווח נוסף נחסם (X אדום). **ניצד לפרש:** השיפוע העולה משקף את קצב המילוי r ; כל ירידה חדה היא צריכת אסימון בידי דיווח; התקרה המקווקוות היא הקיבולת C . **ניתוח "מה יקרה אם":** אם נעלה את r , הדלי יתאושש מהר יותר ויחסמו פחות דיווחים – אך נסתכן בחריגה ממכסת ה-API; אם נקטין את C , נחנוק התפרצויות לגיטימיות. בחירת r ו- C היא, אפוא, איזון בין תפוקה לבין בטיחות.

דוגמה: Token Bucket rate limiter

```
import time

class TokenBucket:
    """Simple token-bucket rate limiter for outgoing API
    reports."""

    def __init__(self, capacity: float, refill_rate: float):
        self.capacity = capacity          # max tokens the
        bucket holds
        self.refill_rate = refill_rate    # tokens added per
        second
        self.tokens = capacity            # start full
        self.last = time.monotonic()      # last refill
        timestamp

    def _refill(self) -> None:
        now = time.monotonic()
        elapsed = now - self.last
        # continuous refill, clamped to capacity
        self.tokens = min(self.capacity, self.tokens +
        elapsed * self.refill_rate)
        self.last = now

    def allow(self, cost: float = 1.0) -> bool:
        """Return True and spend a token if one is available,
        else block."""
        self._refill()
        if self.tokens >= cost:
```

```

        self.tokens -= cost
        return True
    return False                                     # caller must back
off / retry later

```

9.3.3 מבנה הדיווח: JSON חתום ומחייב The Mandatory Signed Report

דיווח המשחק אינו טקסט חופשי. הוא נארז במבנה JSON אחיד ומחייב, ונשלח כקובץ מצורף להודעת הדואר. ה-JSON מכיל את כל פרטי הזהות של הקבוצה, כתובות ה-GitHub שלה, כתובות שרתי ה-FastMCP, הצהרות חומרה חתומות קריפטוגרפית, חותמת-הזמן של המשחק, ואישורי הסכמה-הדדית מגובי SHA-256. כל ניסיון לשלוח דיווח פתוח, שאינו קריא-מכונה (plaintext), מוביל לדחיית הדיווח.

חובה: הסכמה על התוצאה ושני דוחות נפרדים

בתום המשחק שתי הקבוצות חייבות להסכים על תוצאתו, וכל קבוצה חייבת לשלוח בעצמה את דוח סיום המשחק אל המרצה – בנפרד ובפורמט המחייב. הדיווח אינו באחריות צד אחד בלבד. אם לא יתקבל דוח מאחד הצדדים, אותו צד לא יזוכה בניקוד עבור המשחק – גם אם ניצח על הלוח. הסכמה על התוצאה ומשלוח שני הדוחות הנפרדים הם התנאי לכך ששתי הקבוצות יזוכו בניקוד המגיע להן.

שלד הדיווח המחייב (report.json)

```

{
  "schema_version": "1.0",
  "group": {
    "group_id": "team-07",
    "members": ["id-1001", "id-1002"],
    "github": ["https://github.com/team07/cop",
               "https://github.com/team07/thief"],
    "mcp_servers": ["https://team07-cop.mcp.example.org",
                    "https://team07-thief.mcp.example.org"]
  }
}

```

```

    },
    "hardware_declaration": {
      "cpu": "8-core", "ram_gb": 16, "gpu": "none",
      "signature": "ed25519:base64-signed-blob"
    },
    "match_games": [
      {"game_id": 1, "opponent": "team-13", "winner": "cop",
       "timestamp": "2026-07-09T08:15:00Z"},
      {"game_id": 2, "opponent": "team-13", "winner": "thief",
       "timestamp": "2026-07-09T08:41:00Z"}
    ],
    "mutual_agreement": {
      "opponent_group": "team-13",
      "sha256": "9f2c...ela4",
      "confirmed": true
    }
  }
}

```

כללי ברזל: Rate Limit 429 ודיווח קריא-מכונה

מכסה וקצב. חריגה ממכסת ה-API של Google מוחזרת כשגיאת HTTP 429 (Too Many Requests). שגיאה זו אינה תקלה חולפת – התעקשות עיוורת ושליחה חוזרת מיד לאחריה עלולה להוביל להשעיית החשבון בידי הספק. יש לכבד את ה-429, לסגת (back off), ולהמתין לחלון-הזמן הבא. **פורמט הדיווח.** דיווח חייב להיות JSON מובנה, אחיד וקריא-מכונה, הנשלח כקובץ מצורף. כל ניסיון לשלוח דיווח פתוח בטקסט חופשי (plaintext), שאינו ניתן לניתוח אוטומטי, יוביל לדחיית הדיווח – ומשמעות הדחייה עלולה להיות אובדן נקודות הליגה של אותו סבב.

שכבת האבטחה של הדיווח נשענת גם על הרשאה מבוקרת אל ה-API. הגישה של הסוכן אל Gmail אינה מסתמכת על סיסמה גולמית אלא על אסימוני הרשאה מדודים לפי תקן OAuth 2.0 [31], המאפשר הענקת הרשאה מצומצמת (scoped) וביטולה בעת הצורך. תהליך ההגדרה המלא של OAuth ושל אישורי ה-Gmail API מפורט בנספח [א](#).

9.4 סיכום הפרק

ראינו שהפרויקט נבחן לא במעבדה סגורה אלא בליגה חיה, שבה תמריץ הגיוון מתגמל התמודדות מול יריבים חדשים וההוגנות החישובית מתגמלת חוכמה אלגוריתמית על פני כוח חישוב גולמי. פירקנו את אוטומציית הדיווח מעל Gmail API ואת תבנית ה-Gatekeeper – מנהל המכסה, דלי-האסימונים וגלאי ה-DOS – כשכבת-הגנה שבלעדיה סוכן אוטונומי עלול להמיט אסון על חשבון חי. עמדנו על הכלל המתמטי של דלי-האסימונים ועל מבנה ה-JSON החתום והמחייב. בפרק הבא נרחיב את היריעה אל שיקולי הפיתוח הכוללים של המערכת – עמידות, בדיקות ופריסה.

10 סדר עדיפויות מומלץ בפיתוח ותהליך הפיתוח

10.1 מטרות הפרק

פרק זה הוא פרק ההרכבה, והוא כולו בגדר המלצה: לאחר שהבנו את התאוריה, את הלוח, את התקשורת ואת האסטרטגיה, נותרה השאלה המכרעת – באיזה סדר כדאי לבנות את כל אלה? בסיום הפרק תדעו: מדוע מומלץ שמערכת רב-סוכנית מורכבת תיבנה בשכבות מדורגות ולא בבת אחת; מהו סדר העדיפויות המומלץ בן שבעת השלבים; אילו אבני-דרך (Milestones) מומלץ שיעבדו לפני מעבר לשלב הבא; ומדוע דילוג על היסודות אל עבר ההצפנה או הענן עלול להיות מתכון לכשל. זהו הפרק שממיר את הידע התאורטי של הספר כולו לתוכנית עבודה מומלצת וניתנת לביצוע.

10.2 מדוע בונים בשכבות Why Build in Layers

הפיתוי הגדול של המפתח המתחיל הוא לבנות את המערכת המרשימה תחילה – ההצפנה, המנהרות אל הענן, הבינה המלאכותית שמחברת שקרים. אך מערכת מבוזרת אינה מגדל שנבנה מן הגג כלפי מטה. מהו הסיכון בהצבת שכבת ההצפנה מעל תשתית תקשורת שטרם הוכחה? כאשר מסר מוצפן אינו מגיע ליעדו, לא נדע האם האשם הוא בקריפטוגרפיה, בשרת, או בלוגיקה הבסיסית – ריבוי המשתנים הבלתי-מוכחים הופך כל תקלה לחקירה בלתי אפשרית.

עקרון ה-Incremental Delivery – אספקה מצטברת – קובע שכל שכבה נבנית, נבדקת ומיוצבת לפני שהשכבה מעליה מונחת עליה[4]. כל שלב מסתיים במערכת שעובדת מקצה לקצה, גם אם צרה בהיקפה. כך, בכל רגע נתון, מרחב התקלות האפשריות מצומצם לשכבה האחרונה שהוספנו בלבד. בצד השני של אותו עקרון עומדת מוכנות לייצור (Production Readiness): מערכת אינה נחשבת גמורה כשהיא רצה על מחשב המפתח, אלא כשהיא עומדת בכשלים, בעומסים ובניתוקים של העולם האמיתי[27]. שני העקרונות הללו – לבנות מצטבר ולהתכונן לכשל – הם עמוד השדרה של סדר העדיפויות שנפרש כעת.

מומלץ: בנייה שכבה-על-שכבה בעזרת מספר קובצי PRD

דרך מומלצת ליישם את הבנייה המדורגת היא לפצל את אפיון התוכנה למספר קובצי הגדרת מוצר נפרדים (PRD – Product Requirements Document), אחד לכל שכבה. מתחילים מן ה-PRD הראשון, מייצרים ממנו את הקוד, בודקים שהכול פועל כשורה – ורק אז מוסיפים את שכבת ה-PRD הבאה. כך כל שכבה מוגדרת, מיוצרת ונבדקת בנפרד, ומרחב התקלות בכל רגע מצטמצם לשכבה האחרונה שהוספתם. הגדרה טובה של ה-PRD עבור סוכן הבינה המלאכותית היא מיומנות בפני עצמה – ראו את קובץ ההמלצות לכתיבה והגשה של תוכנה בעזרת סוכני AI שבמבוא לקורס.

פרק זה הוא נקודת ההתכנסות של הקורס אורקסטרציה של סוכני IA כולו, מ-L01 ועד L11. תהליך הפיתוח המדורג הוא בדיוק מחזור חיי הפיתוח (SDLC) בגישת ה-vibe-coding שפתחה את הקורס (L01): מגדירים יעד צר בשפה טבעית, מממשים, בודקים, ומרחיבים. השכבות עצמן ממחזרות את מסע הלמידה כולו – יסודות הלמידה העמוקה (L02) עד L04: נוירונים ובאקפרופגיישן, רצפים ו-RNN/LSTM, טרנספורמרים וקשב-עצמי; הסוכנים, האורקסטרטור וצריכת הטוקנים (L05) והרכבת צוותי סוכנים וכלים (L06); גרפי הידע של Graphify (L07) והרצת מודל שפה מקומי (L08); השיחה בין שני סוכנים מעל MCP (L09) והזיקוק מן הענן אל המודל המקומי (L10); ולבסוף נחיל הסוכנים עם שבילי הפרומון הדינמיים והזיכרון הקולקטיבי (L11). הפרויקט הסופי אינו נושא חדש אלא האיינטגרציה של כל מה שלמדתם: בכל שלב תזהו הרצאה שהכשירה אתכם לבנותו.

10.3 שבעת שלבי סדר העדיפויות בפיתוח – שבעה קובצי PRD

The Seven Development Priorities

כדי לרסן את מורכבות הפרויקט מומלץ סולם מדורג של עדיפויות פיתוח. מומלץ לממש כל שלב כקובץ PRD נפרד ולפי הסדר, כך שכל שלב מניח תשתית מוצקה שעליה נשען הבא אחריו – שבעה שלבים, שבעה קובצי PRD.

10.3.1 שלב 1: לוגיקת בסיס Base Logic

ראשית מוקם הגרעין הפיזי של המשחק, ללא כל תקשורת או בינה: הגריד בגודל [גודל הלוח] (למשל 10×10), חוקי התנועה, [מכסת המחסומים], וזיהוי לכידה פשוט על בסיס חפיפת קואורדינטות. בשלב זה המערכת כולה רצה בתהליך יחיד. משמעות מעשית: אם שני סוכנים אינם יכולים לזוז נכונה על לוח מקומי, אין טעם לחבר ביניהם רשת. זהו הבסיס שהוצג בפרק 3.

10.3.2 שלב 2: תשתית FastMCP בסיסית Basic MCP Infrastructure

כעת מפרידים את הסוכנים לתהליכים נפרדים: מקימים את השרתים ומתכנתים את הכלים (Tools) לקבלת ושליחת מידע גאומטרי טהור מעל Lo-calhost [7]. הסוכנים עדיין מדברים בקואורדינטות מספריות בלבד. משמעות מעשית: מטרת השלב היא להוכיח שהצינור עובד – שמסר שיוצא מסוכן אחד מגיע לשני – לפני שנטעין אותו בתוכן מורכב. תשתית זו נבנתה בפרק 2.

10.3.3 שלב 3: מודול אסטרטגיה "עיוור" Blind Strategy

עם צינור תקשורת עובד, מחווטים גרסה ראשונית של מודול אסטרטגיה – כלי קבלת החלטות פשוט הפועל בעולם של מידע מלא ומדויק. הבחירה במימוש נתונה בידיכם: היוריסטיקה ישירה (מרחק מנהטן, אמונה בייסיאנית), מדיניות המבוססת על מודל שפה (LLM) הממופה ישירות לצעד, או – אופציונלית – משוואת בלמן/Q-Learning למציאת המסלול הקצר ביותר [20] עבור קבוצות הבחורות בכך. המודול "עיוור" במובן שאין עדיין ריח, שפה טבעית או הטעיה. משמעות מעשית: כך מבודדים את נכונות ליבת קבלת ההחלטות מרעשי אי-הוודאות, שיתווספו רק בשלב הבא. יסודות אלה נדונו בפרק 6.

10.3.4 שלב 4: שפה טבעית ושילוב ריח Language and Scent

זהו שלב קפיצת המדרגה. מחליפים את הקואורדינטות הנוקשות בדיווח בשפה חופשית; מטמיעים את משוואות הפרומון הדינמיות ואת דעיכתן; ובמקביל משבצים את מודל השפה (LLM) להסקה ולחיבור שקרים. משמעות מעשית: כאן נולדת אי-הוודאות שהיא לב הפרויקט – שילוב של דינמיקת הריח (פרק 4) עם ההסקה האסטרטגית (פרק 6). זהו השלב הרגיש ביותר, ולכן הוא בא רק לאחר שהתשתית והלוגיקה הוכחו.

10.3.5 שלב 5: חשיפה לענן ומנהור Cloud Exposure and Tunneling

עוברים מ-localhost אל כתובות ציבוריות באמצעות ngrok או Localtonet, ומחברים סוכנים ממחשבים מרוחקים [8]. משמעות מעשית: מרגע זה המערכת אינה עוד סימולציה על מכונה אחת אלא מערכת מבוזרת אמיתית, על כל אתגרי ההשהיה והניתוקים שבה. זו הרחבה של אותה תשתית MCP מפרק 2, כעת מעבר לרשת.

10.3.6 שלב 6: אבטחה וקריפטוגרפיה Security and Cryptography

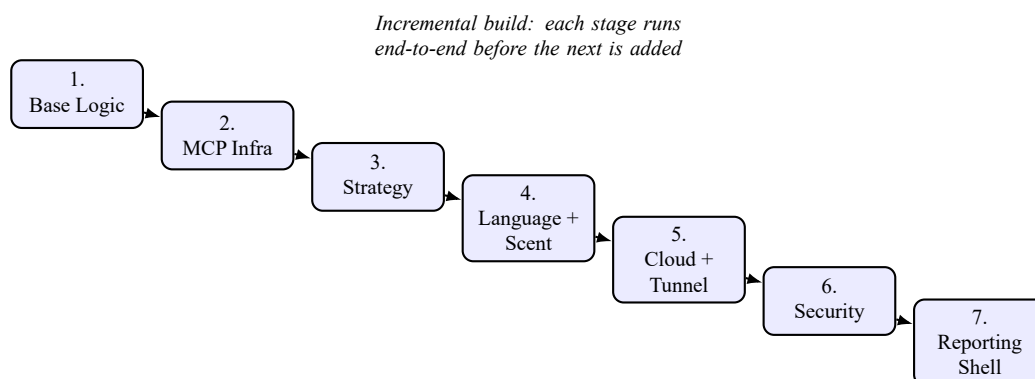
רק כשהתקשורת המרוחקת עובדת, עוטפים אותה במנגנוני ה-Commit-Reveal, כותבים את מחולל ה-Nonce, ומשלבים את הצהרות החומרה (Step-0). משמעות מעשית: ההצפנה מוסיפה שכבת אמון מעל תקשורת שכבר הוכחה כאמינה תפעולית – סדר שמונע בלבול בין תקלת רשת לתקלת קריפטוגרפיה. מנגנונים אלה פורטו בפרק 5.

10.3.7 שלב 7: מעטפת דיווח והמחזה Reporting and Visualization Shell

לבסוף נבנית המעטפת החיצונית: חיבור Gmail API דרך OAuth 2.0 (מפורט בנספח א), השלמת ה-GUI, וליטוש אפליקציית השחזור (Replay App). משמעות מעשית: זוהי שכבת החוויה והתיעוד, הנבנית אחרונה מפני שהיא צורכת את כל השכבות שמתחתיה. נושאים אלה נשענים על פרקי הליגה והממשק, פרקים 9 ו-7.

טבלה 3: מיפוי שבעת השלבים (שבעה קובצי PRD) אל פרקי הספר Mapping the seven PRD stages to book chapters

הפרק הרלוונטי	מה בונים	השלב (PRD)
פרק 3	גריד [גודל הלוח], חוקי תנועה, [מכסת המחסומים], זיהוי לכידה	1
פרק 2	שרתי FastMCP וכלים גאומטריים מעל Localhost	2
פרק 6	מודול אסטרטגיה ראשוני: היוריסטיקה, מדיניות LLM או בלמן/Q-Learning (אופציונלי)	3
פרק 4, פרק 6	שפה טבעית, משוואות ריח ודעיכה, שילוב LLM להטעה	4
פרק 2	מעבר לכתובות ציבוריות ומנהור (Localtonet/ngrok)	5
פרק 5	Commit-Reveal, מחולל Nonce, הצהרות חומרה (Step-0)	6
פרק 9, פרק 7, א	Gmail API מעל OAuth 2.0, GUI, אפליקציית Replay	7



איור 15: מפת הדרכים לפיתוח כמדרגות עולות: כל שלב מונח על קודמו, ורק לאחר שהוא רץ מקצה לקצה מתווסף השלב הבא. The engineering roadmap as an ascending staircase: each stage rests on the previous one, and only after it runs end-to-end is the next stage added.

מה רואים באיור: שבע תיבות ממוספרות היורדות כמדרגות משמאל-למעלה לימין-למטה, כשחץ בנייה מצטבר מחבר כל שלב אל הבא. **כיצד לפרש:** מבנה המדרגות מדגיש שאין קפיצות – כל מדרגה נשענת על זו שמתחתיה, כשם שכל שכבת קוד נשענת על יציבות קודמתה. הכיתוב העליון מזכיר שהמעבר בין מדרגות מותנה בכך שהשלב הנוכחי רץ מקצה לקצה. **ניתוח "מה יקרה אם":** אם נשמיט מדרגה – למשל נקפוץ משלב 2 ישירות לשלב 6 – המדרגות שמעליה תלויות באוויר: מנגנון ה-Commit-Reveal ייבנה מעל תקשורת שלא הוכחה, וכל תקלה תהפוך לחקירה רב-משתנית בלתי פתירה.

10.4 אבני דרך ומשמעת פיתוח Milestones and Development

Discipline

כוחו של סדר העדיפויות המומלץ טמון בעקביות היישום. לכל שלב מומלץ להגדיר אבן-דרך בדידה: קריטריון בינארי שכדאי שיתקיים בטרם מעבר קדימה. אבן-דרך אינה "הקוד נכתב" אלא "ההתנהגות נצפתה מקצה לקצה" – בדיוק רוח המוכנות לייצור[27].

רשימת תיוג לאבני דרך

- ודאו שכל פריט עובד ונצפה לפני המעבר לשלב הבא:
- **שלב 1:** שני סוכנים זזים חוקית על גריד [גודל הלוח]; מחסום מעבר ל[מכסת המחסומים] נדחה; חפיפת קואורדינטות מפעילה לכידה.
- **שלב 2:** מסר גאומטרי היוצא מסוכן A מעל Localhost מתקבל ומפוענח נכונה אצל סוכן B.
- **שלב 3:** בהינתן מיקום יעד ידוע, הסוכן מחשב ומבצע את המסלול הקצר ביותר ללא התערבות ידנית.
- **שלב 4:** דיווח בשפה חופשית מתורגם להסקה; מפת הריח מתעדכנת ודועכת בכל צעד; ה-LLM מפיק רמז (אמת או שקר).
- **שלב 5:** סוכן על מחשב מרוחק מתחבר דרך ngrok ומשחק סבב מלא מול הסוכן המקומי.
- **שלב 6:** מהלך מחויב ב-Commit ואז נחשף ב-Reveal עם Nonce תקין; Step-0 מאמת חומרה.
- **שלב 7:** סיכום משחק נשלח ב-Gmail; ה-GUI מציג את המצב; ה-Replay App משחזר סבב שהוקלט.

מומלץ: אל תדלגו קדימה

מומלץ שלא לגשת לקריפטוגרפיה או לענן לפני שלוגיקת הבסיס ותשתית ה-MCP מעל Localhost עובדות מקצה לקצה. דילוג על היסודות עלול שלא לחסוך זמן אלא להכפילו: תקלה בשכבה עליונה תסתתר מאחורי חוסר-יציבות בשכבה שמתחתיה, ותאבדו שעות בחקירת מקור שאינו קיים. מומלץ לבנות את המדרגות מלמטה למעלה.

10.5 סיכום הפרק

פרסנו את סדר העדיפויות המומלץ בפיתוח בן שבעת השלבים – מלוגיקת בסיס, דרך תשתית MCP, אסטרטגיה עיוורת, שפה וריח, ענן, אבטחה, ועד מעטפת הדיווח – והמלצנו לממש כל שלב כקובץ PRD נפרד, כיישום ישיר של עקרונות האספקה המצטברת והמוכנות לייצור. ראינו שכל שלב מקביל לפרק בספר ולהרצאה בקורס, וכי אבני-דרך בדידות ומשמעת מומלצת של בנייה מדורגת הן שהופכות תוכנית לתוצר. עם מפת הדרכים הזו בידכם, הידע התאורטי של הספר כולו הופך לתהליך בנייה בר-ביצוע – וזהו, בסופו של דבר, מבחן הפיתוח האמיתי.

11 סיכום ומבט קדימה

11.1 מטרות הפרק

בסיום פרק זה תבינו מדוע הפרויקט שלפניכם איננו מטלת תכנות בלבד אלא תרגיל בפיתוח מערכות מורכבות תחת תנאי רשת אמיתיים; תזהו את ארבעת המדדים שקובעים את הצלחת הקבוצה – תיאום, הסתגלות, שלמות, וארכיטקטורה – ותדעו כיצד המיומנויות שרכשתם משתקפות במערכות בינה מלאכותית מבוזרות בתעשייה. פרק זה איננו מוסיף מנגנון חדש; הוא קושר יחד את חוטי הספר כולו לכדי תמונה אחת.

11.2 הקשת של הספר: ממידול אי-ודאות אל ליגה חיה The Arc of the Book

כאשר פתחנו את הספר, מיסגרנו את המרוץ כ- Dec-POMDP : שני סוכנים מבוזרים, מרחב מצבים רב-ממדי, ותצפית חלקית שהיא לב אי-הוודאות (פרק 1) [1]. הבחנה זו הייתה נקודת המוצא לכל מה שבא אחריה, שכן ברגע שקיבלנו שאין שרת מרכזי ואין שופט חיצוני – נאלצנו לבנות את כללי המשחק, את האמון, ואת ההכרעה מן היסוד.

מהי, אם כן, הדרך שעברנו? היא איננה רשימה של נושאים אלא נרטיב פיתוחי בעל היגיון פנימי. מן המידול המופשט של אי-הוודאות עברנו אל התשתית שמאפשרת לשני זרים לתקשר ללא מתווך – ארכיטקטורת ה-P2P מעל FastMCP (פרק 2). אך תקשורת בין יריבים חסרי-אמון מחייבת מנגנון שימנע רמאות: לכן צללנו אל הקריפטוגרפיה של Commit-Reveal ואל הוכחות היושרה (פרק 5) [18], שהופכות הבטחה מילולית לחוזה מתמטי כופה. במקביל, למדנו כיצד סוכן יכול לפעול בתבונה דווקא כשאיננו רואה את יריבו: באמצעות עקבות ריח בהשראת סטיגמרגיה (פרק 4) [12], שממירות זיכרון מרחבי מבוזר להסתברות ניתנת-לחישוב.

על התשתית הזו הצבנו את מוח ההחלטות. מפת האמונות ואלגוריתמי הבחירה (פרק 6) לימדו את הסוכן לשקלל תגמול מיידי מול סבלנות אסטרטגית, ולתרגם שדה ריח דועך לכדי מהלך. ואז – לאחר שהמנוע פעל – הרכבנו סביבו את מעטפת הפיתוח: תבניות ה-Gatekeeper וה-Orchestrator (פרק 8), פרק 10) שמבטיחות שקוד עמיד לא יקרוס תחת קלט זדוני, וכלי ה-Live GUI וה-Replay (פרק 7) שהופכים ריצה אטומה למשחק שקוף וניתן-לביקורת. לבסוף יצאנו מן המעבדה אל העולם: הליגה החיה, המשחקים מול קבוצות אחרות, ודיווח התוצאות ב-Gmail API (פרק 9). כל שכבה נשענת על קודמתה; הסר אחת, והמגדל כולו מתנדנד.

11.3 הפרויקט כפיתוח מערכות, לא כתרגיל תכנות Systems Development, Not a Coding Task

מהו ההבדל המהותי בין תרגיל תכנות לבין פיתוח מערכות? תרגיל תכנות נבחן בסביבה סטטית, שבה הקלט צפוי והיריב איננו קיים. מערכת, לעומת זאת, נמדדת בעולם רועש: קווי תקשורת נופלים, יריב שולח הודעה מעוותת, שעון מקומי סוטה, וכתובת URL ציבורית מתנתקת באמצע תור. הפרויקט הזה מכריח אתכם להתמודד עם כל אלו בעת ובעונה אחת.

זו התובנה שיש לשאת מכאן והלאה: איכות הקוד שלכם נמדדת לא כשהכול תקין, אלא כשמשוהו משתבש. סוכן שמנצח מול שותף ידידותי אך קורס מול יריב עוין לא באמת פתר את הבעיה – הוא רק פתר את הגרסה הקלה שלה. ההבחנה הזו היא שמפרידה בין מי שכתב תוכנית לבין מי שפיתח מערכת.

11.4 ארבעת מדדי ההצלחה The Four Metrics of Success

כאשר נבחן כיצד קבוצה מתמודדת בעולם האמיתי של מערכות AI מבוזרות, ההצלחה מתכנסת לארבעה מדדים. אין הם עצמאיים זה מזה – הם ארבע פנים של אותה יכולת פיתוחית – אך כל אחד נבחן בפרק אחר ובא לידי ביטוי ברכיב קוד מוחשי.

טבלה 4: מדדי ההצלחה של הפרויקט וקריטריון ההגשה
Success metrics and the submission criterion

מדד	ביטוי בפרויקט	הפרק
תיאום (Coordination)	ניהול תורות, פרוטוקול P2P מעל FastMCP, וסנכרון שני סוכנים ללא שופט מרכזי	פרק 2
הסתגלות (Adaptation)	שני הסוכנים מתמודדים סימטרית עם אי-ודאות: כל צד בונה אמונה על מיקום יריבו ממפת הריח הדועכת של היריב ומרמז מילולי, ומעדכן מפת אמונות הסתברותית	פרק 4, 6
שלמות (Integrity)	מניעת רמאות בעזרת Commit-Reveal ו-SHA-256, ומעבר ביקורת (Audit) מלא	פרק 5
ארכיטקטורה (Architecture)	הקפדה על תבניות Gatekeeper ו-Orchestrator וקוד עמיד לכשלים	פרק 8, 10
קריטריון ההגשה	הפרויקט כולו – קוד, מבנה והגשה – נבחן על-פי קובץ ההמלצות לכתיבה והגשה של תוכנה בעזרת סוכני AI שבמבוא לקורס	מבוא לקורס

כיצד לקרוא את הטבלה: כל שורה היא שאלה שהעולם האמיתי ישאל את המערכת שלכם. האם היא יודעת לתאם מול צד שאיננו בשליטתה? האם היא מסתגלת כשהמידע חלקי? האם היא נשארת ישרה כשמשתלם לרמות? והאם הארכיטקטורה שלה מחזיקה מעמד תחת עומס וכשל? קבוצה שעונה בחיוב על כל הארבע איננה רק מריצה סוכן – היא מפעילה מערכת.

ארבעת המדדים אינם ייחודיים למרוץ שוטר-גנב; הם המטבע העובר לסוחר בכל מערכת AI מבוזרת בתעשייה, וקושרים ישירות אל שלושה נדבכים בקורס אורקסטרציה של סוכני AI. תיאום מבוזר הוא בדיוק האתגר של שני סוכנים המשוחחים מעל MCP וקוראים לכלים חיצוניים (הרצאה L09), ושל ייצור-המוני של צוותי סוכנים בכלים כ-LangChain, LangGraph ו-CrewAI (הרצאה L06) [4]. ההסתגלות תחת אי-ודאות שאבה את השראתה מנחיל הסוכנים המבוזר של הרצאה L11: עקבות ריח דינמיות וזיכרון-נחיל קולקטיבי, שבו הצלחות מעדכנות ייצוג משותף ומנגנון דעיכה מונע קיבעון – הכול ללא בקר-על. שלמות קריפטוגרפית היא הבסיס ל-supply-chain security, לחתימות מודלים, ולמסחר מבוזר. וארכיטקטורה עמידה – תבניות ה-Gatekeeper וה-Orchestrator – היא בדיוק מה שמפריד בין הדגמה שעובדת פעם אחת לבין שירות ייצור שרץ חודשים. המיומנויות שרכשתם כאן אינן תרגיל אקדמי; הן ערכת הכלים של מפתח מערכות AI מבוזרות.

11.5 רשימת בדיקה אחרונה לפני הגשה Final Pre-Submission

Checklist

לפני שתשלחו את הפרויקט, עצרו וודאו שכל שכבה שבניתם לאורך הספר אכן מתפקדת מקצה-לקצה. רשימת הבדיקה הבאה ממפה כל דרישה אל הפרק שבו נלמדה, ואל הנספחים המלווים.

רשימת בדיקה אחרונה לפני הגשה

- עברו על כל סעיף וודאו שהוא מסומן בפועל, לא רק בכוונה:
- **לוגיקת בסיס פועלת:** מנוע המשחק מריץ מרוץ שלם ללא קריסה, וכללי הניקוד (פרק 3) נאכפים כהלכה.
- **FastMCP מעל URL ציבורי:** שני הסוכנים מתקשרים בפרוטוקול P2P (פרק 2) דרך כתובת נגישה, לא רק ב-localhost.
- **Commit-Reveal וביקורת עוברת:** מנגנון ההתחייבות-וחשיפה (פרק 5) פעיל, וה-Audit מסתיים בהצלחה ללא זיהוי זיוף.
- **מפת ריח ומפת אמונות:** עקבות הסטיגמרגיה (פרק 4) ומפת האמונות (פרק 6) מחושבות ומשפיעות על ההחלטות בפועל.
- **Live GUI ו-Replay App עם Verified OK:** כלי הצפייה (פרק 7) מציגים את המשחק בזמן אמת ובשחזור, עם חותמת אימות תקינה.
- **דיווח Gmail API כ-JSON – משני הצדדים:** בתום המשחק שתי הקבוצות מסכימות על התוצאה, וכל קבוצה שולחת בעצמה את דוח הסיום בפורמט JSON מובנה דרך Gmail API (פרק 9); אם צד לא שלח דוח – אותו צד לא יזוכה בניקוד. להגדרת ההרשאות ראו [א](#).
- **מאגר GitHub עם Git Tag ו-README אקדמי:** הקוד מתויג בגרסה ומלווה ב-README מסודר; לנוהל ההגשה ראו [ג](#).
- **לפחות [מינימום משחקים למעבר] משחקים מול קבוצות שונות:** השלמתם [מינימום משחקים למעבר] מרוצים לכל הפחות מול יריבים שונים בליגה החיה (פרק 9); לקובץ התצורה המשותף ראו [ב](#), ולערכים המחייבים ראו את טבלת הפרמטרים המרכזית בנספח [ה](#).

רשימת הגשה: מודל, GitHub ודוח PDF

- א. חלק מהמטלה הוא לדעת להגדיר ולאפיין לסוכן את ההוראות המתאימות ליצירת הקוד המבוקש. ודאו שמצורפת ל-GitHub תיקייה המכילה את קובצי ה-markdown (קובצי ה-PRD), וששורש מאגר ה-GitHub קריא)
(README.md). ודאו שהקוד והפרויקט כולו עומדים בכל ההנחיות שבמבוא לקורס – בקובץ "המלצות לכתיבה והגשה של תוכנה בעזרת סוכני AI"; בדיקת המטלה תתבצע על-פי העקרונות שבקובץ זה.
- ב. ההגשה מתבצעת במודל על-פי ההנחיות הקבועות. יש להגיש את קוד התוכנה ב-GitHub ולשתפו עם המרצה.
- ג. כל אחד מחברי הקבוצה מגיש את המטלה בנפרד במודל.
- ד. יש לתת לקבוצת המגישים קוד מזהה ייחודי בן שמונה תווים, ללא רווחים.
- ה. המודל כולל קובץ Word ובו תבנית ליצירת קובץ ה-PDF להגשה. **אין לשנות שדות או להזיז מיקום בתבנית** – יש רק למלא את הפרטים, לשמור כ-PDF ולהגיש.
- ו. יש לתת ציון עצמי לאיכות הקוד בלבד – לא לתוצאת משחק הליגה. ציון עצמי המבוסס על תוצאת המשחק יעוות את הקריטריון למדידת איכות הקוד.

11.6 מבט קדימה: אל עבר AI מבוזר אוטונומי Looking Forward

המרוץ שבניתם הוא מודל מוקטן של שאלה שתלווה את תחום הבינה המלאכותית בעשור הקרוב: כיצד גורמים לישויות אוטונומיות רבות – שאינן סומכות זו על זו, אינן רואות את התמונה המלאה, ואינן כפופות לבקר מרכזי – לפעול יחד באופן קוהרנטי, ישר ועמיד? זו איננה שאלה תיאורטית. היא לב הלב של מערכות ה-AI המבוזרות מרובות-הסוכנים שהמחקר רק מתחיל למצות את פוטנציאלן; למידת חיזוקים (Multi-Agent RL) היא אך אחד הכלים האופציונליים שקבוצות מסוימות עשויות לבחור בו לצד אסטרטגיות מבוססות-LLM והיוריסטיקות [23].

אם למדתם כאן דבר אחד שיישאר אתכם, שיהיה זה: מערכת מבוזרת טובה איננה אוסף של סוכנים חכמים, אלא ארכיטקטורה שמאפשרת לסוכנים בינוניים לפעול יחד באמון, בהסתגלות, ובעמידות. את היכולת הזו – לתאם, להסתגל, לשמור על שלמות, ולפתח נכון – נשאתם עמכם החוצה מן הספר הזה. העולם של הבינה המלאכותית המבוזרת האוטונומית רק נפתח לפניכם; יש לכם עתה את הכלים לבנות בו, ולא רק להשתמש בו. צאו ובנו.

References

- 1 D. S. Bernstein, R. Givan, N. Immerman, and S. Zilberstein, “The complexity of decentralized control of Markov decision processes,” *Mathematics of Operations Research*, vol. 27, no. 4, 819–840, 2002. DOI: [10.1287/moor.27.4.819.297](https://doi.org/10.1287/moor.27.4.819.297)
- 2 F. A. Oliehoek and C. Amato, *A Concise Introduction to Decentralized POMDPs* (SpringerBriefs in Intelligent Systems). Springer, 2016. DOI: [10.1007/978-3-319-28929-8](https://doi.org/10.1007/978-3-319-28929-8)
- 3 L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, “Planning and acting in partially observable stochastic domains,” *Artificial Intelligence*, vol. 101, no. 1–2, 99–134, 1998. DOI: [10.1016/S0004-3702\(98\)00023-X](https://doi.org/10.1016/S0004-3702(98)00023-X)
- 4 S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. O’Reilly Media, 2021.
- 5 Anthropic. “Introducing the Model Context Protocol,” Accessed: Jul. 9, 2026. [Online]. Available: <https://www.anthropic.com/news/model-context-protocol>
- 6 Model Context Protocol. “Model context protocol specification,” Accessed: Jul. 9, 2026. [Online]. Available: <https://modelcontextprotocol.io/specification>
- 7 J. Lowin. “FastMCP: The fast, Pythonic way to build MCP servers and clients,” Accessed: Jul. 9, 2026. [Online]. Available: <https://gofastmcp.com/>

- 8 ngrok. “ngrok documentation: Secure tunnels to localhost,” Accessed: Jul. 9, 2026. [Online]. Available: <https://ngrok.com/docs>
- 9 J. Rosenberg, R. Mahy, P. Matthews, and D. Wing, “Session traversal utilities for NAT (STUN),” Internet Engineering Task Force, RFC 5389, 2008. DOI: [10.17487/RFC5389](https://doi.org/10.17487/RFC5389)
- 10 R. Nowakowski and P. Winkler, “Vertex-to-vertex pursuit in a graph,” *Discrete Mathematics*, vol. 43, no. 2–3, 235–239, 1983. DOI: [10.1016/0012-365X\(83\)90160-7](https://doi.org/10.1016/0012-365X(83)90160-7)
- 11 T. D. Parsons, “Pursuit-evasion in a graph,” *Theory and Applications of Graphs, Lecture Notes in Mathematics*, vol. 642, 426–441, 1978. DOI: [10.1007/BFb0070400](https://doi.org/10.1007/BFb0070400)
- 12 G. Theraulaz and E. Bonabeau, “A brief history of stigmergy,” *Artificial Life*, vol. 5, no. 2, 97–116, 1999. DOI: [10.1162/106454699568700](https://doi.org/10.1162/106454699568700)
- 13 E. Bonabeau, M. Dorigo, and G. Theraulaz, *Swarm Intelligence: From Natural to Artificial Systems*. Oxford University Press, 1999.
- 14 M. Dorigo, V. Maniezzo, and A. Coloni, “Ant system: Optimization by a colony of cooperating agents,” *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, vol. 26, no. 1, 29–41, 1996. DOI: [10.1109/3477.484436](https://doi.org/10.1109/3477.484436)
- 15 S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.
- 16 M. Blum, “Coin flipping by telephone: A protocol for solving impossible problems,” in *Proceedings of the 24th IEEE Computer Society International Conference (COMPCON)*, 1983, 133–137.
- 17 National Institute of Standards and Technology, “Secure hash standard (SHS),” NIST, Federal Information Processing Standards Publication FIPS PUB 180-4, 2015. DOI: [10.6028/NIST.FIPS.180-4](https://doi.org/10.6028/NIST.FIPS.180-4)
- 18 S. Goldwasser, S. Micali, and C. Rackoff, “The knowledge complexity of interactive proof systems,” *SIAM Journal on Computing*, vol. 18, no. 1, 186–208, 1989. DOI: [10.1137/0218012](https://doi.org/10.1137/0218012)

- 19 R. C. Merkle, "A digital signature based on a conventional encryption function," *Advances in Cryptology — CRYPTO '87, Lecture Notes in Computer Science*, vol. 293, 369–378, 1987. DOI: [10.1007/3-540-48184-2_32](https://doi.org/10.1007/3-540-48184-2_32)
- 20 C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, 279–292, 1992. DOI: [10.1007/BF00992698](https://doi.org/10.1007/BF00992698)
- 21 R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- 22 R. Bellman, "A Markovian decision process," *Journal of Mathematics and Mechanics*, vol. 6, no. 5, 679–684, 1957.
- 23 L. Buşoniu, R. Babuška, and B. De Schutter, "A comprehensive survey of multiagent reinforcement learning," *IEEE Transactions on Systems, Man, and Cybernetics, Part C*, vol. 38, no. 2, 156–172, 2008. DOI: [10.1109/TSMCC.2007.913919](https://doi.org/10.1109/TSMCC.2007.913919)
- 24 R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- 25 A. Vaswani et al., "Attention is all you need," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- 26 Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, 1–38, 2023. DOI: [10.1145/3571730](https://doi.org/10.1145/3571730)
- 27 M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed. Pragmatic Bookshelf, 2018.
- 28 E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- 29 Google. "Gmail API: Usage limits and sending email," Accessed: Jul. 9, 2026. [Online]. Available: <https://developers.google.com/gmail/api/reference/quota>

- 30 A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Pearson, 2011.
- 31 D. Hardt, "The OAuth 2.0 authorization framework," Internet Engineering Task Force, RFC 6749, 2012. DOI: [10.17487/RFC6749](https://doi.org/10.17487/RFC6749)

נספח א: מדריך הגדרת Gmail API

ו-OAuth 2.0

תשתית הדיווח האוטומטי של הפרויקט – שבה סוכן שולח לעצמו, למנחה או לצוות דוחות מצב בסיום כל ריצה – נשענת על יכולת שליחת דואר אלקטרוני תוכניתית דרך Gmail API. אלא שגישה מודרנית ומאובטחת אינה משתמשת בסיסמת המשתמש הרגילה: במקום זאת היא נשענת על אסימון (Token) מאובטח המונפק בתקן OAuth 2.0 [31]. תקן זה מפריד בין זהות המשתמש לבין ההרשאה שהוא מעניק לאפליקציה, ובכך מאפשר לסוכן לפעול בשמכם מבלי שסודכם האישי ייחשף אי-פעם בקוד. נספח זה מוליך אתכם, צעד אחר צעד, מהקמת הפרויקט בענן ועד לזרימת ההרשאה הראשונה שמעניקה לסוכן אוטונומיה מלאה [29].

1 חמשת שלבי ההגדרה The Five Setup Steps

התהליך המלא מורכב מחמישה שלבים סדורים. בצעו אותם לפי הסדר; דילוג על שלב (במיוחד על הגדרת מסך ההסכמה) יגרום לזרימת ההרשאה להיכשל בשלב מאוחר ומבלבל יותר.

1.1 שלב א': פתיחת פרויקט והפעלת השירות Cloud Console

היכנסו אל Google Cloud Console וצרו פרויקט חדש (או בחרו קיים). בתוך הפרויקט, גשו אל ספריית ה-API והפעילו במפורש את שירות ה-Gmail API. הפעלה זו היא שמסמנת לתשתית של Google שהפרויקט שלכם רשאי לקרוא לנקודות הקצה של הדואר.

1.2 שלב ב': הגדרת מסך ההסכמה OAuth Consent Screen

הגדירו את מסך ההסכמה (OAuth Consent Screen) – המסך שבו Google מיידעת את המשתמש אילו הרשאות האפליקציה מבקשת. בחרו במצב External (למשתמשים מחוץ לארגון) או Internal (בתוך ארגון בעל Google Workspace), והוסיפו את כתובות הדואר של הסטודנטים לקבוצת משתמשי-הבדיקה (Test Users) המורשים. בזמן שהאפליקציה במצב Testing, רק משתמשים ברשימה זו יורשו להשלים את זרימת ההרשאה.

1.3 שלב ג': צמצום ההרשאות למינימום ההכרחי Scope Restriction

הגדירו את היקף ההרשאות (Scope) למינימום ההחלטי הנדרש בלבד: `https://www.googleapis.com/auth/gmail.send`. היקף זה מתיר שליחה של דואר – ותו לא. אל תעניקו לעולם הרשאת קריאה (read) לפרויקט שאינו זקוק לה. מדובר בעיקרון אבטחת-מידע יסודי: ככל שהאסימון מסוגל לפחות, כך קטן הנזק אם ידלוף.

1.4 שלב ד': יצירת אישורי הגישה Create Credentials

בעמוד Credentials צרו OAuth Client ID מסוג Desktop Application. הורידו את הקובץ

`credentials.json` לתיקיית העבודה המקומית של הפרויקט. **חובה מוחלטת** להוסיף קובץ זה אל

`.gitignore`. לפני דחיפת קוד אל GitHub, כדי למנוע חשיפת סוד ציבורית. שכחה בשלב זה היא אחת התקלות הנפוצות והמסוכנות ביותר בפרויקטים מבוססי-ענן.

1.5 שלב ה': זרימת ההרשאה הראשונה First Authorization Flow

בהרצה הראשונה של הקוד, הספריות הרשמיות של Google יפתחו חלון דפדפן ויבקשו מכם לאשר את ההרשאה. בתום האישור ייוצר אוטומטית הקובץ `token.json`, המכיל Access Token קצר-חיים לצד Refresh Token ארוך-חיים. הודות ל-Refresh Token, הסוכן יוכל לשלוח דוחות באופן אוטונומי לחלוטין – במשך חודשים רבים וללא כל התערבות ידנית נוספת.

קריטי: לעולם אל תדחפו סודות למאגר

שני הקבצים

`credentials.json` (המזהה הסודי של האפליקציה) ו-
`token.json` (האסימונים החתומים) הם סודות. דחיפתם ל-GitHub שקולה לפרסום מפתח הכניסה לתיבת הדואר שלכם ברשות הרבים. הוסיפו את שתי השורות
`credentials.json` ו-
`token.json` אל הקובץ
`.gitignore`. לפני ה-commit הראשון. זכרו: לאחר שסוד נדחף אף ל-commit אחד בהיסטוריה, מחיקתו מהקוד הנוכחי אינה מספיקה – עליכם להחליף (rotate) את האישורים בקונסולה.

2 אנטומיה של אסימון: Access מול Refresh Token Anatomy

כדי להבין מדוע התשתית פועלת ללא סיסמאות, יש להבחין בין שני סוגי האסימונים שתקן OAuth 2.0 מגדיר [31].

Access Token מול Refresh Token

Access Token – אסימון קצר-חיים (לרוב פג בתוך כשעה) המצורף לכל בקשת API בפועל ומאשר אותה. פקיעתו המהירה מצמצמת את חלון הסיכון אם ידלוף.
Refresh Token – אסימון ארוך-חיים שאינו נשלח ל-API של הדואר עצמו, אלא משמש להשגת Access Token חדש כאשר הקודם פג. הוא זה שמעניק לסוכן את האוטונומיה ארוכת-הטווח: כל עוד ה-Refresh Token תקף, אין צורך בהתערבות אנושית חוזרת.

עקרון ההרשאה הפחותה (Least Privilege)

שימו לב שביקשנו את היקף `gmail.send` בלבד, ולא היקף רחב יותר כגון `gmail.modify` או `mail.google.com`. זהו יישום ישיר של עקרון ההרשאה הפחותה: העניקו לרכיב בדיוק את ההרשאות שהוא זקוק להן למשימתו – ולא יותר. סוכן הדיווח צריך רק לשלוח; לפיכך אין כל סיבה שיוכל לקרוא או למחוק דואר. צמצום ההיקף הופך אסימון גנוב מנשק רב-עוצמה לכלי מוגבל ובלתי-מזיק כמעט.

3 מימוש: זרימת שליחה מינימלית ב-Send-Only Flow Python

הקוד שלהלן ממחיש את הזרימה המלאה: טעינת האסימון מ-`token.json` (או יצירתו בפעם הראשונה), בניית שירות ה-Gmail, הרכבת הודעת MIME וקידודה, ולבסוף שליחתה. שימו לב שההיקף המבוקש מוגבל ל-`gmail.send` בלבד, כמתחייב מעקרון ההרשאה הפחותה.

שליחת דוח דרך Gmail API עם OAuth 2.0

```
import base64
from email.mime.text import MIMEText
from google.oauth2.credentials import Credentials
from google_auth_oauthlib.flow import InstalledAppFlow
from googleapiclient.discovery import build

# Least-privilege scope: send only, no read/modify access
SCOPES = ["https://www.googleapis.com/auth/gmail.send"]

def get_service():
    # Reuse token.json if it exists; otherwise run the
    # consent flow once
    creds = Credentials.from_authorized_user_file("token.json", SCOPES)
    return build("gmail", "v1", credentials=creds)

def send_report(service, to_addr, subject, body):
```

```

message = MIMEText(body) # build a plain-text
MIME message
message["to"] = to_addr
message["subject"] = subject
raw = base64.urlsafe_b64encode(message.as_bytes()).decode()
()
return service.users().messages().send(
    userId="me", body={"raw": raw}).execute()

if __name__ == "__main__":
    svc = get_service()
    send_report(svc, "grader@example.com", "Run report", "
Episode finished.")

```

בפועל, בהרצה הראשונה מחליפים את הקריאה הטוענת אסימון
 קיים בזרימת ההרשאה InstalledAppFlow.from_client_se-
 creds_file(...) המייצרת את
 token.json ; לאחר מכן כל ההרצות הבאות טוענות את האסימון הקיים
 ומרעננות אותו אוטומטית.

4 סיכום הקבצים Required Files

שני קבצים בלבד נדרשים לתשתית, ושניהם סודיים ושניהם חייבים להיכלל ב-
 .gitignore . הטבלה שלהלן מסכמת את תפקידם ומקורם.

טבלה 5: הקבצים הנדרשים לתשתית OAuth, מקורם ורגישותם

קובץ	מקור	תוכן	ב- ?.gitignore
credentials.json	מקור – הורדה מהקונסולה	מזהה סודי של האפליקציה	כן – חובה
token.json	נוצר – בהרצה הראשונה	אסימוני גישה וריענון	כן – חובה

נספח ב: תצורה אחידה לקובץ ההגדרות

1 מדוע חוקה משותפת? קובץ תצורה בעולם ללא שופט Why a Shared Constitution?

במשחק מבוזר מסוג P2P, שבו שני הסוכנים מתעמתים ישירות ללא שרת מרכזי המשמש שופט, נולדת שאלה יסודית: מי קובע את חוקי הפיזיקה של המשחק? כאשר קיים שרת מרכזי, הוא לבדו אוסף את גודל הגריד, את מספר המהלכים המרבי ואת קצב התפוגגות הריח, ושני השחקנים כפופים לפסיקתו. אך בהיעדר שופט, כל צד מריץ עותק משלו של לוגיקת המשחק – ואם שני העותקים אינם מסכימים על אותם ערכים בדיוק, המרוץ מתפרק לשתי מציאויות סותרות שאינן ניתנות ליישוב.

הפתרון המעשי הוא להפוך את כל הפרמטרים המרכזיים למקור אמת יחיד, קריא וגלוי, המרוכז בקובץ תצורה אחד:

`config/game.toml` . קובץ זה איננו רק אוסף של קבועים; הוא החוקה שאליה שני הצדדים מסכימים בטרם יורד המסך. כשם שאזרחים שונים כפופים לאותה חוקה כתובה, כך שני הסוכנים – שכל אחד מהם רץ על מכונה אחרת, ואולי אף נכתב בידי צוות אחר – כפופים לאותם ערכי תצורה בדיוק. כל סוכן טוען את הקובץ בעצמו ואוסף באמצעותו את אותה פיזיקה: אותו לוח, אותם גבולות, אותו קצב דעיכה. כך, אף על פי שאין ישות שלישית שתפסוק, שני הצדדים מחשבים את אותה תוצאה מתוך אותם כללים.

יתרון נוסף הוא הקריאות וההגדרות (Configurability). הפרדת הפרמטרים מן הקוד מאפשרת לשנות את תנאי הקרב – גריד קטן יותר, מגבלת זמן

נוקשה יותר, שדה ריח רחב יותר – מבלי לגעת בשורת קוד אחת של לוגיקה. הערכים המוצגים כאן הם ברירות המחזל המוסכמות של הספר, וכל אחד מהם ניתן לכיוון מחדש בכל מפגש (per-match), כל עוד שני הצדדים טוענים את אותו קובץ.

2 הקובץ הקנוני The Canonical Config File

להלן הקובץ המלא

config/game.toml על ארבעת מקטעיו: מקטע המשחק (game), מקטע הסוכנים (agents), מקטע הסביבה (environment) ומקטע הרשת (network). ההערות בקוד באנגלית, כמקובל בקובצי תצורה. הרכיב mcp_host_public_url הוא כתובת ה-URL הציבורית שדרכה נחשף שרת MCP של השוטר, כפי שנדון בהקשר של תצורת הרשת [7] וחשיפת מנהרה ציבורית באמצעות ngrok [8].

הקובץ config/game.toml

```
[game]
grid_size = 10           # board is a 10x10 grid
max_moves = 50           # hard cap on turns per match
num_games = 6            # number of rounds in a series

[agents]
cop_start = [1, 1]       # cop spawn cell (row, col)
thief_start = [5, 5]     # thief spawn cell = board center
max_barriers = 20        # total barriers a side may place

[environment]
pheromone_center_intensity = 0.9 # scent value at the source
                                cell
pheromone_decay = 0.10      # fraction lost per step
                                outward
pheromone_grid_size = 5    # scent field is a 5x5
                                window

[network]
```



```
mcp_host_public_url = "https://my-cop-agent.ngrok-free.app"
timeout_seconds_limit = 120      # per-request network
    timeout
```

3 מילון הפרמטרים Parameter Dictionary

הטבלה שלהלן מתעדת כל מפתח בקובץ: את שמו, את משמעותו היישומית ואת ערך ברירת המחדל המוסכם. שם המפתח מוצג באמצעות , ערכים מספריים באמצעות וכתובת ה-URL באמצעות . כל מפתח כאן מתאים לשם-קוד עברי שבו נעשה שימוש לאורך הספר (למשל `grid_size` ↔ [גודל הלוח]); הערכים המחייבים והרשמיים מרוכזים בטבלת הפרמטרים המחייבת שבנספח ה.

טבלה 6: מילון מלא של מפתחות
Full dictionary of config keys config/game.toml

המפתח	משמעות	ערך ברירת מחדל
grid_size	מספר התאים בכל צלע של הלוח (לוח ריבועי)	10
max_moves	תקרת המהלכים המרבית למשחק בודד	50
num_games	מספר הסיבובים בסדרת המשחקים	6
cop_start	תא הפתיחה של השוטר (row, col)	[1, 1]
thief_start	תא הפתיחה של הגנב – מרכז הלוח	[5, 5]
max_barriers	מספר החסמים המרבי שצד רשאי להציב	20
pheromone_center_intensity	עוצמת הריח בתא המקור (הגבוהה ביותר)	0.9
pheromone_decay	שיעור הדעיכה של הריח בכל צעד החוצה	0.10
pheromone_grid_size	ממדי חלון שדה הריח (חלון ריבועי)	5
mcp_host_public_url	כתובת ה-URL הציבורית של שרת ה-MCP	https://my-cop-agent.ngrok-free.app
timeout_seconds_limit	מגבלת הזמן (שניות) לבקשת רשת בודדת	120

4 היכן משמש כל פרמטר? Where Each Parameter Is Used

ערכי התצורה אינם עומדים לעצמם – כל אחד מהם מזין רכיב אחר של המערכת, ולכן חשוב לדעת לאן להביט כאשר משנים ערך:

- `max_barriers` – קובע את תקציב החסמים העומד לרשות כל צד ומזין את מכניקת הצבת המחסומים והפיכתם לכלי אסטרטגי, כפי שנדון בפרק 3.

- `pheromone_center_intensity`, `pheromone_decay`, `pheromone_grid_size` – מגדירים את המבנה, העוצמה וקצב הדעיכה של שדה עקבות הריח, שהוא ליבת מנגנון התצפית של השוטר, כמפורט בפרק 4.

- `mcp_host_public_url` – כתובת החשיפה הציבורית של שרת ה-MCP, המאפשרת לשני הסוכנים לתקשר מעבר לרשת המקומית; תצורת הרשת נדונה בפרק 2.

- `num_games`, `max_moves` – מתחמים את אורך המשחק ואת מספר הסיבובים, ולכן הם משפיעים הן על חוקי הלוח והניקוד (פרק 3) והן על ניתוח התוצאות והביצועים (פרק 9).

5 מן הדוגמה אל המימוש: סכמת התצורה בפועל - From the Example to the Reference Schema

הקובץ הקנוני שהוצג לעיל הוא דוגמה לימודית פשוטה ושטוחה. מאגר הקוד לדוגמה (נספח ד) מפצל אותו לשתי ספריות תצורה נפרדות –

1- `config/police/`

`config/thief/` – כל אחת עם קובץ

`game.toml` מקוטע למקטעים (`[board]`, `[smell]`, `[rules]`, `[positions]` וקובץ `[network]` ועוד)

`rate_limits.json` נפרד למגביל-הקצב. שמות המפתחות במימוש שונים מן הדוגמה השטוחה, וההתאמה היא:

`board.size` → `grid_size` -

`rules.max_steps` → `max_moves` -

`rules.barriers_max` → `max_barriers` -

posi- ,positions.cop_start → thief_start ,cop_start -
 tions.thief_start
 smell.emit_intensity → pheromone_center_intensity -
 smell.decay_per_step → pheromone_decay -
 smell.grid_size → pheromone_grid_size -
 network.turn_timeout_sec- → timeout_seconds_limit -
 llm.timeout_seconds (כלב-שמירה) ו-onds

num_games ו-mcp_host_public_url אינם בשימוש במימוש המקומי
 (משחקון יחיד לכל תהליך, וכתובת localhost במקום מנהור ציבורי). כלל
 "זהות הבייטים" שלהלן חל על מפתחות התנאים החתומים בלבד (לוח, ריח,
 חוקים, עמדות פתיחה, וזירה), ולא על ההגדרות הפרטיות של כל צד.

כלל ברזל: זהות בייטים

שני הצוותים חייבים לטעון ערכי תצורה זהים לחלוטין – עד כדי זהות
 בייט-אחר-בייט – מתוך
 config/game.toml. אי-התאמה ולו בערך יחיד (למשל, צד אחד עם
 grid_size = 10 וצד שני עם 12, או קצב דעיכה שונה של הריח)
 גורמת לשני הסוכנים לחשב פיזיקה שונה, וכל תוצאה שתיגזר ממנה
 חסרת תוקף. במקרה כזה המשחק בטל: אין דרך להכריע בין שתי
 מציאויות שאינן חולקות אותה חוקה. לפני כל מפגש יש לוודא שהקובץ
 המשותף זהה בשני הקצוות.

נספח ג: דרישות הגשה ב-GitHub

ודוח אקדמי

נספח זה מגדיר את תנאי הסף הפורמליים להגשת הפרויקט. חשוב להפנים כבר בפתח: ההגשה איננה קובץ מקור בודד המצורף בדוא"ל, אלא ארטיפקט פיתוחי שלם – מאגר קוד ציבורי, מתועד, ומתויג – המספר את סיפורה של המערכת שבניתם. אופן ההגשה נמדד באותה קפדנות שבה נמדד הקוד עצמו, משום שבעולם האמיתי של מערכות בינה מלאכותית מבוזרות, יכולת השחזור (Reproducibility) ושקיפות התהליך הן חלק בלתי נפרד מן התוצר.

1 מאגר ה-GitHub: מבנה, ענפים ותיג Repository, Branches and Tagging

תשתית ההגשה היא מאגר GitHub ציבורי ומאורגן היטב. הדרישה לפומביות אינה גחמה טכנית אלא עמדה מקצועית: קוד מקצועי טוב נכתב כדי להיקרא, להיבחן, ולהישחזר בידי אחרים. הפיתוח מתנהל באמצעות ענפים (Branches) – כל יכולת מהותית מתפתחת בענף ייעודי ומתמזגת אל הענף הראשי רק לאחר שהתייצבה – בהתאם לנוהגי הפיתוח של מערכות מבוזרות ומיקרו-שירותים[4].

גרסת ההגשה הסופית אינה מסומנת ב"מצב הענף האחרון" העמוס, אלא מקובעת באמצעות תג Git מתועד (Annotated Git Tag). התג מקפיד נקודת זמן ודאית ובלתי ניתנת לערעור בהיסטוריית המאגר, ומאפשר לבחון לשחזר במדויק את הקוד שהוגש – ולא גרסה מאוחרת יותר שאולי נכתבה לאחר תום המועד.

תיוג גרסת ההגשה

```
# Create an annotated, documented tag for the submission
commit.
# The -a flag makes it an annotated tag (stored as a full
object),
# and -m attaches the mandatory documentation message.
git tag -a v1.0-submission -m "Final submission: Police-Thief
P2P, group N"

# Push the tag to the public remote so the grader can check
it out.
git push origin v1.0-submission

# (Optional) verify the tag was created and points to the
right commit.
git show v1.0-submission
```

התג v1.0-submission הופך את הקומיט הנבחר לנקודת ייחוס יציבה. שימו לב שמדובר בקטע פקודות מעטפת (shell), שבו ההערות באנגלית בלבד – כמקובל בכל ההנחיות התפעוליות שבספר זה.

2 הדוח האקדמי:

The Academic README Report README.md

לב ההגשה התיעודית הוא דוח אקדמי מורחב הכתוב בקובץ README.md שבשורש המאגר. אין זה קובץ הוראות התקנה בלבד, אלא מסמך מדעי המסביר את ההחלטות התכנוניות, מנמק אותן, ומציג את הראיות האמפיריות להצלחתן. הרשימה שלהלן מפרטת את מרכיבי החובה של הדוח – היעדר כל אחד מהם גורע מן ההגשה.

תוכן החובה של הדוח האקדמי

1. **מודל ה-Dec-POMDP הנבחר.** תיאור מדעי של הפורמליזם שאימצתם למידול המרוץ – מרחב המצבים, התצפיות, ואי-הוודאות – כפי שנפרש בפרק פרק 1.
2. **דילמות התזמור של FastMCP.** דיון בהתלבטויות הפיתוח סביב תזמור התקשורת בין הסוכנים: ניהול תורות, טיפול בכשלי רשת, ותפקידי ה-Gatekeeper וה-Orchestrator. ראו פרק פרק 2 ופרק פרק 8.
3. **האסטרטגיות שמומשו.** פירוט מדויק של מנגנון קבלת ההחלטות שבחרתם – היוריסטיקות (מרחק מנהטן, מפת אמונות בייסיאנית), אסטרטגיה מבוססת מודל שפה (LLM), או, כאפשרות אופציונלית, נוסחת ה-Q-Learning (פונקציית הערך, מדיניות החקירה, קצב הלמידה) [20] – כמפורט בפרק פרק 6.
4. **עקומות למידה (אם נעשה שימוש ב-RL).** אם אימנתם סוכן בלמידת חיזוקים, יש להציג את עקומות הלמידה (Learning Curves) – תגמול מצטבר או שיעור לכידה כפונקציה של מספר הפרקים – כראיה אמפירית להתכנסות המדיניות [20].
5. **צילומי מסך – חובה מוחלטת.** יש לצרף צילומי מסך מן הסימולטור (Live GUI) המציגים את מפת האמונות (Belief Map), וכן צילום מסך מיישום השחזור (Replay App) המדגים תקינות קריפטוגרפית – כלומר את החיווי Verified OK. ראו פרק פרק 7.

הדרישה לצילומי המסך אינה פורמלית בלבד: מפת האמונות מוכיחה שהסוכן

אכן מנהל הסקה הסתברותית תחת תצפית חלקית, והחיווי Verified OK מוכיח שההגינות (Integrity) של המשחק נשמרה – ששרשרת המהלכים המוצפנת נבדקה ואומתה, בדומה למנגנוני הוכחה קריפטוגרפיים המבססים אמון ללא צורך בגורם מרכזי מהימן[18].

לעולם אין להעלות סודות למאגר הציבורי

מאחר שהמאגר ציבורי, כל קובץ שיועלה אליו גלוי לעולם כולו. חל איסור מוחלט להעלות פרטי הזדהות ואסימוני גישה – ובכללם `credentials.json` ו-`token.json` של OAuth (ראו [נספח א](#)) וכל מפתח או סוד תצורה (ראו [נספח התצורה ב](#)). חובה לכלול בשורש המאגר קובץ `.gitignore`. המחריג במפורש קבצים אלה, כך שלא ייכללו בקומיט בטעות. סוד שהודלף פעם אחת נחשב חשוף לצמיתות – גם אם יימחק בקומיט מאוחר יותר, הוא נותר בהיסטוריית ה-Git.

3 רשימת התיוג להגשה Submission Checklist

הטבלה שלהלן מרכזת את תנאי הסף. יש לוודא שכל פריט עומד בסטטוס הנדרש טרם יצירת תג ההגשה.

טבלה 7: רשימת תיוג ההגשה

פריט	סטטוס נדרש
מאגר GitHub ציבורי ומאורגן	קיים וגלוי
תג Git מתועד לגרסת ההגשה	v1.0-submission נדחף
כל חמשת מרכיבי הדוח ב- README.md	שלמים
צילומי מסך של מפת האמונות (GUI)	מצורפים
צילום מסך Replay עם Verified OK	מצורף
לפחות שני משחקים מול קבוצות שונות	2 ומעלה
דוא"ל סיום משחק – כל קבוצה בנפרד	שני הצדדים שלחו
אין סודות שהועלו למאגר () .gitignore	מאומת

זכרו את המסר החורז את הספר כולו: הפרויקט שלפניכם איננו מטלת תכנות גרידא, כי אם תרגיל בפיתוח מערכות מורכב תחת תנאי רשת אמיתיים. ההצלחה נמדדת בארבעה מדדים מרכזיים – תיאוס (Co-ordination) בין הסוכנים; הסתגלות לאי-ודאות באמצעות שובלי עקבות מבוססי סטיגמרגיה[12]; הבטחת הגינות (Integrity) בעזרת מנגנוני גיבוב מתקדמים[18]; ודבקות בארכיטקטורת קוד נכונה (תבניות Gatekeeper ו-Orchestrator)[4]. ארבעה מדדים אלה – ולא יופיו של אלגוריתם בודד – הם שיקבעו את הצלחת כל קבוצה ואת יכולתה להתמודד בעולם האמיתי של מערכות בינה מלאכותית מבוזרות. סיכום מלא של מדדים אלה מובא בפרק פרק 11.

נספח ד: מאגר הקוד לדוגמה – מימוש סימולציה בסיסי

לצד ספר החוקים וההנחיות מצורף מאגר קוד לדוגמה – מימוש בסיסי וציבורי של משחק השוטר והגנב, המשותף לסטודנטים בקורס. המאגר זמין ב-GitHub:

<https://github.com/rmisegal/Game-P2P-Cop-Chase>

נספח זה מתאר את **גרסת הקוד 1.11** של המאגר (אותה גרסה מופיעה גם בעמוד השער של הספר). קיים קישור-גרסה עמוק דו-כיווני בין הקוד לספר: גרסת הקוד נקראת מקובץ הגרסה של המאגר ומתעדכנת כאן אוטומטית בכל הידור מחדש של הספר, וגרסת הספר (1.0.21) מתעדכנת בקובץ ה-

README של המאגר בכל commit.

מה המאגר הזה – וחשוב מכך, מה הוא איננו

מאגר זה נועד ללימוד בלבד. הוא מדגים את זרימת המשחק הבסיסית ואת הממשק הגרפי הפשוט, **ללא כל אסטרטגיה** – הסוכנים נעים באופן מינימלי כדי להראות כיצד המערכת מורכבת ורצה מקצה לקצה. **אין להתחיל את הפרויקט מן המאגר הזה**, משום שהוא אינו עומד במלוא מפרט הפרויקט: הוא נכתב כדוגמה מצומצמת ולא כפתרון להגשה. מותר לכם להשתמש בחלקים מן הקוד או לשנותו, ומומלץ להיעזר בו כדי ללמוד כיצד עומש רכיב מסוים או כדי להבהיר נקודה שלא הובנה מן הספר – אך הפתרון שלכם חייב לעמוד בעצמו במלוא הדרישות (ראו טבלת הפרמטרים המחייבת שבנספח האחרון).

1 מה המאגר מדגים What the Example Shows

המאגר מריץ שני עמיתים (peers) עצמאיים – שוטר וגנב – כל אחד בתהליך נפרד, עם קובץ תצורה משלו ושרת FastMCP משלו, בדיוק כשני סטודנטים המשחקים זה מול זה משתי מכוונות. הוא מדגים: תנועה על הלוח, הצבת מחסומים, מנגנון עקבות הריח ומפת האמונות (belief), פרוטוקול Commit-Reveal מבוסס SHA-256 עם ביקורת מלאה בתום המשחק, מד צריכת טוקנים, ודוח JSON הנשלח כטיוטת Gmail. ההיגיון האסטרטגי – בחירת המהלכים החכמה – הושאר בכוונה מינימלי, והוא הליבה שאתם אמורים לפתח.

2 מבנה הקוד Code Layout

לפי קובץ ה-

README.md של המאגר, הארכיטקטורה בנויה בשכבות:

- **ממשק (Tkinter GUI/CLI)** – חלון חי ויישום שחזור (Replay).
 - **SimulationSdk** – נקודת כניסה עסקית יחידה.
 - **PeerRuntime** – עמית עצמאי אחד: משא ומתן → לולאת תורות → ביקורת.
 - **domain** – לוח, ריח, אמונה, מצב, חוקים, קריפטוגרפיה, משא ומתן, פרוטוקול ו"מוח" ההחלטה.
 - **infra** – ספק ה-Claude CLI, תעבורת ה-MCP אל שרת היריב, ושולח הדוא"ל.
 - **shared** – מנהל התצורה, מגביל-הקצב, מידע-המערכת והגרסה.
- כל קובץ פייתון קצר (עד כ-150 שורות קוד), הפיתוח מלווה בבדיקות (pytest), וכל התצורה חיצונית –

מול config/police/
בהפרדה מלאה, config/thief/

3 כיצד מריצים How to Run

הרצת שני העמיתים בשני מסופים

```
uv sync

# Terminal 1
uv run python -m police_thief peer --role police
# Terminal 2
uv run python -m police_thief peer --role thief

# Replay a saved match:
uv run python -m police_thief replay --log logs/police_match.json
```

טיפ: הפכו את המאגר ל"צ'אט-בוט" על הקוד בעזרת NotebookLM

דרך נוחה ללמוד את הקוד היא להמיר את כל קובצי המאגר לפורמט טקסט (.txt), לטעון אותם אל NotebookLM, ואז לשאול שאלות על הקוד כאילו היה לכם סוכן-שיחה ייעודי לסימולציה: "היכן מחושבת מפת האמונות?", "כיצד נאכף פרוטוקול ה-Commit-Reveal?" וכדומה. כך תוכלו להבין רכיב מסוים במהירות, מבלי לקרוא את כל המאגר ידנית.

4 תנאי שימוש How You May Use It

מותר להשתמש בחלקים מן הקוד, ללמוד ממנו ולשנותו לצורכי הפרויקט. עם זאת, זכרו שני עקרונות: (1) המאגר הוא נקודת מוצא לימודית, לא שלד הגשה – הפתרון שלכם נמדד מול מלוא המפרט; (2) רישיון המאגר הוא רישיון שימוש חינוכי (ראו קובץ

LICENSE). בכל מקום שבו המאגר סוטה מן הספר, הספר וטבלת הפרמטרים המחייבת גוברים.

נספח ה: טבלת הפרמטרים המחייבת

נספח זה הוא מקור האמת היחיד לכל ערך כמותי בפרויקט. לאורך הספר כולו, ערכים מספריים אינם מופיעים כמספר "קשיח" בגוף הטקסט, אלא כשם-קוד עברי אינטואיטיבי הכלוא בסוגריים מרובעים – למשל [גודל הלוח]. הערך בפועל נקבע אך ורק כאן, בטבלאות שלהלן.

כיצד לקרוא את הטבלה

הערכים המוצגים בעמודת "ערך לדוגמה" הם ערכי ברירת המחדל של הפרויקט וגם הפיינום המחייב: מותר להעלות אותם בהסכמה הדדית בין שתי הקבוצות המשחקות, אך **אסור** להנמיך אותם מתחת לרף זה. פרמטר המסומן "קבוע" אינו ניתן לשינוי; פרמטר המסומן "משא ומתן" נקבע כולו בשלב המשא ומתן בין הצדדים, והערך המוצג הוא דוגמה בלבד.

1 לוח וסוכנים

טבלה 8: פרמטרי הלוח ועמדות הפתיחה

שם הפרמטר	משמעות	ערך לדוגמה	סטטוס
[גודל הלוח]	צלע רשת המשחק הריבועית	7×7	מינימום
[מספר הסוכנים]	מספר השחקנים במרוץ	2	קבוע
[עמדת פתיחה - גנב]	תא הפתיחה של הגנב	מרכז (5,5)	משא ומתן
[עמדת פתיחה - שוטר]	תא הפתיחה של השוטר	פינה (1,1)	משא ומתן

2 תנועה, מחסומים וניקוד

טבלה 9: פרמטרי התנועה והניקוד

שם הפרמטר	משמעות	ערך לדוגמה	סטטוס
[מערך התנועה]	מהלך אורתוגונלי יחיד או עמידה; ללא אלכסונים	4 + עמידה	קבוע
[מכסת המחסומים]	מספר המחסומים המרבי לכל צד	20	מינימום
[תקרת הצעדים]	מספר המהלכים המרבי במשחקון	50	מינימום
[סף ההישרדות]	צעדים שעל הגנב לשרוד לזכייה	50	מינימום
[ניקוד לכידה - שוטר]	ניקוד לשוטר בלכידה מוצלחת	20	מינימום
[ניקוד לכידה - גנב]	ניקוד לגנב בלכידה	5	מינימום
[ניקוד הישרדות - שוטר]	ניקוד לשוטר בהישרדות הגנב	5	מינימום
[ניקוד הישרדות - גנב]	ניקוד לגנב בהישרדות מוצלחת	10	מינימום

3 מנגנון הריח הדינמי

טבלה 10: פרמטרי הפרומונים הדינמיים

שם הפרמטר	משמעות	ערך לדוגמה	סטטוס
[עוצמת הריח במוקד]	עוצמת הפרומון בתא הפולט	0.9	מינימום
[קצב דעיכת הריח]	שיעור הדעיכה בכל תור	0.10	מינימום
[גודל שדה הריח]	צלע חלון הפליטה סביב הסוכן	5×5	מינימום

4 רשת וליגה

טבלה 11: פרמטרי הרשת והליגה

שם הפרמטר	משמעות	ערך לדוגמה	סטטוס
[מגבלת זמן התגובה]	פסק זמן לכל בקשת רשת	120 שני'	מינימום
[סף כלב השמירה]	זמן קיפאון עד התערבות Watchdog	180 שני'	מינימום
[מספר המשחקונים]	משחקונים בסדרה מול יריבה	6	מינימום
[תגמול גיוון]	ניקוד על ניצחון מול יריבה חדשה	10	מינימום
[מינימום משחקים למעבר]	משחקים מול קבוצות שונות למעבר	2	מינימום

5 מגביל-הקצב וההגנה Rate Limiter & Gatekeeper

טבלה 12: פרמטרי מגביל-הקצב (תבנית ה-Gatekeeper)

שם הפרמטר	משמעות	ערך לדוגמה	סטטוס
[בקשות לדקה]	קצב מרבי של בקשות API יוצאות	30	מינימום
[בקשות מקבילות]	מספר הבקשות המקבילות המרבי	2	מינימום
[השהיה לאחר שגיאה]	המתנה לפני ניסיון חוזר	5 שני'	מינימום
[ניסיונות חוזרים]	מספר הניסיונות לפני כישלון	3	מינימום
[עומק התור]	גודל תור הבקשות בעת עומס	100	מינימום